

SERVICIO NACIONAL DE APRENDIZAJE
SENA

Centro de Gestión de Mercados, Logística y Tecnologías de la Información
Tecnólogo en Análisis y Desarrollo de Software

Evidencia GA10-220501097-AA10-EV01.

Elabora documentos técnicos y de usuario del software.

Alumno: Gerson Jonnathan López Oviedo

Programa: Tecnólogo en Análisis y Desarrollo de Software

Instructor: Arnaldo Montiel

Fecha

Ciudad: Bogotá D.C. 14 septiembre 2026.

Tabla de contenido:

2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.1. Prerrequisitos de instalación del sistema

Este tema se desarrolló la evidencia:

Evidencia GA10-220501097-AA2-EV01. Elaborar plan de validación de características mínimas de hardware para el software.

2.2. Frameworks y estándares con los que se construyó el sistema

Este tema se desarrolló la evidencia:

Evidencia GA7-220501096-AA1-EV01. Informe técnico de plan de trabajo para construcción de software.

2.3 Diagrama de casos de uso del sistema,

Este tema se desarrolló la evidencia:

Evidencia GA2-220501093-AA2-EV01. Elaboración de los diagramas del modelo de dominio del proyecto.

2.4 . Modelo entidad relación de la base de datos

Este tema se desarrolló las evidencias:

Evidencia GA6-220501096-AA1-EV01. Resolución de Problemas Aplicando el Modelo Relacional, Cardinalidad y Normalización

Evidencia GA6-220501096-AA1-EV02. Crear un modelo de base de datos, con base a los requerimientos del cliente.

2.5. Diccionario de datos

Este tema se desarrolló en la evidencia:

GA4-220501095-AA1-EV02. Modelo conceptual y lógico para el proyecto desarrollo de software.

2.6. Scripts de instalación

Este tema se desarrolló en las evidencias:

Evidencia GA10-220501097-AA5-EV01. Configuración de servicios, bases de datos y software en el equipo del cliente.

Evidencia GA10-220501097-AA6-EV01. Archivos cargados en la plataforma de producción.

Se adiciono llevar Trueque Match a internet.

2.6. Diagrama de componentes

Este tema se desarrolló en la evidencia:

Evidencia GA4-220501095-AA2-EV05. Desarrollar la arquitectura de software, de acuerdo con el patrón de diseño seleccionado.

2. Introducción

El desarrollo de software es muy parecido a la arquitectura e ingeniería civil: antes de que las personas puedan habitar un edificio y usar sus espacios, los ingenieros necesitan planos detallados, especificaciones de los materiales y guías de mantenimiento que garanticen la estabilidad de la estructura a lo largo del tiempo. En el ámbito informático, el Manual Técnico cumple exactamente esa función. Es el "plano estructural" que le permite a cualquier desarrollador, administrador o equipo de infraestructura comprender, desplegar y mantener el sistema sin perderse en el camino.

El presente documento constituye la Evidencia GA10-220501097-AA10-EV01: Elaboración de documentos técnicos y de usuario del software. En él se consolida toda la documentación arquitectónica del proyecto TRUEQUE MATCH, una plataforma web diseñada para facilitar el intercambio ágil de bienes y servicios. A lo largo de este manual, se detallan paso a paso los requisitos previos de instalación, los estándares de desarrollo empleados, el modelo conceptual de interacción (Casos de Uso), la estructura y diccionario de datos de la base de datos MySQL, así como los scripts de instalación y el diagrama de componentes que sostiene la solución.

2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.2. Prerrequisitos de instalación del sistema

Este tema se desarrolló la evidencia:

Evidencia GA10-220501097-AA2-EV01. Elaborar plan de validación de características mínimas de hardware para el software

Contenido

1. Introducción	6
2. Características de la aplicación a desplegar	7
3. Componentes de hardware a validar	7
4. Requisitos mínimos calculados para el servidor	8
5. Consideraciones de licenciamiento de software	9
6. Plan de validación: cómo comprobar si un equipo cumple los mínimos	10
6.1 Método manual (verificación desde el propio sistema operativo)	10
6.2 Método con herramienta de software.....	11
7. Aplicación práctica: verificación sobre un equipo real.....	11
8. Anexo: Entorno actual de desarrollo y pruebas (servidor local con XAMPP)	12
8. Conclusiones.....	14

1. Introducción

Antes de poner en marcha cualquier aplicación, no basta con que el código funcione: el equipo donde se va a ejecutar tiene que ser capaz de soportarla. En este informe explico cómo se determina si un equipo cumple o no con las características mínimas de hardware necesarias para desplegar una aplicación web, tomando como caso de estudio los requisitos de una aplicación desarrollada en PHP con acceso a información persistente de usuarios, pensada para atender alrededor de 250 usuarios activos no concurrentes.

El informe se divide en dos partes: primero defino cuáles son los componentes de hardware que hay que revisar y qué cantidad mínima necesita cada uno según las características de la aplicación, y después explico dos formas distintas de comprobar si un equipo real cumple con esos mínimos: la revisión manual desde el propio sistema operativo, y el uso de una herramienta de software especializada.

2. Características de la aplicación a desplegar

Para calcular los requisitos de hardware primero hay que tener claro qué le vamos a pedir al equipo. La aplicación de referencia tiene tres características que determinan directamente su consumo de recursos:

- Es una aplicación web desarrollada en PHP, que necesita un servidor con Apache (o Nginx) y el intérprete de PHP en ejecución constante.
- Accede a información persistente para identificar y registrar usuarios, lo que implica un motor de base de datos (MySQL/MariaDB) corriendo en paralelo al servidor web, con lectura y escritura constante.
- Está proyectada para alrededor de 250 usuarios activos no concurrentes, es decir, personas que usan la aplicación en momentos distintos del día, no todas al mismo tiempo. Esto es clave: dimensionar el servidor pensando en 250 conexiones simultáneas sería un desperdicio de recursos; lo correcto es estimar cuántos de esos 250 usuarios podrían coincidir en el pico más alto de uso.

Para ese último punto, en dimensionamiento de servidores de baja-mediana demanda se suele asumir, como criterio conservador de planeación, que entre un 10 % y un 20 % de los usuarios activos pueden coincidir en el momento de mayor tráfico. Tomando un 15 % sobre 250 usuarios, el pico esperado ronda las 35 a 40 conexiones simultáneas, una carga baja, perfectamente manejable por un servidor de gama modesta, siempre que esté bien configurado.

3. Componentes de hardware a validar

El profesor mencionó tres componentes centrales, procesador, disco duro y memoria RAM, y en este informe se complementan con dos más que también inciden directamente en que la aplicación funcione bien: la conexión de red y el sistema operativo junto con su licenciamiento. En total, los cinco elementos que se deben revisar antes de poner en marcha la aplicación son:

- Procesador (CPU): determina cuántas peticiones puede atender el servidor al mismo tiempo sin que las respuestas se vuelvan lentas.
- Memoria RAM: es el recurso más sensible en un servidor con base de datos, porque tanto Apache/PHP como MariaDB usan memoria para atender cada conexión y cachear datos.

- Almacenamiento (disco duro): debe alcanzar para el sistema operativo, el software del servidor, la base de datos y su crecimiento (incluyendo imágenes u otros archivos que suba el usuario), más espacio para copias de seguridad.
- Conexión de red: un servidor con buen procesador y RAM igual falla si el ancho de banda no soporta el tráfico de los usuarios conectados.
- Sistema operativo y licenciamiento: no es un componente físico, pero condiciona directamente el costo y las capacidades del hardware, un sistema operativo con licencia paga suma un costo fijo que hay que considerar junto con el hardware.

4. Requisitos mínimos calculados para el servidor

A partir de las tres características de la aplicación (PHP, base de datos persistente, aproximadamente 35 y 40 conexiones concurrentes en pico) y contrastando con la documentación oficial de MariaDB y recomendaciones de proveedores de hosting para aplicaciones PHP con MySQL de tráfico bajo-medio, se definieron los siguientes valores mínimos y recomendados.

Componente	Mínimo	Recomendado	Justificación
Procesador	2 núcleos, 1.8 GHz	4 núcleos, 2.0 GHz o más	Un procesador dual-core es el piso aceptado para correr Apache + PHP + MariaDB simultáneamente sin cuellos de botella.
Memoria RAM	4 GB	8 GB	Con tráfico bajo-medio se recomiendan al menos 2 GB solo para el servidor web con MySQL; sumando margen para el sistema operativo y las ~40 conexiones pico, 4 GB es el

Componente	Mínimo	Recomendado	Justificación
			piso real y 8 GB de holgura.
Almacenamiento	20 GB SSD	50 GB SSD	Cubre sistema operativo, software del servidor, base de datos, archivos subidos por los usuarios y espacio para copias de seguridad periódicas.
Red	10 Mbps simétricos	50 Mbps o más, IP fija	Necesaria para atender los picos de conexión sin demoras perceptibles y para mantener el servicio disponible de forma estable.
Sistema operativo	Linux (Ubuntu Server LTS)	Linux (Ubuntu Server LTS)	Es software libre, sin costo de licencia, y es el sistema operativo más usado para este tipo de despliegues LAMP.

4.1 Prerrequisitos de software

Además del hardware, el sistema necesita las siguientes versiones mínimas de software instaladas antes de desplegar la aplicación:

- **PHP:** versión 8.2 o superior

- **Motor de base de datos:** MariaDB 10.x (o MySQL 8.x como alternativa compatible)
- **Servidor web:** Apache 2.4 o superior
- **Para la app móvil (Android):** Android 8.0 (Oreo) o superior en el dispositivo del usuario final
- **Para compilar la app móvil:** Node.js 18 LTS o superior, y Expo CLI
- **Navegador web (cliente):** cualquier navegador moderno con soporte de JavaScript ES6 (Chrome, Firefox, Edge en sus últimas 2 versiones)

5. Consideraciones de licenciamiento de software

El licenciamiento incide directamente en cuánto hardware se puede pagar, porque cambia el presupuesto disponible. Para esta aplicación, la opción más eficiente en costos es usar exclusivamente software libre: sistema operativo Linux (Ubuntu Server), servidor web Apache, motor de base de datos MariaDB y el propio lenguaje PHP, todo bajo licencias de código abierto (GPL o similares), sin costo de licencia recurrente.

La alternativa sería usar Windows Server como sistema operativo y SQL Server como motor de base de datos, ambos con licencia comercial paga (por núcleo de procesador o por usuario, según el modelo de licenciamiento de Microsoft). Para un proyecto de este tamaño (250 usuarios activos no concurrentes), esa opción encarece significativamente el despliegue sin aportar una ventaja real sobre la opción libre, por lo que se recomienda la primera.

6. Plan de validación: cómo comprobar si un equipo cumple los mínimos

Definidos los requisitos, la otra mitad de la evidencia es explicar cómo se verifica, en la práctica, si un equipo específico los cumple. Existen dos métodos válidos y complementarios, no son excluyentes, lo ideal es usar los dos para confirmar el mismo dato desde dos fuentes distintas.

6.1 Método manual (verificación desde el propio sistema operativo)

Es el camino más directo: usar las herramientas que ya trae el sistema operativo instalado, sin instalar nada adicional.

- En Windows: clic derecho sobre "Este equipo" → "Propiedades" muestra el procesador y la RAM instalada; "Configuración" → "Sistema" → "Almacenamiento" muestra el espacio de disco usado y disponible.

- También puede ejecutarse el comando `msinfo32` (Información del sistema) desde el cuadro de "Ejecutar" (Windows + R), que muestra en una sola pantalla el procesador, la RAM total y el tipo de sistema operativo.
- En un servidor Linux (el sistema operativo recomendado para este despliegue): el comando `lscpu` muestra el procesador, `free -h` muestra la memoria RAM disponible, y `df -h` muestra el espacio de disco por partición.
- Ventaja de este método: no requiere instalar nada, es inmediato. Desventaja: hay que revisar cada dato por separado y anotarlo a mano, lo que es más lento si se necesita comparar varios equipos.

6.2 Método con herramienta de software

Es el camino que describe Juan: instalar un programa que analice el equipo automáticamente y entregue un reporte completo del hardware en una sola pantalla, sin tener que ir a buscar cada dato por separado.

- Para equipos Windows: herramientas como CPU-Z o Speccy muestran en detalle el procesador (modelo, núcleos, velocidad), la memoria RAM (cantidad, tipo, velocidad) y el almacenamiento (tipo de disco, capacidad total y libre), todo en una sola interfaz.
- Para servidores Linux: herramientas como `neofetch` o `hwinfo` entregan un resumen similar directamente en la terminal, útil para revisar rápidamente un servidor remoto por SSH.

Ventaja de este método: el reporte es más completo, más rápido de leer y más fácil de comparar entre varios equipos. Desventaja: requiere instalar un programa adicional, lo cual no siempre es posible en un entorno de producción restringido.

7. Aplicación práctica: verificación sobre un equipo real

Para cerrar el informe con un ejemplo concreto, se aplicó el método manual (opción 1) sobre el equipo de desarrollo real del proyecto, siguiendo los pasos de la sección 6.1, y se comparó contra los mínimos definidos en la sección 4.

Componente	Equipo real (Acer Aspire A515-56)	Mínimo requerido	¿Cumple?
Procesador	Intel Core i7-1165G7, 4 núcleos, 2.80 GHz	2 núcleos, 1.8 GHz	Sí
RAM	12 GB instalados (11.8 GB usable)	4 GB	Sí
Almacenamiento	SSD 477 GB (476 GB con formato)	20 GB SSD	Sí
Sistema operativo	Windows 11 Home Single Language	Linux recomendado	Parcial *

Nota: El equipo cumple sobradamente en procesador, RAM y almacenamiento para trabajar como entorno de desarrollo y pruebas (uso actual con XAMPP). Para el despliegue final en producción se recomienda migrar a un servidor con Linux, tal como se justificó en la sección 5, ya que Windows no es el sistema operativo más eficiente en costos para este tipo de despliegue, aunque sí es completamente funcional para el desarrollo.

Nota sobre el despliegue real: en la práctica, Trueque Match terminó desplegado en InfinityFree, un proveedor de hosting compartido gratuito, en vez de un servidor propio dimensionado a mano como se calculó en la sección 4. Esto significa que el proveedor de hosting ya garantiza que el hardware subyacente cumple (y de sobra) los mínimos aquí calculados el análisis de esta evidencia sigue siendo válido como ejercicio de planeación y como referencia para un eventual servidor dedicado si el proyecto creciera más allá de lo que el hosting gratuito puede sostener.

8. Anexo: Entorno actual de desarrollo y pruebas (servidor local con XAMPP)

Es importante aclarar que los requisitos mínimos calculados en la sección 4 de este informe corresponden a un **servidor de producción** pensado para atender a los 250 usuarios

activos no concurrentes en un entorno real de despliegue. El servidor donde el proyecto corre **hoy** es distinto: es un entorno local de desarrollo y pruebas montado con XAMPP sobre el mismo equipo de trabajo (Acer Aspire A515-56), usado únicamente por el desarrollador para programar y probar, no por usuarios finales.

Elemento	Configuración actual (XAMPP local)
Servidor web	Apache (incluido en XAMPP)
Lenguaje	PHP 8.2
Base de datos	MariaDB, puerto 3307
Sistema operativo	Windows 11 Home Single Language
Hardware	Intel Core i7-1165G7, 12 GB RAM, SSD 477 GB (ver sección 7)
Usuarios simultáneos reales	1 (el propio desarrollador, durante pruebas)

Esta configuración **no está pensada para escalar** a los 250 usuarios de producción, es intencionalmente un entorno de un solo puesto, y por eso mismo cumple de sobra los requisitos mínimos calculados en la sección 4 (que fueron pensados para hasta aproximadamente 40 conexiones simultáneas en el peor pico). Cuando el proyecto pase a producción real, este entorno local deja de ser el servidor final: se debe migrar a un servidor dedicado (físico, VPS o en la nube) con las características descritas en la sección 4 y, preferiblemente, sobre Linux según lo justificado en la sección 5, dado que XAMPP en sí mismo es una herramienta pensada para desarrollo local, no para servir tráfico de producción de forma estable y segura.

8. Conclusiones

- Los requisitos mínimos de hardware de una aplicación no se inventan: se derivan de sus características reales (tecnología usada, si maneja datos persistentes, y cuántos usuarios va a atender en el peor momento posible).
- Para esta aplicación, con 250 usuarios activos no concurrentes, el servidor no necesita hardware de gama alta, un equipo con 2 núcleos, 4 GB de RAM y 20 GB de almacenamiento SSD ya cubre el mínimo, gracias a que no todos los usuarios se conectan al mismo tiempo.
- Existen dos métodos válidos para verificar si un equipo cumple esos mínimos: el manual (usando las herramientas propias del sistema operativo) y el automatizado (usando un programa especializado). Ninguno reemplaza al otro; se complementan.
- El equipo de desarrollo real del proyecto (Acer Aspire A515-56) supera cómodamente los mínimos definidos, lo que confirma que es apto tanto para el desarrollo como para pruebas locales del proyecto.

2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.2. Frameworks y estándares con los que se construyó el sistema

Este tema se desarrolló la evidencia:

Evidencia GA7-220501096-AA1-EV01. Informe técnico de plan de trabajo para construcción de software.

TABLA DE CONTENIDO	15
1. Introducción.....	17
2. Objetivo	18
2.1 Objetivo General.....	18
2.2 Objetivos Específicos	18
3. Estándares de Codificación.....	19
3.1 ¿Qué son los Estándares de Codificación?	19
3.2 ¿Por qué son Necesarios los Estándares de Codificación?	19
3.2.1 Legibilidad y Comprensión del Código	20
3.2.2 Reducción de Errores	20
3.2.3 Facilita la Revisión de Código (Code Review)	20
3.2.4 Escalabilidad del Proyecto	20
3.2.5 Automatización y Calidad Continua.....	20
3.3 Prácticas Recomendadas para el Uso de Estándares de Codificación	21
3.3.1 Definir los Estándares al Inicio del Proyecto.....	22
3.3.2 Usar Herramientas de Análisis Estático (Linters).....	22
3.3.3 Configurar Formateadores Automáticos de Código	22
3.3.4 Integrar los Estándares en el Control de Versiones	23
3.3.5 Documentar el Código con JSDoc	23

3.3.6 Convenciones de Nomenclatura	23
4. Herramientas de Versionamiento.....	24
4.1 ¿Qué es el Versionamiento de Software?	24
4.2 Tipos de Sistemas de Control de Versiones.....	24
4.2.1 Sistemas Locales.....	25
4.2.2 Sistemas Centralizados (CVCS)	25
4.2.3 Sistemas Distribuidos (DVCS)	25
4.3 Herramientas Principales de Versionamiento	25
4.3.1 Git	25
4.3.2 GitHub.....	26
4.3.3 GitLab	27
4.3.4 Bitbucket.....	27
4.3.5 Subversion (SVN).....	27
4.4 Comparativa de Herramientas de Versionamiento	27
5. Selección de Herramientas para Trueque Match	28
5.1 Contexto del Proyecto.....	28
5.2 Herramientas Seleccionadas	29
5.2.1 Git como Sistema de Control de Versiones	29
5.2.2 GitHub como Plataforma de Repositorio Remoto	30
5.2.3 ESLint como Herramienta de Análisis Estático.....	30
5.2.4 Prettier como Formateador de Código	31
5.2.5 Conventional Commits como Estándar de Mensajes.....	31
5.3 Justificación Final de la Selección.....	32
6. Conclusiones.....	33
7. Referencias	¡Error! Marcador no definido.

1. Introducción

El desarrollo de software moderno exige no solo habilidades técnicas en programación, sino también la adopción de metodologías y herramientas que garanticen la calidad, trazabilidad y mantenibilidad del código fuente a lo largo del ciclo de vida de un proyecto. En este contexto, los estándares de codificación y las herramientas de versionamiento constituyen pilares fundamentales en cualquier proceso de construcción de software profesional.

El presente informe técnico corresponde a la evidencia GA7-220501096-AA1-EV01 del programa Tecnólogo en Análisis y Desarrollo de Software del SENA, y tiene como propósito investigar, analizar y seleccionar las herramientas y estándares de codificación más adecuados para el desarrollo del proyecto Trueque Match, una aplicación móvil y web orientada a facilitar el intercambio de bienes, servicios, conocimientos y experiencias entre personas sin necesidad de dinero.

A través de este documento se abordarán los conceptos fundamentales de los estándares de codificación, su importancia en proyectos de software reales, las prácticas recomendadas por la industria, las herramientas disponibles con ejemplos concretos, y finalmente la selección justificada de las herramientas que mejor se adaptan a las necesidades técnicas y contextuales del proyecto Trueque Match.

2. Objetivo

2.1 Objetivo General

Elaborar un informe técnico que identifique, analice y seleccione las herramientas y estándares de codificación más apropiados para la construcción del software Trueque Match, garantizando buenas prácticas de desarrollo, trazabilidad del código y trabajo colaborativo efectivo.

2.2 Objetivos Específicos

- Definir qué son los estándares de codificación y su rol dentro del desarrollo de software.
- Explicar por qué los estándares de codificación son necesarios en proyectos profesionales.
- Describir las prácticas recomendadas para el uso de estándares de codificación.
- Identificar y comparar las principales herramientas de versionamiento disponibles en la industria.
- Seleccionar y justificar las herramientas de versionamiento más adecuadas para el proyecto Trueque Match.

3. Estándares de Codificación

3.1 ¿Qué son los Estándares de Codificación?

Los estándares de codificación son un conjunto de reglas, convenciones y directrices acordadas que definen la forma en que el código fuente debe ser escrito dentro de un proyecto o equipo de desarrollo. Según Pressman y Maxim (2021), estos estándares abarcan aspectos como la nomenclatura de variables, la indentación, la longitud máxima de las líneas de código, la estructura de los comentarios, la organización de los archivos y la forma en que se documentan las funciones o módulos.

De manera análoga, si un equipo de construcción civil trabaja con planos y normas técnicas unificadas, un equipo de desarrollo de software trabaja con estándares de codificación para asegurar que todos hablen el mismo idioma al escribir código. Sin estos acuerdos, el proyecto se convierte en una torre de Babel donde nadie entiende el trabajo de los demás.

En el ámbito del desarrollo web y móvil con JavaScript y TypeScript, los estándares más reconocidos incluyen la guía de estilo de Airbnb JavaScript Style Guide, las convenciones de Google JavaScript Style Guide y las recomendaciones del consorcio W3C para HTML y CSS. Para proyectos con React y React Native, se siguen además las convenciones propias del ecosistema de Meta (antes Facebook), tales como el uso de componentes funcionales, hooks, y la separación clara entre lógica de negocio y presentación.

3.2 ¿Por qué son Necesarios los Estándares de Codificación?

La necesidad de contar con estándares de codificación responde a múltiples factores críticos en el desarrollo de software. Sommerville (2016) argumenta que la mantenibilidad del software depende directamente de la legibilidad y coherencia del código, características que solo se logran cuando todos los integrantes del equipo siguen las mismas convenciones.

A continuación se presentan las razones principales por las que los estándares son indispensables:

3.2.1 Legibilidad y Comprensión del Código

Cuando múltiples desarrolladores trabajan en el mismo proyecto, cada uno tiene sus propios hábitos de codificación. Sin estándares, el código de una persona puede resultar incomprensible para otra. Un estándar garantiza que cualquier miembro del equipo pueda leer, comprender y modificar cualquier parte del código sin necesidad de consultar permanentemente al autor original.

3.2.2 Reducción de Errores

Muchas convenciones de codificación nacieron precisamente para prevenir errores comunes. Por ejemplo, la regla de siempre usar llaves en los bloques if/else en JavaScript previene bugs sutiles que ocurren cuando se añade una línea sin envolver correctamente. El linter ESLint, que aplica estas reglas automáticamente, puede detectar estos errores antes de que el código llegue a producción.

3.2.3 Facilita la Revisión de Código (Code Review)

En equipos de desarrollo, es común que un desarrollador revise el código de otro antes de fusionarlo al repositorio principal. Esta práctica, conocida como code review o revisión de pares, se vuelve considerablemente más eficiente cuando el código sigue convenciones predefinidas, ya que el revisor puede concentrarse en la lógica y el funcionamiento, no en discutir si las variables deben ir en camelCase o snake_case.

3.2.4 Escalabilidad del Proyecto

A medida que un proyecto crece, la cantidad de código aumenta exponencialmente. Un proyecto que comienza con 500 líneas de código puede llegar a tener 50.000 o más en producción. Sin estándares definidos desde el inicio, mantener y escalar ese código se convierte en una tarea extraordinariamente costosa en tiempo y recursos. Los estándares permiten que nuevos desarrolladores se integren al proyecto rápidamente y contribuyan de manera productiva.

3.2.5 Automatización y Calidad Continua

Los estándares de codificación permiten la implementación de herramientas de análisis estático de código, como ESLint o SonarQube, que verifican automáticamente el cumplimiento de las reglas cada vez que se realiza un cambio en el código. Esto crea una red de seguridad

automatizada que reduce la deuda técnica y asegura niveles mínimos de calidad de manera continua.

La siguiente tabla resume las principales necesidades que atienden los estándares de codificación:

Necesidad	Problema que Resuelve	Beneficio Obtenido
Legibilidad	Código incomprensible entre miembros	Reducción del tiempo de onboarding
Consistencia	Estilos distintos en el mismo proyecto	Código uniforme y profesional
Detección de errores	Bugs por malas prácticas	Menor deuda técnica
Mantenibilidad	Dificultad para modificar código antiguo	Cambios más rápidos y seguros
Colaboración	Conflictos de estilo en revisiones	Code reviews más eficientes
Automatización	Revisión manual ineficiente	CI/CD con control de calidad automático

3.3 Prácticas Recomendadas para el Uso de Estándares de Codificación

La adopción de estándares de codificación no consiste únicamente en elegir un conjunto de reglas y publicarlas en un documento. Para que sean efectivos, deben implementarse de manera sistemática a lo largo de todo el ciclo de vida del proyecto. Las siguientes son las prácticas más recomendadas por la industria:

3.3.1 Definir los Estándares al Inicio del Proyecto

Uno de los errores más comunes es intentar implementar estándares cuando el proyecto ya lleva semanas o meses de desarrollo. Refactorizar código existente para que cumpla con las nuevas reglas es costoso y riesgoso. La práctica recomendada es definir los estándares antes de escribir la primera línea de código funcional.

3.3.2 Usar Herramientas de Análisis Estático (Linters)

Un linter es una herramienta que analiza el código fuente en busca de errores, violaciones de estilo y patrones problemáticos sin necesidad de ejecutar el programa. Para JavaScript y TypeScript, ESLint es el estándar de facto. Se configura mediante un archivo `.eslintrc.json` o `.eslintrc.js` en la raíz del proyecto y puede integrarse directamente con el editor de código Visual Studio Code para mostrar advertencias en tiempo real mientras se escribe.

Ejemplo de configuración básica de ESLint para un proyecto React Native:

```
// Archivo: .eslintrc.json

{
  "extends": ["eslint:recommended", "plugin:react/recommended"],
  "rules": {
    "no-unused-vars": "warn",
    "eqeqeq": "error",
    "semi": ["error", "always"],
    "quotes": ["error", "single"]
  }
}
```

3.3.3 Configurar Formateadores Automáticos de Código

Prettier es un formateador de código que aplica automáticamente un estilo consistente al guardar un archivo. A diferencia de ESLint, que detecta problemas de lógica y estilo, Prettier se enfoca exclusivamente en el formato visual del código: indentación, longitud de líneas, uso de

comillas, espaciado, entre otros. La combinación de ESLint y Prettier es una práctica estándar en proyectos modernos de JavaScript.

3.3.4 Integrar los Estándares en el Control de Versiones

Una práctica avanzada pero muy recomendada es la integración de los estándares con el sistema de control de versiones. Herramientas como Husky permiten ejecutar el linter automáticamente antes de cada commit, de tal manera que si el código no cumple con los estándares, el commit es rechazado. Esto garantiza que el repositorio siempre contenga código que respeta las convenciones definidas.

3.3.5 Documentar el Código con JSDoc

Los comentarios de documentación siguen un formato estándar que permite generar documentación técnica automáticamente. En JavaScript, el estándar JSDoc define cómo deben estructurarse los comentarios de funciones, parámetros y valores de retorno. Esta práctica es especialmente importante en proyectos que serán mantenidos por múltiples personas o que tienen ciclos de vida largos.

```
/**
 * Autentica al usuario con correo y contraseña
 *
 * @param {string} correo - Dirección de correo electrónico
 *
 * @param {string} password - Contraseña del usuario
 *
 * @returns {Promise<string>} Token JWT de la sesión
 */
```

3.3.6 Convenciones de Nomenclatura

Una de las áreas más importantes de los estándares de codificación es la nomenclatura. Las convenciones más utilizadas en JavaScript y TypeScript son:

Tipo de Identificador	Convención	Ejemplo
Variables y funciones	camelCase	nombreUsuario, calcularTotal()
Componentes React	PascalCase	PantallaLogin, TarjetaOferta
Constantes globales	UPPER_SNAKE_CASE	MAX_INTENTOS, URL_BASE
Archivos de componentes	PascalCase.tsx	PantallaHome.tsx
Archivos de utilidades	camelCase.ts	validarEmail.ts
Interfaces TypeScript	PascalCase con I-	IUsuario, IOferta
Tipos TypeScript	PascalCase	TipoCategoria, EstadoTrueque

4. Herramientas de Versionamiento

4.1 ¿Qué es el Versionamiento de Software?

El versionamiento o control de versiones es la práctica de registrar y gestionar los cambios realizados al código fuente de un software a lo largo del tiempo. Un sistema de control de versiones (VCS, por sus siglas en inglés) permite a los desarrolladores guardar el historial completo de modificaciones, revertir cambios, trabajar en paralelo en diferentes funcionalidades y fusionar el trabajo de múltiples colaboradores sin perder información.

Chacon y Straub (2014) definen el control de versiones como un sistema que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, de tal manera que sea posible recuperar versiones específicas en cualquier momento. Esta capacidad es especialmente crítica en proyectos de software donde un error introducido en una nueva versión puede afectar a todos los usuarios del sistema.

4.2 Tipos de Sistemas de Control de Versiones

4.2.1 Sistemas Locales

Los sistemas de control de versiones más simples funcionan localmente en el equipo del desarrollador. Un ejemplo histórico es RCS (Revision Control System), que guarda parches de las diferencias entre versiones en formato especial. Aunque funcional, este enfoque presenta el grave problema de no permitir la colaboración entre múltiples desarrolladores.

4.2.2 Sistemas Centralizados (CVCS)

Los sistemas de control de versiones centralizados, como Subversion (SVN) y CVS, resuelven el problema de la colaboración mediante un servidor central que almacena todas las versiones del proyecto. Los desarrolladores realizan checkout del código, trabajan localmente y luego hacen commit al servidor central. El mayor riesgo de este modelo es el servidor único: si falla, todo el equipo queda bloqueado y si se corrompe sin respaldo, se pierde toda la historia del proyecto.

4.2.3 Sistemas Distribuidos (DVCS)

Los sistemas distribuidos, como Git y Mercurial, superan las limitaciones de los sistemas centralizados. En un DVCS cada desarrollador tiene una copia completa del repositorio, incluida toda la historia del proyecto. Esto significa que casi todas las operaciones son locales y extremadamente rápidas, y que el proyecto puede continuar aunque el servidor central falle temporalmente.

4.3 Herramientas Principales de Versionamiento

4.3.1 Git

Git es el sistema de control de versiones distribuido más utilizado en el mundo. Fue creado en 2005 por Linus Torvalds, el mismo creador del kernel de Linux, con el objetivo de gestionar el desarrollo colaborativo de ese sistema operativo. Según la encuesta anual de Stack Overflow 2023, el 93.87% de los desarrolladores profesionales utiliza Git como su herramienta principal de control de versiones.

Git introduce el concepto de área de preparación (staging area o índice), que permite a los desarrolladores preparar exactamente qué cambios van a incluir en el próximo commit, ofreciendo un control muy granular sobre la historia del proyecto.

Comandos fundamentales de Git:

Comando	Función
git init	Inicializa un nuevo repositorio Git en el directorio actual
git clone <url>	Clona un repositorio remoto en el equipo local
git add <archivo>	Agrega archivos al área de preparación (staging)
git commit -m 'mensaje'	Guarda los cambios en el historial con un mensaje descriptivo
git push origin main	Sube los commits locales al repositorio remoto
git pull	Descarga y fusiona los cambios del repositorio remoto
git branch <nombre>	Crea una nueva rama de trabajo
git checkout <rama>	Cambia a una rama existente
git merge <rama>	Fusiona una rama con la rama actual
git log --oneline	Muestra el historial de commits de forma resumida

4.3.2 GitHub

GitHub es la plataforma de alojamiento de repositorios Git más popular del mundo, con más de 100 millones de desarrolladores y 420 millones de repositorios según datos de la propia plataforma (GitHub, 2024). GitHub no es simplemente un servidor para guardar código: es una plataforma completa de colaboración que incluye control de versiones, revisión de código mediante pull requests, gestión de issues, automatización con GitHub Actions, wikis de documentación y tableros de proyectos.

Una de las características más poderosas de GitHub para equipos de desarrollo es el flujo de trabajo basado en ramas (branching strategy). El modelo más utilizado es GitHub Flow, que

propone mantener siempre una rama principal estable (main o master) y crear ramas separadas para cada nueva funcionalidad o corrección de error, las cuales se fusionan mediante pull requests después de ser revisadas por otros miembros del equipo.

4.3.3 GitLab

GitLab es una plataforma DevOps completa de código abierto que ofrece funcionalidades similares a GitHub, pero con la ventaja de poder instalarse en servidores propios (self-hosted). GitLab se distingue por integrar en una sola plataforma todas las etapas del ciclo DevOps: desde la planificación y el control de versiones, hasta la integración continua, el despliegue continuo y el monitoreo. Es ampliamente utilizado en empresas que requieren mayor control sobre sus datos y su infraestructura.

4.3.4 Bitbucket

Bitbucket es la plataforma de alojamiento de repositorios Git de Atlassian, la empresa detrás de Jira y Confluence. Su principal ventaja competitiva radica en su integración nativa con las herramientas del ecosistema Atlassian, lo que la hace especialmente atractiva para equipos que ya usan Jira para la gestión de proyectos y Confluence para la documentación técnica. Bitbucket ofrece repositorios privados ilimitados en su plan gratuito.

4.3.5 Subversion (SVN)

Apache Subversion, comúnmente conocido como SVN, es un sistema de control de versiones centralizado que fue el estándar de la industria antes de la masificación de Git. A diferencia de Git, en SVN el historial completo del proyecto reside en el servidor central y los desarrolladores trabajan con copias de trabajo locales. Aunque ha perdido terreno frente a Git, SVN sigue siendo utilizado en algunas organizaciones con proyectos legacy o con requisitos estrictos de control centralizado.

4.4 Comparativa de Herramientas de Versionamiento

La siguiente tabla presenta una comparativa objetiva de las principales herramientas de versionamiento analizadas, considerando los criterios más relevantes para un proyecto de desarrollo de software:

Criterio	Git	GitHub	GitLab	Bitbucket	SVN
Tipo	DVCS local	Plataforma Git	Plataforma DevOps	Plataforma Git	CVCS
Modelo	Distribuido	Distribuido	Distribuido	Distribuido	Centralizado
Costo base	Gratuito	Gratuito/Pro	Gratuito/Premium	Gratuito/Premium	Gratuito
CI/CD integrado	No nativo	GitHub Actions	GitLab CI/CD	Pipelines	No nativo
Trabajo offline	Sí	Sí	Sí	Sí	Limitado
Pull Requests	No aplica	Sí	Merge Requests	Sí	No nativo
Gestión de issues	No	Sí	Sí	Sí	No
Self-hosted	Sí (local)	GitHub Enterprise	Sí (gratuito)	Datacenter	Sí
Curva aprendizaje	Media-Alta	Baja-Media	Media	Baja-Media	Baja
Popularidad 2024	93.87%	#1 plataforma	#2 plataforma	#3 plataforma	< 5%

5. Selección de Herramientas para Trueque Match

5.1 Contexto del Proyecto

Trueque Match es una aplicación móvil y web desarrollada con React Native + Expo para el cliente móvil Android, React.js para el cliente web, y Node.js + Express para el backend. El proyecto es desarrollado por un estudiante de manera individual en el marco del programa Tecnólogo en Análisis y Desarrollo de Software del SENA, utilizando un computador portátil ACER Core 7 con Windows y un dispositivo de prueba Redmi 9 para la aplicación móvil.

Dado este contexto, la selección de herramientas debe considerar los siguientes criterios de prioridad:

- Gratuidad: el proyecto no cuenta con presupuesto para licencias de software.
- Facilidad de uso: el desarrollador está en proceso de formación y necesita herramientas con curva de aprendizaje razonable.
- Compatibilidad: las herramientas deben funcionar correctamente en el ecosistema Windows + JavaScript/TypeScript + React Native.
- Integración: las herramientas deben poder conectarse entre sí para crear un flujo de trabajo cohesionado.
- Adopción en la industria: priorizar herramientas ampliamente utilizadas en el mercado laboral colombiano e internacional.

5.2 Herramientas Seleccionadas

5.2.1 Git como Sistema de Control de Versiones

Git es la selección natural e indiscutible como sistema de control de versiones para Trueque Match. Su naturaleza distribuida permite trabajar con total funcionalidad incluso sin conexión a internet, lo cual es relevante dado que el estudiante trabaja desde casa con acceso a internet que puede ser intermitente. Además, Git es la base tecnológica sobre la que se construyen las demás plataformas consideradas.

La adopción de Git en Trueque Match permitirá implementar una estrategia de ramas clara: una rama main con el código estable, una rama develop para integración de nuevas funcionalidades, y ramas feature/* para el desarrollo de cada pantalla o módulo específico. Por ejemplo: feature/pantalla-login, feature/pantalla-home, feature/modelo-bd.

```
# Flujo de trabajo Git para Trueque Match

git checkout -b feature/pantalla-home: nueva función

git add . : preparar cambios

git commit -m "feat: agrega pantalla Home con categorías"

git push origin feature/pantalla-home: subir al remoto

git checkout develop && git merge feature/pantalla-home
```

5.2.2 GitHub como Plataforma de Repositorio Remoto

GitHub es la plataforma seleccionada para alojar el repositorio remoto de Trueque Match. Las razones que justifican esta elección por encima de alternativas como GitLab o Bitbucket son:

- La cuenta del estudiante (gilo) ya está configurada en la plataforma Expo EAS, lo cual facilita la integración con el proceso de compilación del APK.
- GitHub ofrece repositorios privados ilimitados en su plan gratuito, lo que permite proteger el código del proyecto durante su desarrollo.
- GitHub Actions, el sistema de integración continua de la plataforma, permite automatizar en el futuro la ejecución del linter y las pruebas cada vez que se realice un push al repositorio.
- GitHub es la plataforma más demandada en el mercado laboral, por lo que su dominio constituye una competencia profesional directamente aplicable.
- El ecosistema de GitHub cuenta con integraciones nativas con las principales herramientas del stack tecnológico de Trueque Match, incluyendo Expo, VS Code y Node.js.

5.2.3 ESLint como Herramienta de Análisis Estático

Para garantizar el cumplimiento de los estándares de codificación en el código JavaScript y TypeScript del proyecto, se selecciona ESLint como herramienta de análisis estático. ESLint se configura con el conjunto de reglas recomendado para React Native y TypeScript, y se integra con

la extensión ESLint de Visual Studio Code para proporcionar retroalimentación visual inmediata mientras se escribe el código.

5.2.4 Prettier como Formateador de Código

Prettier complementa a ESLint encargándose del formato visual del código. Su configuración para Trueque Match incluirá: uso de comillas simples, punto y coma al final de las instrucciones, indentación de 2 espacios, y longitud máxima de línea de 100 caracteres. Prettier se configura para ejecutarse automáticamente al guardar un archivo en VS Code, eliminando la necesidad de formatear el código manualmente.

5.2.5 Conventional Commits como Estándar de Mensajes

Para mantener un historial de cambios legible y estructurado, Trueque Match adoptará la especificación Conventional Commits para los mensajes de los commits. Este estándar define un formato claro: tipo(alcance): descripción breve. Los tipos más utilizados incluyen feat para nuevas funcionalidades, fix para correcciones de errores, docs para cambios en documentación, style para cambios de formato, y refactor para reestructuración del código sin cambios funcionales.

Tipo de Commit	Uso	Ejemplo Real en Trueque Match
Feat	Nueva funcionalidad	feat(auth): implementa login con JWT
Fix	Corrección de error	fix(chat): corrige badge de mensajes no leídos
Docs	Documentación	docs: agrega JSDoc a funciones de API
Style	Formato de código	style: aplica prettier a pantalla Home
Refactor	Reestructuración	refactor(db): normaliza tabla TRUEQUE a 3FN
Test	Pruebas	test: agrega test unitario para validarEmail()
Chore	Tareas de mantenimiento	chore: actualiza dependencias de Expo SDK

5.3 Justificación Final de la Selección

La combinación Git + GitHub + ESLint + Prettier + Conventional Commits representa el stack de herramientas de versionamiento y estándares de codificación más apropiado para Trueque Match por las siguientes razones finales:

En primer lugar, esta combinación de herramientas es completamente gratuita en su plan básico, lo que la hace accesible para un proyecto académico sin presupuesto. En segundo lugar, es el stack más utilizado en la industria del desarrollo de software con JavaScript y React Native en Colombia y a nivel internacional, lo que significa que las competencias adquiridas son directamente aplicables en el mercado laboral. En tercer lugar, todas las herramientas se integran de manera nativa entre sí: ESLint y Prettier se integran con VS Code, VS Code tiene integración nativa con Git y GitHub, y Expo EAS se conecta directamente con repositorios de GitHub para ejecutar las compilaciones del APK.

Finalmente, este conjunto de herramientas garantiza que el código de Trueque Match sea legible, consistente y mantenible a lo largo de todo el proceso de desarrollo, lo que resulta en un producto de mayor calidad y en un portafolio más sólido para el estudiante.

Herramienta	Rol en el Proyecto	Justificación
Git	VCS distribuido local	Gratuito, estándar industria, trabajo offline
GitHub	Repositorio remoto y CI/CD	Integración con Expo, plan gratuito, portafolio
ESLint	Análisis estático de código	Detecta errores en JS/TS automáticamente
Prettier	Formateador automático	Garantiza consistencia visual del código
Conventional Commits	Estándar de mensajes Git	Historial legible y semántico

6. Conclusiones

Los estándares de codificación y las herramientas de versionamiento no son elementos opcionales en el desarrollo de software profesional, sino componentes fundamentales que determinan la calidad, mantenibilidad y escalabilidad de cualquier proyecto. A través de este informe técnico se ha evidenciado que su adopción desde las etapas iniciales del desarrollo impacta directamente en la eficiencia del trabajo y en la reducción de la deuda técnica.

Para el proyecto Trueque Match, la selección de Git como sistema de control de versiones distribuido, GitHub como plataforma de alojamiento y colaboración, ESLint y Prettier como herramientas de calidad de código, y la especificación Conventional Commits como estándar de mensajes, representa la decisión técnica más fundamentada considerando el contexto académico, las tecnologías utilizadas y los requerimientos del proyecto.

La implementación de estas herramientas no solo beneficia el desarrollo actual de Trueque Match, sino que también dota al estudiante de competencias directamente aplicables en el entorno profesional del desarrollo de software en Colombia y a nivel internacional, donde estas herramientas son consideradas conocimiento fundamental en cualquier equipo de desarrollo moderno.

2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.6 Diagrama de casos de uso del sistema,

Este tema se desarrolló la evidencia:

Evidencia GA2-220501093-AA2-EV01. Elaboración de los diagramas del modelo de dominio del proyecto.

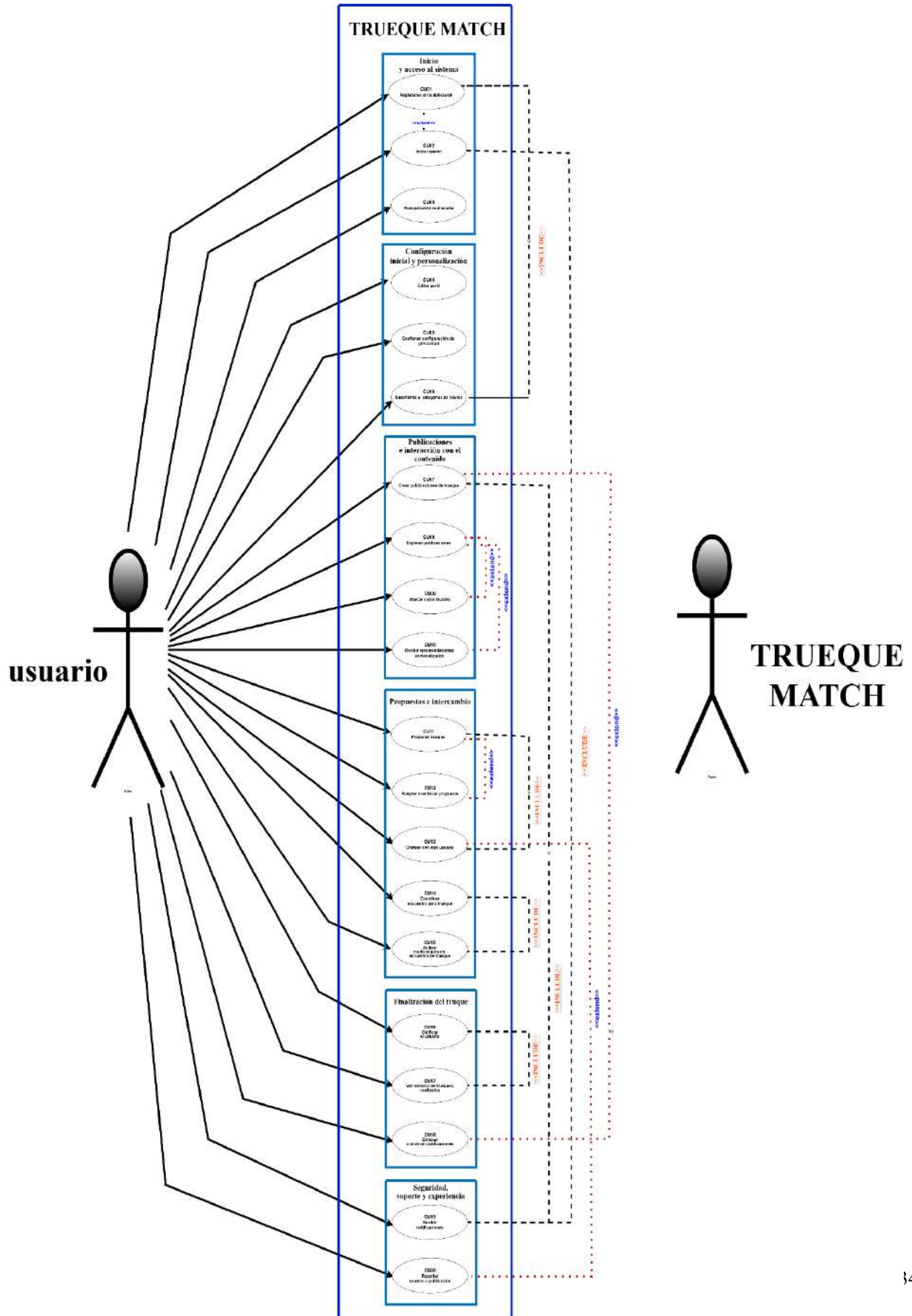
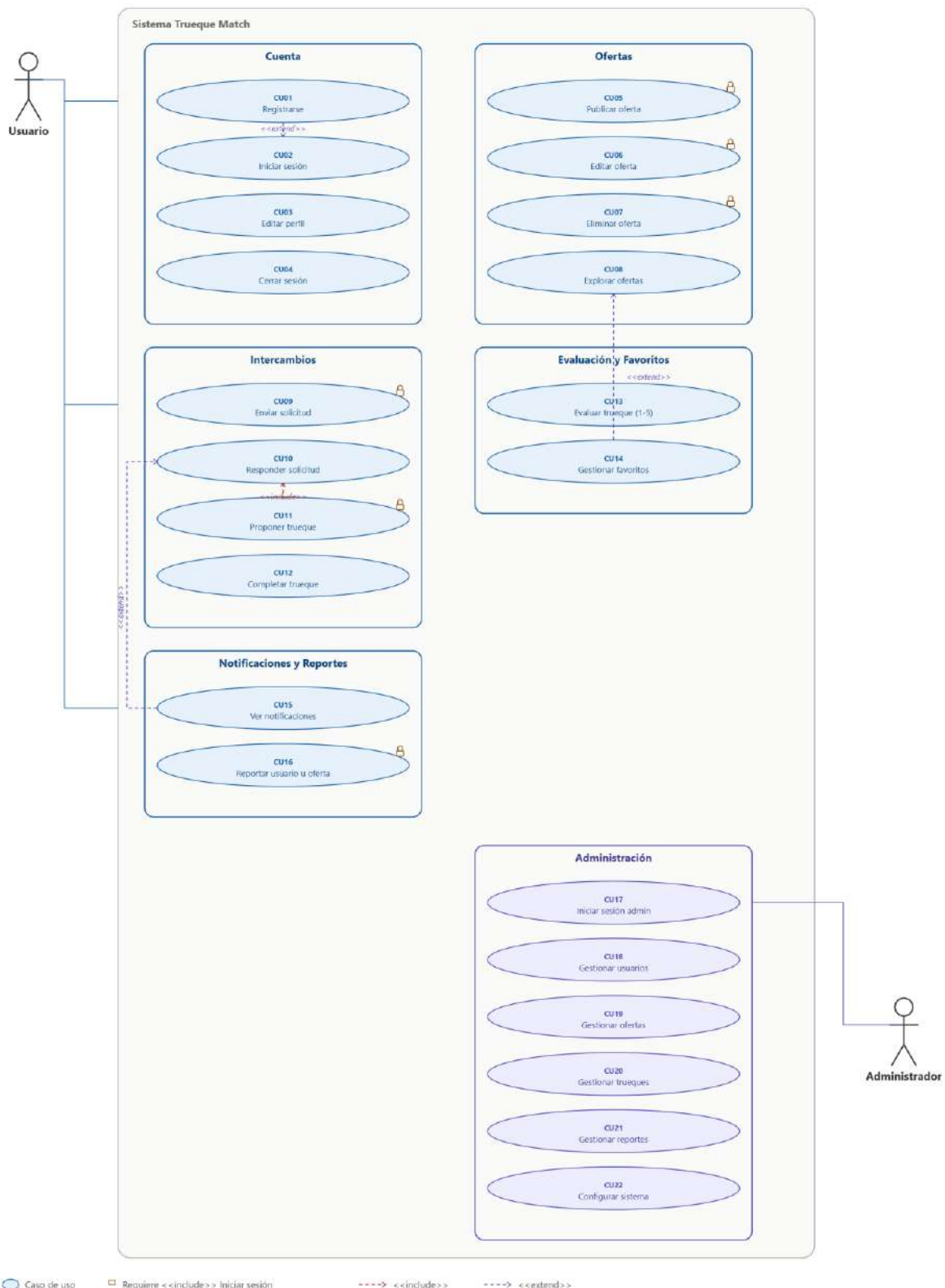


Diagrama de Casos de Uso — Trueque Match

Con relaciones <<include>> y <<extend>> basadas en el sistema real



2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.7 . Modelo entidad relación de la base de datos

Este tema se desarrolló las evidencias:

Evidencia GA6-220501096-AA1-EV01. Resolución de Problemas Aplicando el Modelo Relacional, Cardinalidad y Normalización

Evidencia GA6-220501096-AA1-EV02. Crear un modelo de base de datos, con base a los requerimientos del cliente.

Contenido

Contenido	5
Introducción.....	6
Características de la aplicación a desplegar.....	7
Componentes de hardware a validar	7
Requisitos mínimos calculados para el servidor	8
Consideraciones de licenciamiento de software	9
Plan de validación: cómo comprobar si un equipo cumple los mínimos	10
6.1 Método manual (verificación desde el propio sistema operativo)	10
6.2 Método con herramienta de software.....	11
7. Aplicación práctica: verificación sobre un equipo real.....	11
8. Anexo: Entorno actual de desarrollo y pruebas (servidor local con XAMPP)	12
8. Conclusiones.....	14
TABLA DE CONTENIDO	15
1. Introducción.....	17
2. Objetivo	18
2.1 Objetivo General.....	18
2.2 Objetivos Específicos	18
3. Estándares de Codificación.....	19
3.1 ¿Qué son los Estándares de Codificación?	19

3.2 ¿Por qué son Necesarios los Estándares de Codificación?	19
3.2.1 Legibilidad y Comprensión del Código	20
3.2.2 Reducción de Errores	20
3.2.3 Facilita la Revisión de Código (Code Review)	20
3.2.4 Escalabilidad del Proyecto	20
3.2.5 Automatización y Calidad Continua	20
3.3 Prácticas Recomendadas para el Uso de Estándares de Codificación	21
3.3.1 Definir los Estándares al Inicio del Proyecto	22
3.3.2 Usar Herramientas de Análisis Estático (Linters)	22
3.3.3 Configurar Formateadores Automáticos de Código	22
3.3.4 Integrar los Estándares en el Control de Versiones	23
3.3.5 Documentar el Código con JSDoc	23
3.3.6 Convenciones de Nomenclatura	23
4. Herramientas de Versionamiento.....	24
4.1 ¿Qué es el Versionamiento de Software?	24
4.2 Tipos de Sistemas de Control de Versiones.....	24
4.2.1 Sistemas Locales	25
4.2.2 Sistemas Centralizados (CVCS)	25
4.2.3 Sistemas Distribuidos (DVCS)	25
4.3 Herramientas Principales de Versionamiento	25
4.3.1 Git	25
4.3.2 GitHub	26
4.3.5 Subversion (SVN)	27
4.4 Comparativa de Herramientas de Versionamiento	27
5. Selección de Herramientas para Trueque Match	28
5.1 Contexto del Proyecto.....	28

5.2 Herramientas Seleccionadas	29
5.2.1 Git como Sistema de Control de Versiones.....	29
5.2.2 GitHub como Plataforma de Repositorio Remoto	30
5.2.3 ESLint como Herramienta de Análisis Estático.....	30
5.2.4 Prettier como Formateador de Código	31
5.2.5 Conventional Commits como Estándar de Mensajes	31
5.3 Justificación Final de la Selección	32
6. Conclusiones.....	33
1. Introducción.....	40
2. Objetivo	41
2.1 Objetivo General.....	41
2.2 Objetivos Específicos	41
3. Marco Teórico	42
3.1 Modelo Relacional.....	42
3.2 Conceptos Fundamentales	42
3.3 Cardinalidad de las Relaciones	43
3.4 Normalización	43
4. Modelo Relacional Trueque Match	43
4.1 Convenciones del Modelo	43
4.2 Tablas del Modelo	44
4.2.1 Tabla TIPO_USUARIO.....	44
4.2.2 Tabla USUARIO.....	45
4.2.3 Tabla ADMINISTRADOR.....	46
4.2.4 Tabla OFERTA.....	47
4.2.5 Tabla TRUEQUE.....	49
4.2.6 Tabla EVALUACION	50
4.2.7 Tabla SOLICITUD	52

4.2.8 Tabla NOTIFICACION	53
4.2.9 Tabla REPORTE.....	54
4.2.10 Tabla USUARIO_OFERTA (Favoritos)	56
5. Cardinalidad de las Relaciones	57
6. Aplicación de la Normalización	58
6.2 Segunda Forma Normal (2FN)	59
6.3 Tercera Forma Normal (3FN).....	59
6.4 Resumen de Normalización por Tabla.....	60
7. Modelo relacional Trueque Match.....	21
8. Link modelo entidad relación de caso en MySQL Workbench.....	22
9. Conclusiones.....	25
10. Referencias.....	26

1. Introducción

El presente documento corresponde a la evidencia GA6-220501096-AA1-EV01 del programa Tecnólogo en Análisis y Desarrollo de Software del SENA. Su propósito es presentar el modelo relacional de la base de datos del proyecto Trueque Match, una aplicación móvil diseñada para facilitar el intercambio de productos, servicios, conocimientos y experiencias entre personas sin necesidad de dinero, promoviendo la economía colaborativa y solidaria en Colombia.

El modelo relacional es la representación formal de la estructura de datos de un sistema de información, en la cual los datos se organizan en tablas o relaciones compuestas por filas (tuplas) y columnas (atributos). Este modelo, propuesto por Edgar F. Codd en 1970, es el fundamento teórico de los sistemas de gestión de bases de datos relacionales como MySQL, PostgreSQL y SQL Server (Codd, 1970).

El documento incluye el análisis completo del modelo relacional de Trueque Match, con la definición de las diez tablas del sistema, sus atributos, tipos de datos, llaves primarias y foráneas, la cardinalidad de las relaciones entre entidades y la aplicación de las tres primeras formas normales (1FN, 2FN y 3FN) para garantizar la integridad, consistencia y eficiencia en el almacenamiento de la información.

2. Objetivo

2.1 Objetivo General

Diseñar el modelo relacional de la base de datos de la aplicación Trueque Match, aplicando los principios de cardinalidad y las reglas de normalización (1FN, 2FN y 3FN), con el fin de garantizar la integridad, consistencia y eficiencia en el almacenamiento y gestión de los datos del sistema.

2.2 Objetivos Específicos

- Identificar las entidades principales del sistema Trueque Match y definir sus atributos con sus respectivos tipos de datos.
- Establecer las llaves primarias (PK) y foráneas (FK) de cada tabla del modelo relacional.
- Determinar la cardinalidad de las relaciones entre las entidades del sistema.
- Aplicar las tres formas normales (1FN, 2FN y 3FN) para eliminar redundancias y dependencias no deseadas en el modelo.
- Documentar el modelo relacional mediante tablas detalladas con ejemplos de tuplas que muestren la estructura completa de cada entidad.

3. Marco Teórico

3.1 Modelo Relacional

El modelo relacional es una forma de estructurar y organizar datos en tablas bidimensionales llamadas relaciones. Cada tabla representa una entidad del mundo real y está compuesta por filas (tuplas o registros) y columnas (atributos o campos). El modelo fue propuesto por Edgar F. Codd en 1970 y es la base de la mayoría de los sistemas de gestión de bases de datos actuales (Date, 2004).

3.2 Conceptos Fundamentales

A continuación se presentan los conceptos clave del modelo relacional aplicados al diseño de Trueque Match:

Tabla A

Conceptos fundamentales del modelo relacional

Concepto	Definición
Relación / Tabla	Conjunto de datos organizados en filas y columnas que representa una entidad del sistema.
Tupla / Fila	Registro individual dentro de una tabla que representa una instancia de la entidad.
Atributo / Columna	Característica o propiedad de una entidad que almacena un tipo específico de dato.
Llave Primaria (PK)	Atributo o conjunto de atributos que identifica de forma única cada tupla en una tabla.
Llave Foránea (FK)	Atributo que referencia la llave primaria de otra tabla, estableciendo la relación entre entidades.
Dominio	Conjunto de valores permitidos para un atributo (tipo de dato, rango, restricciones).
Integridad referencial	Regla que garantiza que los valores de una FK existan como PK en la tabla referenciada.
Cardinalidad	Número de instancias de una entidad que se relacionan con instancias de otra entidad.

3.3 Cardinalidad de las Relaciones

La cardinalidad define cuántas instancias de una entidad pueden relacionarse con instancias de otra. Los tipos de cardinalidad son:

- Uno a Uno (1:1): Una instancia de la entidad A se relaciona con exactamente una instancia de la entidad B.
- Uno a Muchos (1:N): Una instancia de la entidad A se relaciona con una o más instancias de la entidad B.
- Muchos a Muchos (N:M): Una o más instancias de la entidad A se relacionan con una o más instancias de la entidad B. Se resuelve mediante una tabla intermedia.

3.4 Normalización

La normalización es el proceso de organizar los atributos de una base de datos para reducir la redundancia de datos y mejorar la integridad. Se aplica mediante formas normales progresivas:

- Primera Forma Normal (1FN): Todos los atributos contienen valores atómicos (indivisibles), sin grupos repetidos ni atributos multivaluados.
- Segunda Forma Normal (2FN): Cumple con 1FN y todos los atributos no clave dependen completamente de la llave primaria (sin dependencias parciales).
- Tercera Forma Normal (3FN): Cumple con 2FN y no existen dependencias transitivas entre atributos no clave.

4. Modelo Relacional Trueque Match

El modelo relacional de Trueque Match está compuesto por 10 tablas que representan las entidades principales del sistema. A continuación se presenta el modelo completo con todos sus atributos, tipos de datos, llaves y restricciones, seguido de un ejemplo de tuplas (filas con datos reales) para cada tabla.

4.1 Convenciones del Modelo

Tabla B

Convenciones de color del modelo relacional

Símbolo	Color	Significado
PK	Rojo	Llave Primaria: Identifica únicamente cada registro de la tabla
FK	Azul	Llave Foránea: Referencia a la PK de otra tabla del modelo
PK+FK	Verde	Llave Primaria y Foránea simultáneamente (tabla intermedia N:M)

4.2 Tablas del Modelo

4.2.1 Tabla TIPO_USUARIO

Tabla 1

Tabla 1: TIPO_USUARIO

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_tipo_usuario	INT	20	PK	NO	Identificador único del tipo de usuario	AUTO_INCREMENT
nombre_tipo	VARCHAR	50	-	NO	Nombre del tipo: 'estandar' o 'administrador'	UNIQUE
descripcion	TEXT	-	-	SÍ	Descripción del rol y permisos del tipo	-
fecha_creacion	DATETIME	-	-	NO	Fecha y hora de creación del registro	CURRENT_TIMESTAMP

Nota. Almacena los tipos de usuario del sistema (estándar y administrador). Permite escalar roles sin modificar la estructura de USUARIO.

Tabla 1.1

Ejemplo de tuplas: Tabla 1

id_tipo_usuario	nombre_tipo	descripcion	fecha_creacion
1	estandar	Usuario que publica y propone trueques	2024-08-01 08:00:00
2	administrador	Usuario con acceso al panel de control	2024-08-01 08:00:00

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.2 Tabla USUARIO

Tabla 2

Tabla 2: USUARIO Usuarios registrados en la plataforma

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_usuario	INT	11	PK	NO	Identificador único del usuario	AUTO_INCREMENT
nombre	VARCHAR	100	-	NO	Nombre completo del usuario	-
correo	VARCHAR	150	-	NO	Correo electrónico del usuario	UNIQUE
clave_acceso	VARCHAR	255	-	NO	Contraseña cifrada con bcrypt	-
teléfono	VARCHAR	15	-	SÍ	Número de teléfono de contacto	-
ciudad	VARCHAR	100	-	SÍ	Ciudad de residencia del usuario	-
foto_perfil	VARCHAR	255	-	SÍ	URL de la foto de perfil	-
reputacion	DECIMAL	3,2	-	NO	Puntuación promedio 0.00 a 5.00	DEFAULT 0.00
fecha_registro	DATETIME	-	-	NO	Fecha y hora de registro	DEFAULT NOW()
activo	TINYINT	1	-	NO	1 = activo, 0 = inactivo	DEFAULT 1
id_tipo_usuario	INT	11	FK	NO	Referencia al tipo de usuario	REF: TIPO_USUARIO

Nota. Entidad central del sistema. Todos los intercambios, ofertas, evaluaciones y notificaciones están ligados a un usuario registrado.

Tabla 2.1

Ejemplo de tuplas Tabla 2

id_usuario	nombre	correo	clave_acceso	ciudad	reputacion	activo	id_tipo_usuario
1	Laura Gómez	laura@correo.com	\$2b\$10\$xH...	Bogotá	4.80	1	1
2	Carlos Rueda	carlos@correo.com	\$2b\$10\$aK...	Medellín	4.60	1	1
3	Sofía Martínez	sofia@correo.com	\$2b\$10\$pL...	Cali	4.90	1	1
4	Jhon Pérez	jhon@correo.com	\$2b\$10\$mQ...	Barranquilla	3.50	1	1
5	Arnaldo Montiel	arnaldo@trueque.com	\$2b\$10\$adm...	Bogotá	5.00	1	2

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.3 Tabla ADMINISTRADOR

Tabla 3

Tabla 3 ADMINISTRADOR Gestores del sistema

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_admin	INT	11	PK	NO	Identificador único del administrador	AUTO_INCREMENT
nombre	VARCHAR	100	-	NO	Nombre completo del administrador	-
correo	VARCHAR	150	-	NO	Correo electrónico del administrador	UNIQUE

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
clave_acceso	VARCHAR	255	-	NO	Contraseña cifrada con bcrypt	-
nivel_acceso	VARCHAR	50	-	NO	'super_admin' o 'moderador'	DEFAULT 'moderador'
fecha_registro	DATETIME	-	-	NO	Fecha y hora de registro	DEFAULT NOW()
activo	TINYINT	1	-	NO	1 = activo, 0 = inactivo	DEFAULT 1

Nota. Contiene los datos específicos de los usuarios con rol administrativo. Se relaciona con TIPO_USUARIO pero mantiene atributos propios del rol.

Tabla 3.1

Ejemplo de tuplas Tabla 3

id_admin	nombre	correo	nivel_acceso	fecha_registro	Activo
1	Arnaldo Montiel	arnaldo@trueque.com	super_admin	2024-08-01 08:00:00	1
2	Sandra López	sandra@trueque.com	moderador	2024-08-15 09:00:00	1
3	Miguel Herrera	miguel@trueque.com	moderador	2025-01-10 10:00:00	1

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.4 Tabla OFERTA

Tabla 4

Tabla 4 OFERTA Publicaciones de intercambio

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_oferta	INT	11	PK	NO	Identificador único de la oferta	AUTO_INCREMENT

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
titulo	VARCHAR	200	-	NO	Título descriptivo de la oferta	-
descripcion	TEXT	-	-	NO	Descripción detallada de lo ofrecido	-
categoría	ENUM	-	-	NO	'producto','servicio','conocimiento','experiencia'	-
Estado	ENUM	-	-	NO	'activa','inactiva','intercambiada'	DEFAULT 'activa'
imagen_url	VARCHAR	255	-	SÍ	URL de la imagen representativa	-
ciudad	VARCHAR	100	-	SÍ	Ciudad donde se realiza el intercambio	-
valor_estimado	DECIMAL	10,2	-	SÍ	Valor estimado en COP (referencia)	-
fecha_publicacion	DATETIME	-	-	NO	Fecha y hora de publicación	DEFAULT NOW()
id_usuario	INT	11	FK	NO	Usuario propietario de la oferta	REF: USUARIO

Nota. Registra cada artículo, servicio, conocimiento o experiencia publicada. Usa ENUM para garantizar valores atómicos en categoría y estado, cumpliendo 1FN.

Tabla 4.1

Ejemplo de tuplas Tabla 4

id_oferta	titulo	categoría	estado	ciudad	valor_estimado	fecha_publicacion	id_usuario
1	Bicicleta MTB talla M	producto	activa	Bogotá	350000	2025-02-01 10:00:00	1
2	Clases de inglés 2h/semana	conocimiento	activa	Medellín	80000	2025-02-10 11:00:00	2

id_oferta	titulo	categoria	estado	ciudad	valor_estimado	fecha_publicacion	id_usuario
3	Teclado mecánico RGB	producto	intercambiada	Cali	220000	2025-02-15 14:00:00	3
4	Tour fotográfico urbano 3h	experiencia	activa	Bogotá	150000	2025-03-05 09:00:00	1
5	Guitarra acústica Yamaha	producto	activa	Barranquilla	400000	2025-03-20 16:00:00	4

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.5 Tabla TRUEQUE

Tabla 5

Tabla 5: TRUEQUE Intercambios formalizados

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_trueque	INT	11	PK	NO	Identificador único del trueque	AUTO_INCREMENT
<u>estado</u>	ENUM	-	-	NO	'pendiente','aceptado','completado','cancelado'	DEFAULT 'pendiente'
fecha_propuesta	DATETIME	-	-	NO	Fecha en que se propuso el trueque	DEFAULT NOW()
fecha_trueque	DATETIME	-	-	SÍ	Fecha acordada para el intercambio físico	-
descripcion_acuerdo	TEXT	-	-	SÍ	Descripción del acuerdo entre las partes	-
id_usuario_propone	INT	11	FK	NO	Usuario que propone el trueque	REF: USUARIO

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_oferta_propone	INT	11	FK	NO	Oferta del usuario proponente	REF: OFERTA
id_usuario_recibe	INT	11	FK	NO	Usuario que recibe la propuesta	REF: USUARIO
id_oferta_recibe	INT	11	FK	NO	Oferta del usuario receptor	REF: OFERTA

Nota. Representa el intercambio acordado entre dos usuarios con sus dos ofertas. Es la entidad central del negocio. Cuatro FK garantizan integridad referencial completa.

Tabla 5.1

Ejemplo de tuplas Tabla 5

id_trueque	estado	fecha_propuesta	fecha_trueque	id_u_propone	id_o_propone	id_u_recibe	id_o_recibe
1	completado	2025-02-20 09:00:00	2025-02-25 10:00:00	1	1	2	2
2	pendiente	2025-03-10 14:00:00	NULL	3	3	1	4
3	aceptado	2025-03-18 11:00:00	2025-03-25 15:00:00	4	5	3	3
4	cancelado	2025-03-22 16:00:00	NULL	2	2	5	1
5	pendiente	2025-04-01 08:00:00	NULL	1	4	4	5

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.6 Tabla EVALUACION

Tabla 6

Tabla 6: EVALUACION Calificaciones post-intercambio

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_evaluacion	INT	11	PK	NO	Identificador único de la evaluación	AUTO_INCREMENT
Puntaje	INT	1	-	NO	Calificación de 1 (pésimo) a 5 (excelente)	CHECK 1-5
comentario	TEXT	-	-	SÍ	Comentario escrito sobre la experiencia	-
fecha_evaluacion	DATETIME	-	-	NO	Fecha y hora de la evaluación	DEFAULT NOW()
id_usuario	INT	11	FK	NO	Usuario que realiza la evaluación	REF: USUARIO
id_trueque	INT	11	FK	NO	Trueque que se está evaluando	REF: TRUEQUE

Nota. Permite calificar la experiencia tras un trueque completado. Genera la reputación visible en el perfil. El puntaje CHECK(1-5) garantiza valores atómicos válidos.

Tabla 6.1

Ejemplo de tuplas: Tabla 6

id_evaluacion	puntaje	comentario	fecha_evaluacion	id_usuario	id_trueque
1	5	Excelente persona, muy puntual y cumplida	2025-02-26 12:00:00	1	1
2	5	Bici en perfecto estado, tal como la describió	2025-02-26 13:00:00	2	1
3	4	Buena comunicación, artículo en buen estado	2025-03-26 10:00:00	4	3
4	3	Tardó en responder, pero cumplió al final	2025-03-26 11:00:00	3	3
5	5	Experiencia increíble, repetiría sin dudar	2025-04-10 09:00:00	3	5

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.7 Tabla SOLICITUD

Tabla 7

Tabla 7 SOLICITUD Solicitudes de contacto previas al trueque

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_solicitud	INT	11	PK	NO	Identificador único de la solicitud	AUTO_INCREMENT
mensaje	TEXT	-	-	SÍ	Mensaje inicial del solicitante	-
estado	ENUM	-	-	NO	'pendiente','aceptada','rechazada'	DEFAULT 'pendiente'
fecha_solicitud	DATETIME	-	-	NO	Fecha y hora de la solicitud	DEFAULT NOW()
fecha_respuesta	DATETIME	-	-	SÍ	Fecha en que se respondió la solicitud	-
id_usuario_solicita	INT	11	FK	NO	Usuario que envía la solicitud	REF: USUARIO
id_usuario_recibe	INT	11	FK	NO	Usuario que recibe la solicitud	REF: USUARIO
id_oferta	INT	11	FK	NO	Oferta sobre la que se hace la solicitud	REF: OFERTA

Nota. Registra el interés de un usuario en una oferta antes de formalizar el trueque. Referencia dos instancias de USUARIO (quien solicita y quien recibe).

Tabla 7.1

Ejemplo de tuplas Tabla 7

id_solicitud	Mensaje	estado	fecha_solicitud	id_u_solicita	id_u_recibe	id_oferta
1	Hola, me interesa tu bici. Tengo clases de inglés para ofrecer.	aceptada	2025-02-18 09:30:00	2	1	1
2	¿Aceptas mi tour fotográfico a cambio del teclado?	pendiente	2025-03-09 16:00:00	1	3	3
3	Puedo ofrecerte mi teclado mecánico por la guitarra.	aceptada	2025-03-19 11:00:00	3	4	5
4	No tengo artículos físicos, solo servicios de diseño.	rechazada	2025-03-21 14:00:00	5	2	2
5	Me encantaría el tour, ofrezco la guitarra Yamaha.	pendiente	2025-04-01 08:30:00	4	1	4

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.8 Tabla NOTIFICACION

Tabla 8

Tabla 8: NOTIFICACION - Alertas del sistema

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_notificacion	INT	11	PK	NO	Identificador único de la notificación	AUTO_INCREMENT
mensaje	TEXT	-	-	NO	Contenido del mensaje de la notificación	-
tipo	ENUM	-	-	NO	'info','alerta','sistema','trueque'	DEFAULT 'info'
leida	TINYINT	1	-	NO	1 = leída, 0 = no leída	DEFAULT 0

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
fecha_envio	DATETIME	-	-	NO	Fecha y hora de envío	DEFAULT NOW()
id_usuario	INT	11	FK	NO	Usuario destinatario de la notificación	REF: USUARIO
id_trueque	INT	11	FK	SÍ	Trueque relacionado (si aplica)	REF: TRUEQUE
id_oferta	INT	11	FK	SÍ	Oferta relacionada (si aplica)	REF: OFERTA

Nota. Gestiona todas las alertas enviadas a los usuarios. Las FK a TRUEQUE y OFERTA son nullable (SÍ en Nulo) porque no toda notificación está ligada a ambas entidades.

Tabla 8.1

Ejemplo de tuplas: Tabla 8

id_notif.	mensaje	tipo	leida	fecha_envio	id_usuario	id_trueque	id_oferta
1	Carlos quiere intercambiar tu bicicleta	alerta	1	2025-02-18 09:31:00	1	NULL	1
2	Tu trueque #1 fue completado exitosamente	trueque	1	2025-02-25 14:00:00	1	1	NULL
3	Recibiste una evaluación de 5 estrellas	info	1	2025-02-26 12:01:00	2	1	NULL
4	Sofia propone intercambio por tu teclado	alerta	0	2025-03-09 16:01:00	1	NULL	3
5	Tu oferta ha sido guardada como favorita	info	0	2025-04-01 08:00:00	4	NULL	4

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.9 Tabla REPORTE

Tabla 9

Tabla 9 REPORTE Denuncias de usuarios

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_reporte	INT	11	PK	NO	Identificador único del reporte	AUTO_INCREMENT
motivo	ENUM	-	-	NO	'fraude','incumplimiento','comportamiento','otro'	-
descripcion	TEXT	-	-	NO	Descripción detallada del incidente	-
estado	ENUM	-	-	NO	'pendiente','revisado','resuelto'	DEFAULT 'pendiente'
fecha_reporte	DATETIME	-	-	NO	Fecha y hora del reporte	DEFAULT NOW()
fecha_resolucion	DATETIME	-	-	SÍ	Fecha en que se resolvió el reporte	-
id_usuario_reporta	INT	11	FK	NO	Usuario que genera el reporte	REF: USUARIO
id_usuario_reportado	INT	11	FK	SÍ	Usuario que es reportado	REF: USUARIO
id_oferta	INT	11	FK	SÍ	Oferta relacionada (si aplica)	REF: OFERTA
id_trueque	INT	11	FK	SÍ	Trueque relacionado (si aplica)	REF: TRUEQUE

Nota. Registra las denuncias sobre comportamientos inapropiados, fraudes o incumplimientos. Incluye id_usuario_reportado para identificar al infractor, campo ausente en versiones anteriores del modelo.

Tabla 9.1

Ejemplo de tuplas: Tabla 9

id_reporte	motivo	estado	fecha_reporte	id_u_reporta	id_u_reportado	id_oferta	id_trueque
1	incumplimiento	resuelto	2025-02-27 10:00:00	1	2	NULL	1

id_reporte	motivo	estado	fecha_reporte	id_u_reporta	id_u_reportado	id_oferta	id_trueque
2	fraude	pendiente	2025-03-12 11:00:00	3	NULL	3	NULL
3	comportamiento	pendiente	2025-03-20 14:00:00	2	4	NULL	3
4	incumplimiento	rechazado	2025-03-24 09:00:00	4	NULL	NULL	4
5	fraude	pendiente	2025-04-02 08:00:00	5	NULL	5	NULL

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

4.2.10 Tabla USUARIO_OFERTA (Favoritos)

Tabla 10

Tabla 10: USUARIO_OFERTA - Favoritos (tabla intermedia N:M)

Atributo / Campo	Tipo de Dato	Long.	Clave	Nulo	Descripción	Restricción
id_usuario	INT	11	PK+FK	NO	Usuario que guarda el favorito	REF: USUARIO
id_oferta	INT	11	PK+FK	NO	Oferta guardada como favorito	REF: OFERTA
fecha_guardado	DATETIME	-	-	NO	Fecha y hora en que se guardó	DEFAULT NOW()

Nota. Resuelve la relación muchos a muchos entre USUARIO y OFERTA para la funcionalidad de favoritos. La PK compuesta (id_usuario + id_oferta) garantiza que un usuario no guarde la misma oferta dos veces.

Tabla 10.1

Ejemplo de tuplas: Tabla 10

id_usuario	id_oferta	fecha_guardado
1	2	2025-02-19 10:00:00
1	5	2025-03-21 14:00:00
2	4	2025-03-06 09:00:00
3	1	2025-02-16 11:00:00
4	2	2025-02-11 16:00:00

Nota. PK = clave primaria (rojo). FK = clave foránea (azul). Los datos son ficticios con fines ilustrativos.

5. Cardinalidad de las Relaciones

A continuación se describen todas las relaciones entre las tablas del modelo relacional de Trueque Match, indicando la cardinalidad de cada una y su descripción de negocio:

Tabla 11

Cardinalidad de las relaciones del modelo Trueque Match

Tabla A	Cardinalidad	Tabla B	Descripción de la relación
TIPO_USUARIO	1 : N	USUARIO	Un tipo de usuario clasifica a muchos usuarios del sistema
USUARIO	1 : N	OFERTA	Un usuario puede publicar muchas ofertas de intercambio
USUARIO	1 : N	TRUEQUE (propone)	Un usuario puede proponer muchos trueques
USUARIO	1 : N	TRUEQUE (recibe)	Un usuario puede recibir muchas propuestas de trueque
OFERTA	1 : N	TRUEQUE	Una oferta puede estar involucrada en muchos trueques
USUARIO	1 : N	EVALUACION	Un usuario puede realizar muchas evaluaciones
TRUEQUE	1 : N	EVALUACION	Un trueque puede recibir evaluaciones de ambas partes

Tabla A	Cardinalidad	Tabla B	Descripción de la relación
USUARIO	1 : N	SOLICITUD	Un usuario puede enviar y recibir muchas solicitudes
OFERTA	1 : N	SOLICITUD	Una oferta puede recibir muchas solicitudes de usuarios
USUARIO	1 : N	NOTIFICACION	Un usuario puede recibir muchas notificaciones del sistema
USUARIO	1 : N	REPORTE	Un usuario puede generar y recibir muchos reportes
USUARIO	N : M	OFERTA (favoritos)	Un usuario guarda muchos favoritos; una oferta es guardada por muchos usuarios. Se resuelve con USUARIO_OFERTA

Nota. La relación N:M entre USUARIO y OFERTA (favoritos) se resuelve mediante la tabla intermedia USUARIO_OFERTA.

6. Aplicación de la Normalización

La normalización es el proceso formal de organizar las tablas de una base de datos para minimizar la redundancia de datos, eliminar anomalías de inserción, actualización y eliminación, y garantizar la integridad estructural del sistema. Para Trueque Match se aplicaron las tres primeras formas normales (1FN, 2FN y 3FN) en el diseño de las diez tablas del modelo.

6.1 Primera Forma Normal (1FN)

Para que una tabla cumpla con la Primera Forma Normal debe garantizar que todos los atributos contengan valores atómicos (no divisibles), que no existan grupos repetidos de atributos y que cada registro sea único identificado por una llave primaria (Date, 2004).

Aplicación en Trueque Match:

- Todas las tablas tienen una llave primaria definida (id_usuario, id_oferta, id_trueque, etc.) que identifica de forma única cada registro del sistema.
- Ningún atributo contiene valores múltiples. Por ejemplo, un usuario no tiene un campo 'teléfonos' con varios números separados por comas; si necesitara múltiples teléfonos, se crearía una tabla USUARIO_TELEFONO separada.

- La relación N:M entre USUARIO y OFERTA (favoritos) se resolvió mediante la tabla intermedia USUARIO_OFERTA, eliminando grupos repetidos y cumpliendo atomicidad.
- Los atributos 'categoría' en OFERTA, 'estado' en TRUEQUE y 'tipo' en NOTIFICACION usan ENUM con valores atómicos predefinidos, garantizando dominios controlados y sin ambigüedades.

6.2 Segunda Forma Normal (2FN)

Para que una tabla cumpla con la Segunda Forma Normal debe cumplir con 1FN y todos los atributos no clave deben depender completamente de la llave primaria, eliminando las dependencias parciales (Silberschatz et al., 2019).

Aplicación en Trueque Match:

- En la tabla OFERTA, todos los atributos (titulo, descripción, categoría, estado, imagen_url, ciudad, valor_estimado) dependen completamente de id_oferta y no de una parte de la clave.
- En la tabla TRUEQUE, todos los atributos (estado, fecha_propuesta, fecha_trueque, descripcion_acuerdo) dependen completamente de id_trueque.
- En la tabla USUARIO_OFERTA, la llave primaria compuesta (id_usuario, id_oferta) garantiza que el atributo fecha_guardado depende de la combinación completa de las dos llaves y no de una sola por separado.
- En la tabla EVALUACION, el puntaje y el comentario dependen del id_evaluación y no de id_usuario o id_trueque de forma individual.

6.3 Tercera Forma Normal (3FN)

Para que una tabla cumpla con la Tercera Forma Normal debe cumplir con 2FN y no deben existir dependencias transitivas, es decir, ningún atributo no clave debe depender de otro atributo no clave (Elmasri y Navathe, 2016).

Aplicación en Trueque Match:

- El tipo de usuario se separó en la tabla TIPO_USUARIO para evitar que atributos como 'descripcion' del tipo dependieran transitivamente de id_usuario a través de nombre_tipo. Esta decisión elimina la dependencia $A \rightarrow B \rightarrow C$.
- En la tabla USUARIO, la ciudad es un atributo directo del usuario y no depende de otro atributo no clave. Si se requiriera información adicional sobre la ciudad (departamento, código postal), se crearía una tabla CIUDAD separada.
- En la tabla TRUEQUE, la información de los usuarios y ofertas involucrados se referencia mediante FK y no se repiten atributos de USUARIO u OFERTA dentro de TRUEQUE, eliminando redundancia transitiva.
- En la tabla REPORTE, el estado del reporte depende directamente de id_reporte y no existe dependencia transitiva con otros campos como motivo o descripción.

6.4 Resumen de Normalización por Tabla

Tabla 12

Aplicación de las formas normales en las tablas de Trueque Match

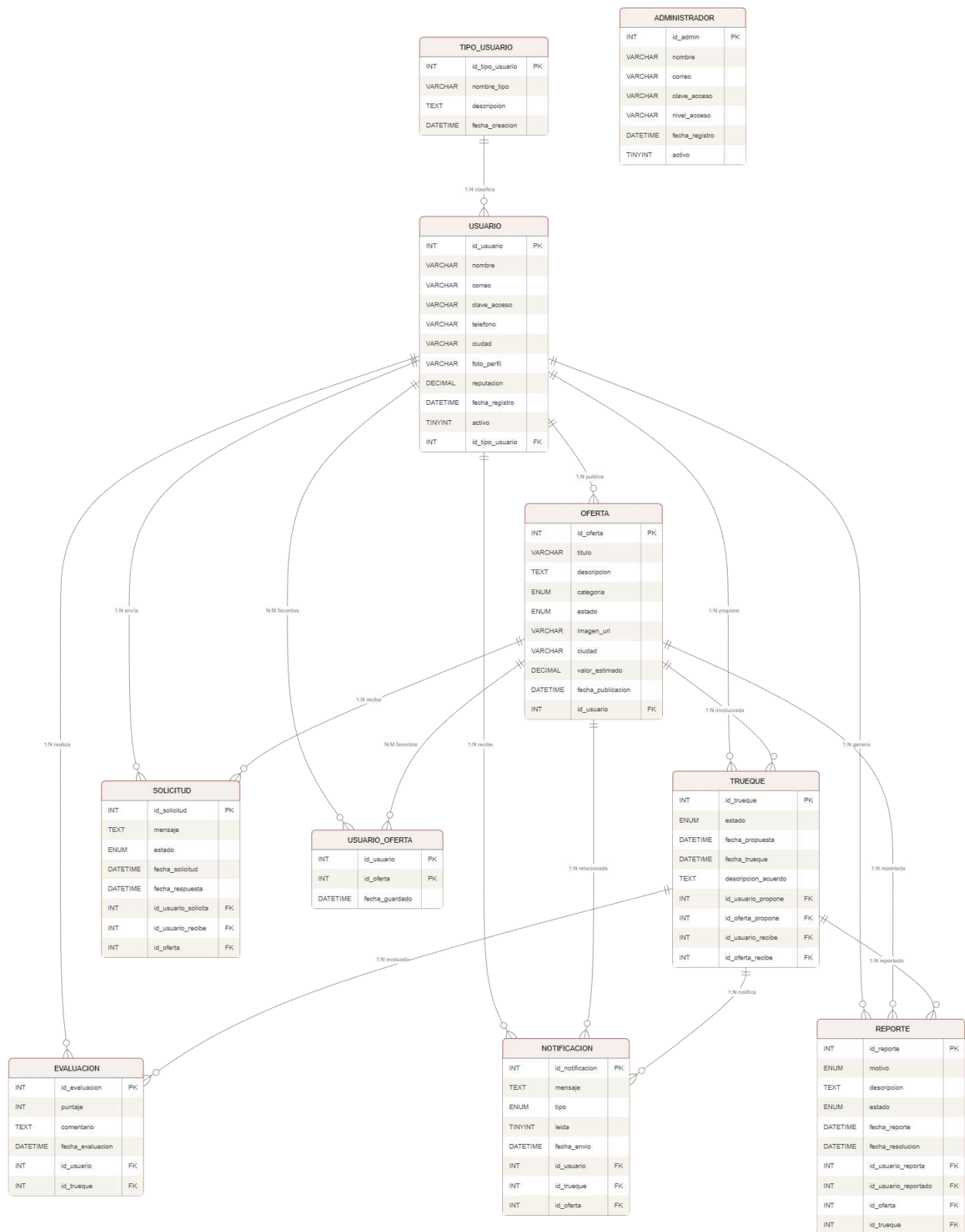
Tabla	1FN	2FN	3FN	Observación
TIPO_USUARIO	✓	✓	✓	Tabla simple. Separar tipo evita dependencia transitiva en USUARIO.
USUARIO	✓	✓	✓	PK simple. Todos los atributos dependen únicamente de id_usuario.
ADMINISTRADOR	✓	✓	✓	PK simple. Separado de USUARIO por roles y atributos distintos.
OFERTA	✓	✓	✓	FK a USUARIO. ENUM atómico en categoría y estado garantiza 1FN.
TRUEQUE	✓	✓	✓	4 FK para los dos usuarios y las dos ofertas involucradas.
EVALUACION	✓	✓	✓	FK a USUARIO y TRUEQUE. Sin dependencias transitivas entre campos.

Tabla	1FN	2FN	3FN	Observación
SOLICITUD	✓	✓	✓	FK a dos instancias de USUARIO y a OFERTA. Sin redundancias.
NOTIFICACION	✓	✓	✓	FK nullable a TRUEQUE y OFERTA según contexto del evento.
REPORTE	✓	✓	✓	FK nullable según tipo de reporte. id_usuario_reportado separado.
USUARIO_OFERTA	✓	✓	✓	PK compuesta elimina duplicados. fecha_guardado depende del par completo.

Nota. ✓ = cumple la forma normal. Elaboración propia (2026).

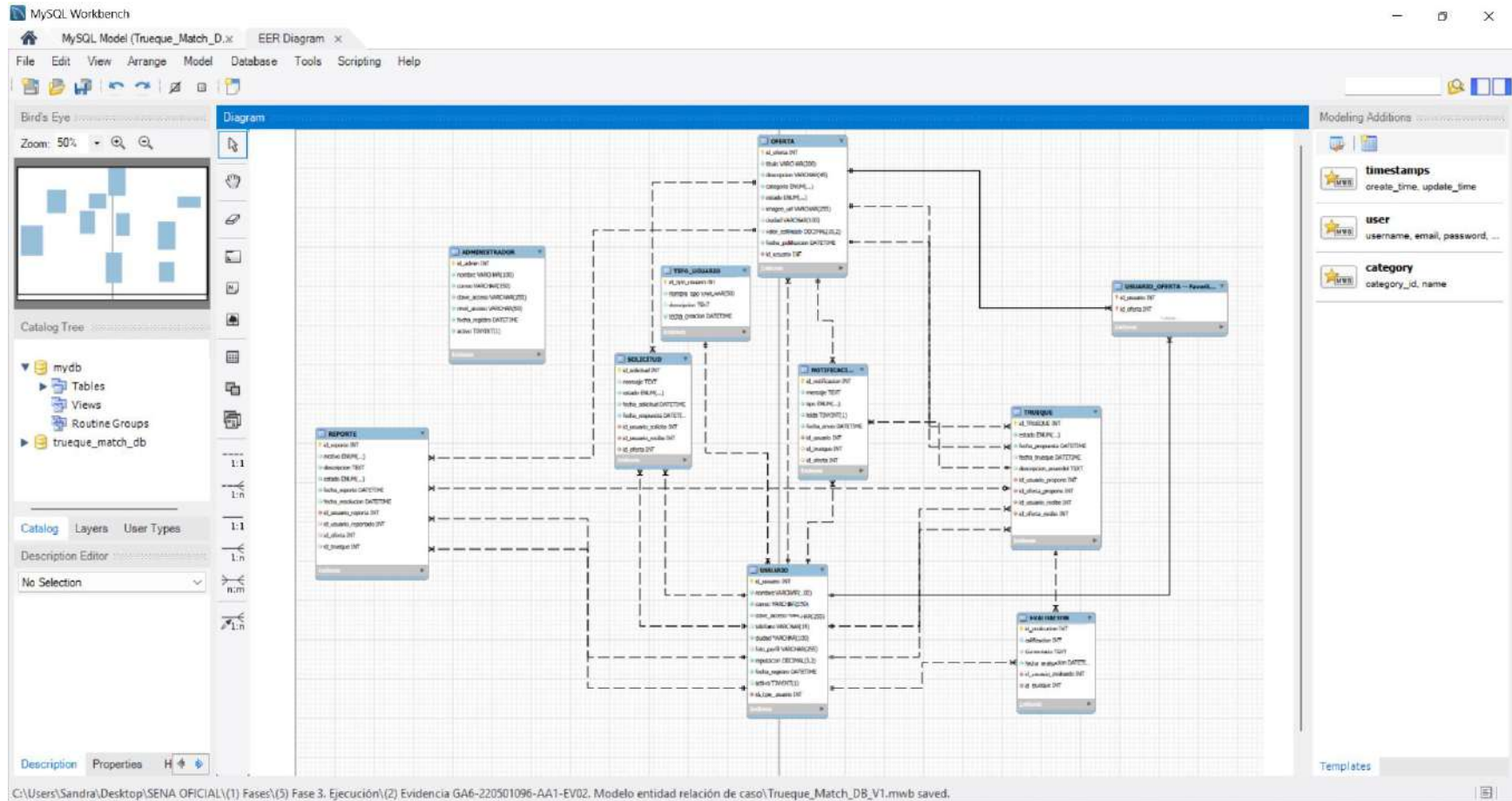
7. Modelo relacional Trueque Match

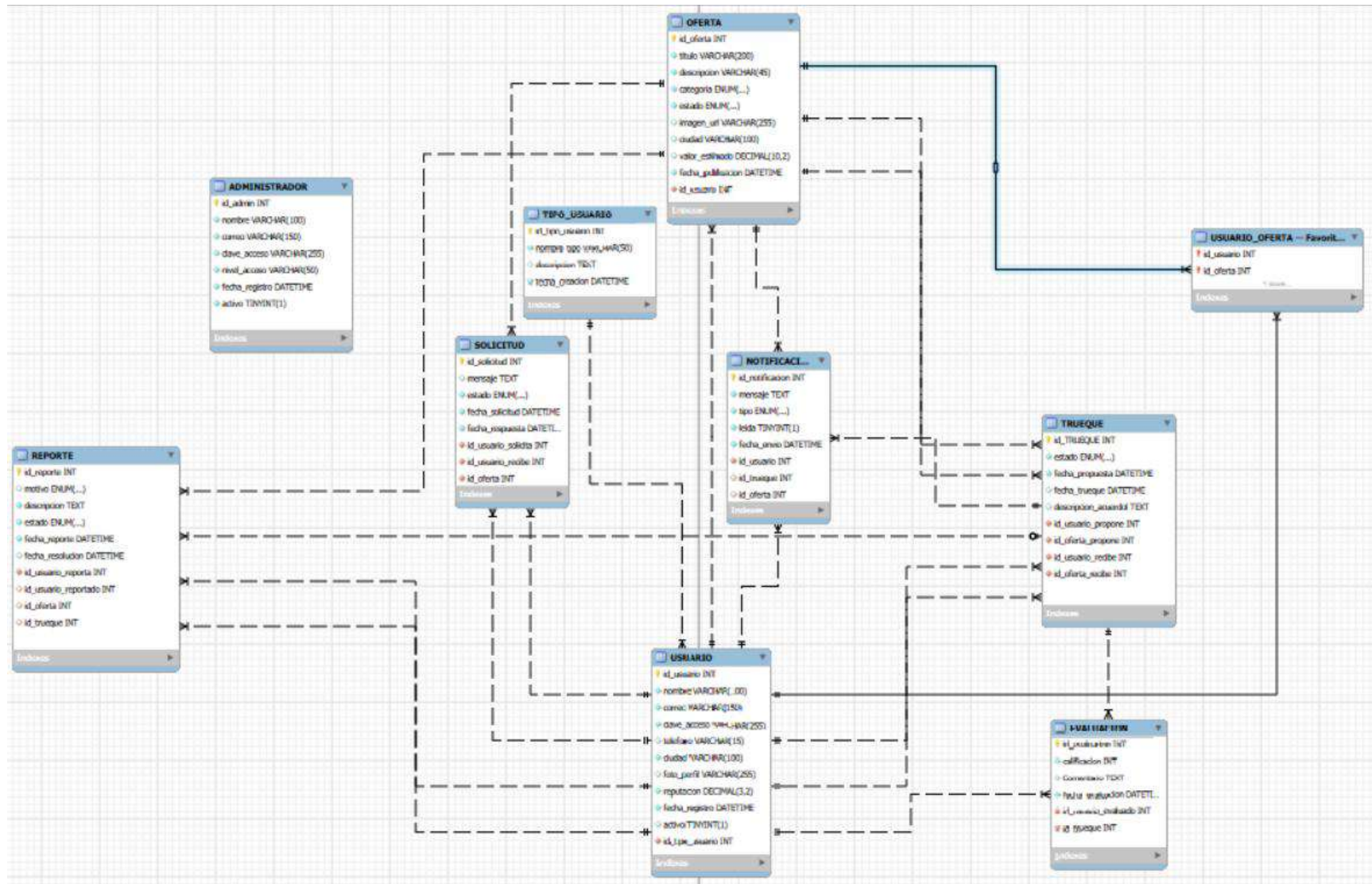
MODELO RELACIONAL TRUEQUE MATCH Y MODELO ENTIDAD RELACIÓN

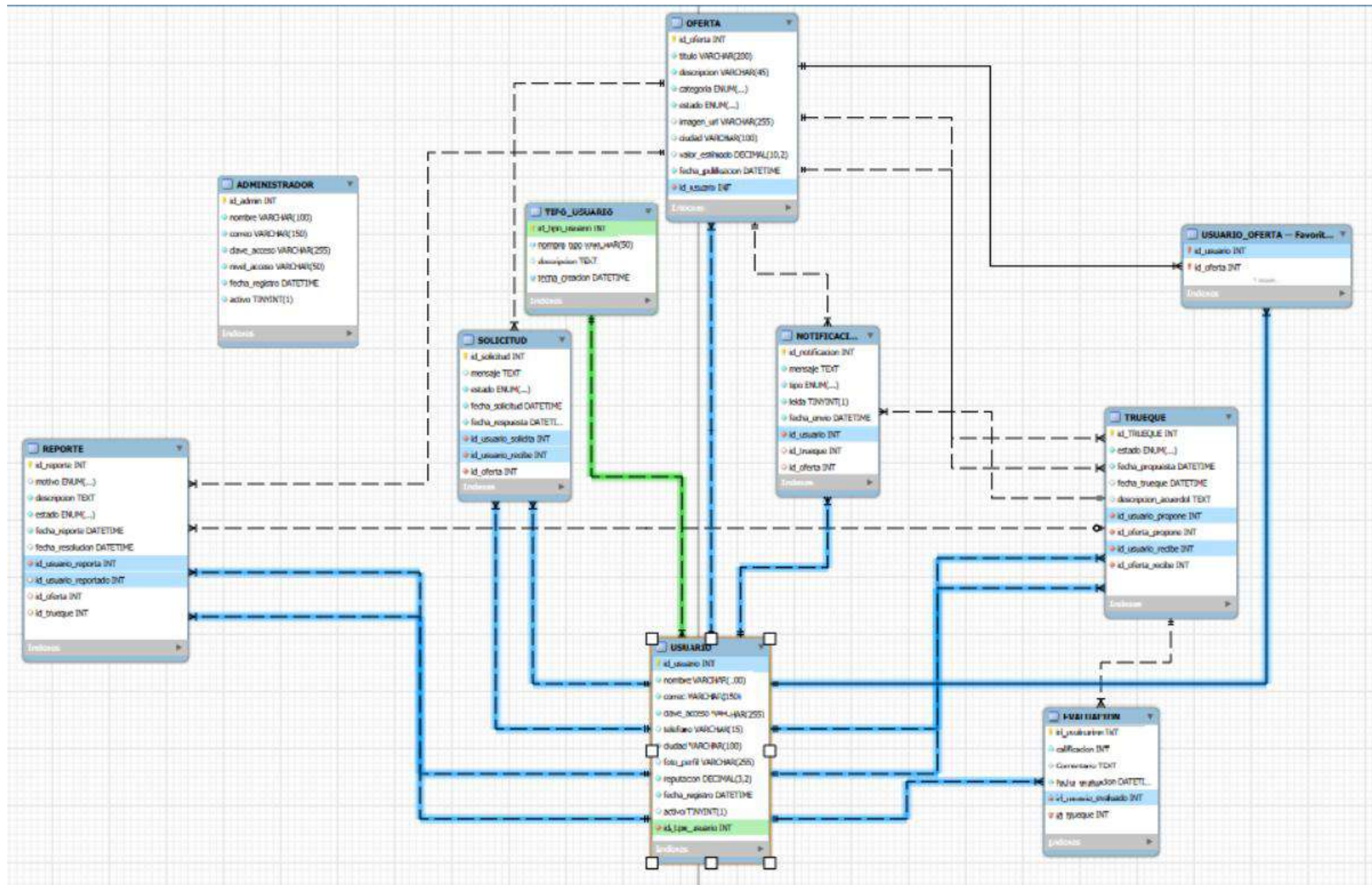


8. Link modelo entidad relación de caso en MySQL Workbench

Link modelo entidad relación de caso: <https://drive.google.com/file/d/1xt6NVhgiFE6b5xg6vk3rIPTvqD6R6rsl/view?usp=sharing>







9. Conclusiones

El diseño del modelo relacional de Trueque Match demostró la importancia de una planificación cuidadosa de la estructura de datos para garantizar la integridad, consistencia y eficiencia del sistema. Las diez tablas definidas cubren todos los requerimientos funcionales del proyecto, desde la gestión de usuarios hasta los reportes, notificaciones y el sistema de favoritos.

La aplicación de las tres formas normales garantizó la eliminación de redundancias y dependencias no deseadas en el modelo. La creación de la tabla TIPO_USUARIO para separar los roles es un ejemplo concreto de la aplicación de la 3FN: sin esta tabla, el atributo 'descripción del rol' dependería transitivamente de id_usuario a través del nombre del tipo, violando la tercera forma normal.

La resolución de la relación Muchos a Muchos entre USUARIO y OFERTA mediante la tabla intermedia USUARIO_OFERTA es un ejemplo claro de cómo la normalización mejora la estructura de la base de datos, convirtiendo una relación N:M en dos relaciones 1:N manejables e íntegras.

La cardinalidad definida para cada relación refleja fielmente la lógica de negocio de la aplicación: un usuario puede publicar muchas ofertas, participar en muchos trueques, enviar y recibir múltiples solicitudes, y recibir notificaciones sobre cada evento del sistema. Este modelo es la base sólida sobre la que se construirá el modelo entidad-relación en MySQL Workbench como segunda actividad del proyecto.

2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.5. Diccionario de datos

Este tema se desarrolló en la evidencia:

GA4-220501095-AA1-EV02. Modelo conceptual y lógico para el proyecto desarrollo de software.

Índice:

- 1. Introducción**
- 2. DIAGRAMA DE CLACES APP TRUEQUE MATCH**
- 3. DICCIONARIO DE DATOS**
- 4. NORMALIZACIÓN DEL MODELO LÓGICO**
- 5. POLÍTICAS DE SEGURIDAD**

1. Introducción

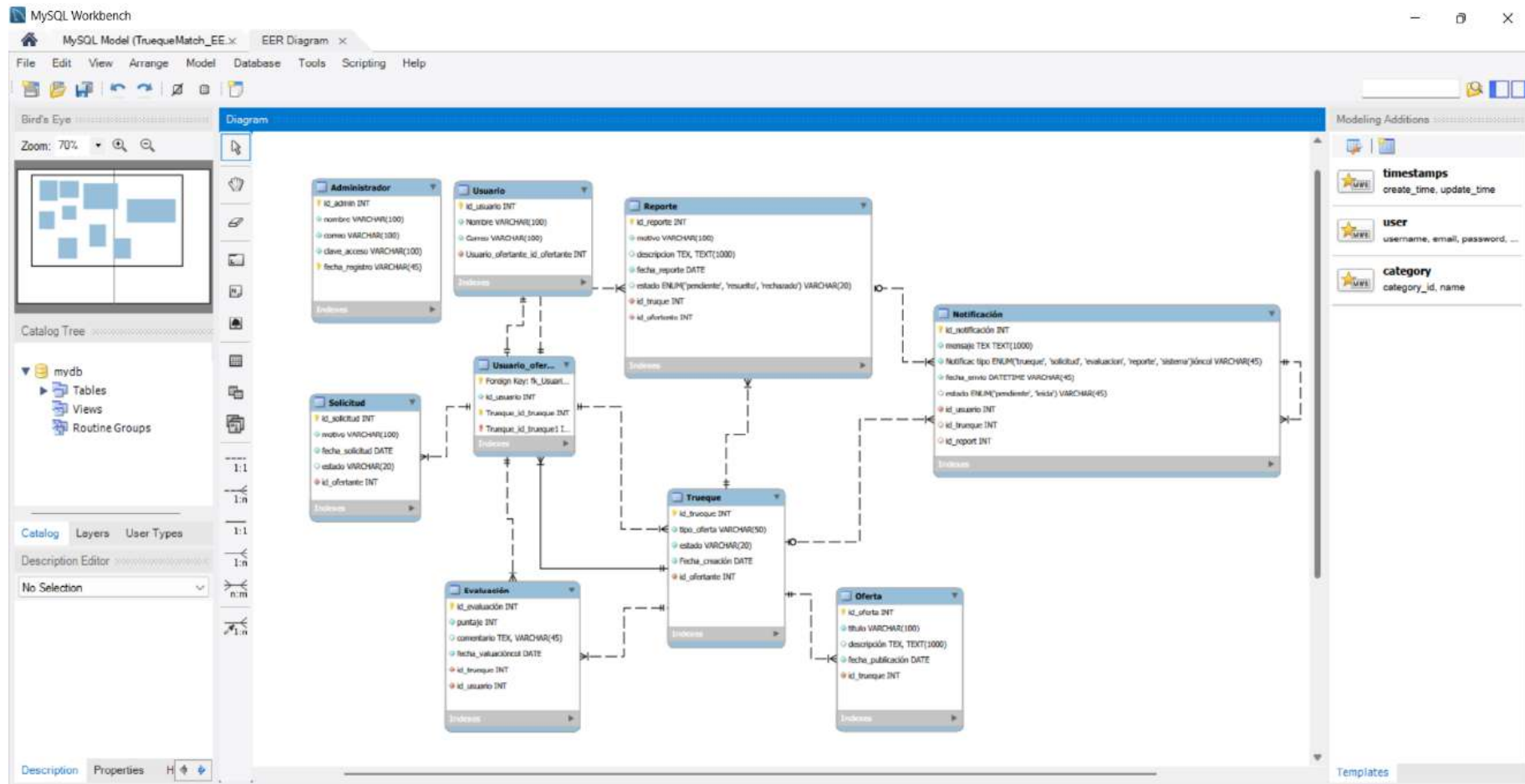
El presente documento tiene como objetivo presentar el modelo conceptual y lógico del sistema TRUEQUE MATCH, desarrollado como parte del proyecto de formación en análisis y desarrollo de software. Este sistema busca facilitar el intercambio de bienes y servicios entre usuarios mediante una plataforma segura, organizada y trazable.

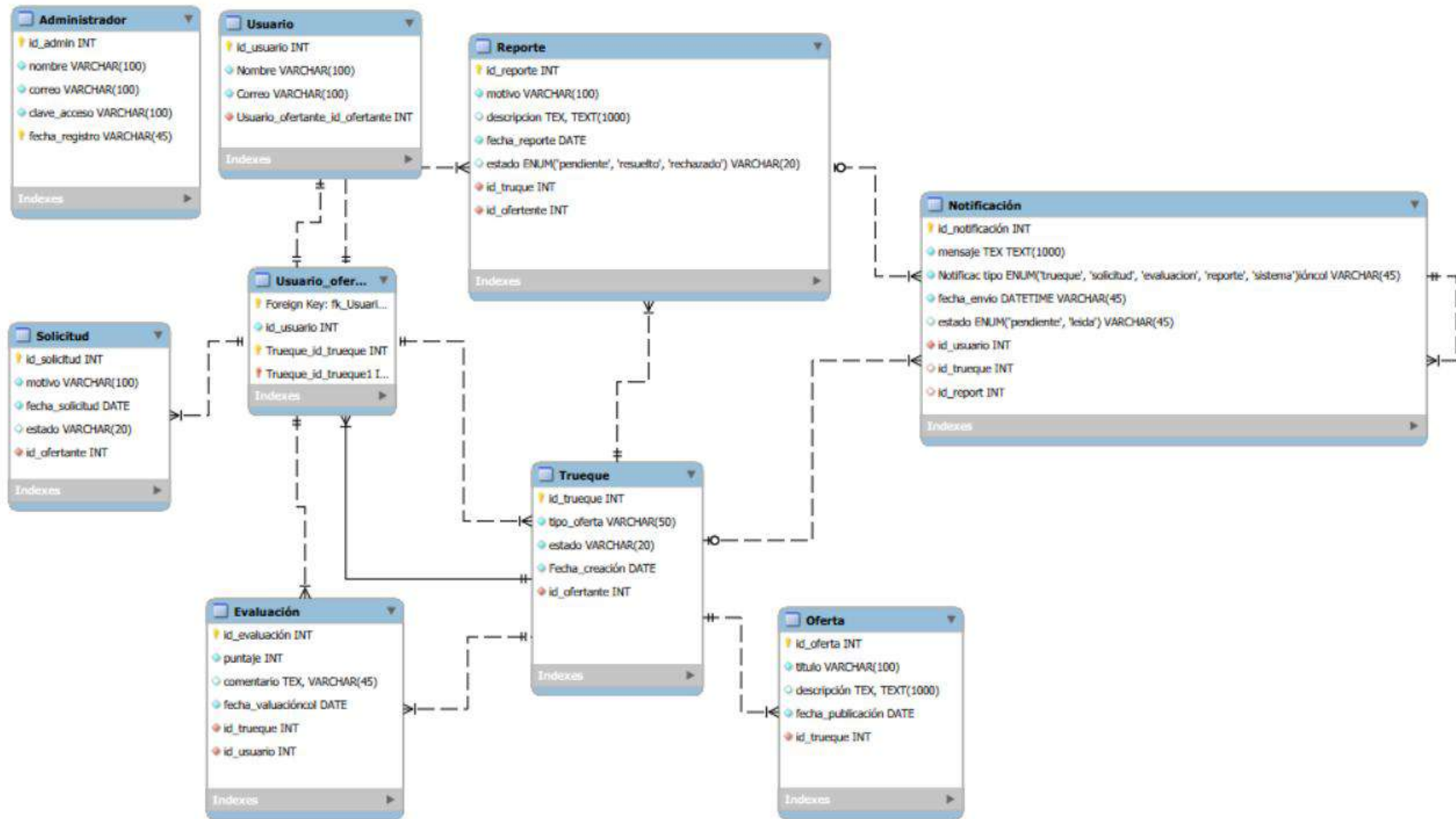
A través del diseño de entidades, relaciones y estructuras de datos, se establece una base sólida para la implementación de una base de datos relacional. El documento incluye el diccionario de datos detallado por tabla, la normalización del modelo lógico, y las políticas de seguridad que garantizan la integridad, confidencialidad y disponibilidad de la información.

Este trabajo refleja el compromiso con las buenas prácticas de modelado, documentación técnica y seguridad de datos, fundamentales para el desarrollo de soluciones tecnológicas eficientes y sostenibles.

2. DIAGRAMA DE CLACES APP TRUEQUE MATCH:

https://drive.google.com/file/d/18gEx6w2oEP_i1_fOBilHv6oFmP0lduK/view?usp=sharing





3. Diccionario de datos trueque_match_db

Corregido y verificado directamente contra el script SQL real del proyecto (trueque_match_db.sql).

Administrador										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_admin	INT	11	✓		Identificador único del administrador	NOT NULL, AUTO_INCREMENT	—	Numérico	—	Clave primaria del sistema de admins
nombre	VARCHAR	100			Nombre completo del administrador	NOT NULL	—	Texto libre	—	—
correo	VARCHAR	150			Correo electrónico del administrador	NOT NULL	—	email@dominio.com	—	—
clave_acceso	VARCHAR	255			Contraseña cifrada para acceso al sistema	NOT NULL	—	Texto cifrado	—	Se almacena con hash (password_hash de PHP)
nivel_acceso	VARCHAR	50			Nivel de privilegios del administrador	NOT NULL, DEFAULT 'moderador'	—	Texto libre	—	(Campo que faltaba en la versión anterior)
fecha_registro	DATETIME	—			Fecha en que se registró el administrador	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	—

Administrador										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
activo	TINYINT	1			Indica si la cuenta está activa	NOT NULL, DEFAULT 1	0, 1	Booleano	—	(Campo que faltaba en la versión anterior)

Usuario										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_usuario	INT	11	✓		Identificador único del usuario	NOT NULL, AUTO_INCREMENT	—	Númérico	—	⚠ Antes decía "IDUsuario"; el campo real es id_usuario
nombre	VARCHAR	100			Nombre completo del usuario	NOT NULL	—	Texto libre	—	—
correo	VARCHAR	150			Correo electrónico del usuario	NOT NULL	—	email@dominio.com	—	—
clave_acceso	VARCHAR	255			Contraseña cifrada del usuario	NOT NULL	—	Texto cifrado	—	⚠ Antes se referenciaba a una tabla "Clave_de_acceso" que no existe; la contraseña

Usuario										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
										vive directo en esta tabla
telefono	VARCHAR	15			Número de contacto del usuario	Puede ser NULL	—	Numérico	—	(Campo que faltaba en la versión anterior)
ciudad	VARCHAR	100			Ciudad donde reside el usuario	Puede ser NULL	—	Texto libre	—	(Campo que faltaba en la versión anterior)
foto_perfil	VARCHAR	255			Ruta o URL de la foto de perfil	Puede ser NULL	—	URL o ruta de archivo	—	(Campo que faltaba en la versión anterior)
reputacion	DECIMAL	3,2			Calificación promedio del usuario	NOT NULL, DEFAULT 0.00	0.00 a 5.00	Numérico decimal	—	Se recalcula automáticamente con cada evaluación
fecha_registro	DATETIME	—			Fecha de registro del usuario	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	—

Usuario										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
activo	TINYINT	1			Indica si la cuenta está activa	NOT NULL, DEFAULT 1	0, 1	Booleano	—	(Campo que faltaba en la versión anterior)
id_tipo_usuario	INT	11		✓	Rol del usuario en el sistema	NOT NULL	—	Numérico	FK hacia tipo_usuario.id_tipo_usuario	⚠ Reemplaza al campo inexistente "Usuario_defensa_datos_ID"

Tipo_usuario										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_tipo_usuario	INT	20	✓		Identificador único del tipo de usuario	NOT NULL, AUTO_INCREMENT	—	Numérico	—	Clave primaria del catálogo de roles. (Tabla completa que faltaba en la versión anterior)
nombre_tipo	VARCHAR	50			Nombre del rol del usuario	NOT NULL	'estandar', 'administrador'	Texto libre	—	—
descripcion	TEXT	—			Descripción del alcance del rol	Puede ser NULL	—	Texto libre	—	—

Tipo_usuario										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
fecha_creacion	DATETIME	—			Fecha de creación del registro	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	—

Oferta										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_oferta	INT	11	✓		Identificador único de la oferta	NOT NULL, AUTO_INCREMENT	—	Numérico	—	—
titulo	VARCHAR	200			Título breve de la oferta	NOT NULL	—	Texto libre	—	—
descripcion	TEXT	—			Descripción detallada de lo ofrecido	NOT NULL	—	Texto libre	—	—
categoria	ENUM	—			Tipo de intercambio ofrecido	NOT NULL	'producto', 'servicio', 'conocimiento', 'experiencia'	Lista fija	—	(Campo que faltaba en la versión anterior)
estado	ENUM	—			Estado actual de la oferta	NOT NULL, DEFAULT 'activa'	'activa', 'inactiva', 'intercambiada'	Lista fija	—	(Campo que faltaba en la versión anterior)
imagen_url	VARCHAR	255			Ruta o URL de la imagen de la oferta	Puede ser NULL	—	URL o ruta de archivo	—	(Campo que faltaba en la versión anterior)
ciudad	VARCHAR	100			Ciudad donde está disponible la oferta	Puede ser NULL	—	Texto libre	—	(Campo que faltaba en la versión anterior)

valor_estimado	DECIMAL	10,2			Valor estimado del producto/servicio	Puede ser NULL	—	Numérico decimal	—	(Campo que faltaba en la versión anterior)
fecha_publicación	DATETIME	—			Fecha de publicación de la oferta	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	—
id_usuario	INT	11		✓	Usuario que publicó la oferta	NOT NULL	—	Numérico	FK hacia usuario.id_usuario	—

Trueque										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_trueque	INT	11	✓		Identificador único del trueque	NOT NULL, AUTO_INCREMENT	—	Numérico	—	—
estado	ENUM	—			Estado actual del trueque	NOT NULL, DEFAULT 'pendiente'	'pendiente', 'aceptado', 'completado', 'cancelado'	Lista fija	—	⚠ Antes decía "pendiente/aceptado/cancelado" (VARCHAR); en realidad es un ENUM con 4 valores, incluye "completado"

Trueque										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
fecha_propuesta	DATETIME	—			Fecha en que se propuso el trueque	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	(Campo que faltaba en la versión anterior)
fecha_trueque	DATETIME	—			Fecha en que se concretó el trueque	Puede ser NULL	—	YYYY-MM-DD HH:MM:SS	—	—
descripcion_acuerdo	TEXT	—			Detalle del acuerdo entre las partes	Puede ser NULL	—	Texto libre	—	(Campo que faltaba en la versión anterior)
id_usuario_propone	INT	11		✓	Usuario que propone el trueque	NOT NULL	—	Número	FK hacia usuario.id_usuario	⚠ Antes decía "id_usuario"; el nombre real es id_usuario_propone
id_oferta_propone	INT	11		✓	Oferta que propone el primer usuario	NOT NULL	—	Número	FK hacia oferta.id_oferta	⚠ Antes decía "id_oferta"; el nombre real es id_oferta_propone
id_usuario_recibe	INT	11		✓	Usuario que recibe la propuesta	NOT NULL	—	Número	FK hacia usuario.id_usuario	⚠ Antes decía "id_usuario2"; el nombre real es id_usuario_recibe

Trueque										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_oferta_recibe	INT	11		✓	Oferta que se ofrece a cambio	NOT NULL	—	Número	FK hacia oferta.id_oferta	⚠ Antes decía "id_oferta2"; el nombre real es id_oferta_recibe

Usuario_oferta										
(Antes esta tabla figuraba con el nombre "Usuario_ofertante" el nombre real en la base de datos es usuario_oferta. Funciona como favoritos: qué usuarios guardaron qué ofertas.)										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_usuario	INT	11	✓ (compuesta)	✓	Usuario que guardó la oferta	NOT NULL	—	Número	FK hacia usuario.id_usuario	⚠ Esta tabla NO tiene un id propio tipo "id_usuario_oferta" como decía la versión anterior; la clave primaria es la combinación de id_usuario + id_oferta

Usuario_oferta										
(Antes esta tabla figuraba con el nombre "Usuario_ofertante" el nombre real en la base de datos es usuario_oferta. Funciona como favoritos: qué usuarios guardaron qué ofertas.)										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_oferta	INT	11	✓ (compuesta)	✓	Oferta guardada como favorita	NOT NULL	—	Numérico	FK hacia oferta.id_oferta	—
fecha_guardado	DATETIME	—			Fecha en que se guardó como favorita	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	(Campo que faltaba en la versión anterior)

Solicitud										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_solicitud	INT	11	✓		Identificador único de la solicitud	NOT NULL, AUTO_INCREMENT	—	Numérico	—	—
mensaje	TEXT	—			Mensaje del usuario solicitante	Puede ser NULL	—	Texto libre	—	⚠ Antes decía "motivo" (VARCHAR 100); el campo real es mensaje (TEXT)
estado	ENUM	—			Estado actual de la solicitud	NOT NULL, DEFAULT 'pendiente'	'pendiente', 'aceptada', 'rechazada'	Lista fija	—	(Campo que faltaba en la versión anterior)
fecha_solicitud	DATETIME	—			Fecha en que se generó la solicitud	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	—
fecha_respuesta	DATETIME	—			Fecha en que se respondió la solicitud	Puede ser NULL	—	YYYY-MM-DD HH:MM:SS	—	(Campo que faltaba en la versión anterior)
id_usuario_solicitante	INT	11		✓	Usuario que envía la solicitud	NOT NULL	—	Numérico	FK hacia usuario.id_usuario	⚠ Antes decía "id_usuario"; el nombre real es id_usuario_solicitante

id_usuario_recibe	INT	11		✓	Usuario que recibe la solicitud	NOT NULL	—	Numérico	FK hacia usuario.id_usuario	(Campo que faltaba en la versión anterior)
id_oferta	INT	11		✓	Oferta relacionada con la solicitud	NOT NULL	—	Numérico	FK hacia oferta.id_oferta	—

Evaluacion										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_evaluacion	INT	11	✓		Identificador único de la evaluación	NOT NULL, AUTO_INCREMENT	—	Numérico	—	—
puntaje	INT	1			Calificación otorgada	NOT NULL	1 a 5	Numérico	—	Escala 1 (baja) a 5 (excelente)
comentario	TEXT	—			Comentario sobre la experiencia	Puede ser NULL	—	Texto libre	—	⚠ Antes decía "comentario_TDX" (VARCHAR 45); el campo real es comentario (TEXT)
fecha_evaluacion	DATETIME	—			Fecha en que se realizó la evaluación	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	⚠ Antes decía "fecha_validacion"; el nombre real es fecha_evaluacion

Evaluacion										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_usuario	INT	11		✓	Usuario que realiza la evaluación	NOT NULL	—	Numérico	FK hacia usuario.id_usuario	—
id_trueque	INT	11		✓	Trueque que está siendo evaluado	NOT NULL	—	Numérico	FK hacia trueque.id_trueque	—

Notificacion										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_notificacion	INT	11	✓		Identificador único de la notificación	NOT NULL, AUTO_INCREMENT	—	Numérico	—	—
mensaje	TEXT	—			Contenido del mensaje	NOT NULL	—	Texto libre	—	—
tipo	ENUM	—			Tipo de notificación	NOT NULL, DEFAULT 'info'	'info', 'alerta', 'sistema', 'trueque'	Lista fija	—	⚠ Antes decía "notificacionBK" (VARCHAR 45); el campo real es tipo (ENUM)

Notificacion										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
leida	TINYINT	1			Indica si la notificación fue leída	NOT NULL, DEFAULT 0	0, 1	Booleano	—	(Campo que faltaba en la versión anterior)
fecha_envio	DATETIME	—			Fecha y hora del envío	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	—
id_usuario	INT	11		✓	Usuario que recibe la notificación	NOT NULL	—	Numérico	FK hacia usuario.id_usuario	—
id_trueque	INT	11		✓	Trueque relacionado (si aplica)	Puede ser NULL	—	Numérico	FK hacia trueque.id_trueque	Se borra la referencia si se elimina el trueque (ON DELETE SET NULL)
id_oferta	INT	11		✓	Oferta relacionada (si aplica)	Puede ser NULL	—	Numérico	FK hacia oferta.id_oferta	Se borra la referencia si se elimina la oferta (ON DELETE SET NULL)

Reporte										
Campo	Tipo dato	Tamaño	PK	FK	Descripción	Restricciones	Valores posibles	Formato esperado	Relación con otras tablas	Observaciones
id_reporte	INT	11	✓		Identificador único del reporte	NOT NULL, AUTO_INCREMENT	—	Numérico	—	—
motivo	ENUM	—			Razón principal del reporte	NOT NULL	'fraude', 'incumplimiento', 'comportamiento', 'otro'	Lista fija	—	⚠ Antes era VARCHAR con 3 valores de ejemplo; el campo real es un ENUM con 4 valores fijos
descripcion	TEXT	—			Detalle completo del incidente	NOT NULL	—	Texto libre	—	⚠ Antes decía "descripcion_TXT"; el nombre real es descripcion
estado	ENUM	—			Estado de revisión del reporte	NOT NULL, DEFAULT 'pendiente'	'pendiente', 'revisado', 'resuelto'	Lista fija	—	(Campo que faltaba en la versión anterior)
fecha_reporte	DATETIME	—			Fecha en que se generó el reporte	NOT NULL, DEFAULT CURRENT_TIMESTAMP	—	YYYY-MM-DD HH:MM:SS	—	—
fecha_resolucion	DATETIME	—			Fecha en que se resolvió el reporte	Puede ser NULL	—	YYYY-MM-DD HH:MM:SS	—	(Campo que faltaba en la versión anterior)

id_usuario_reporta	INT	11		✓	Usuario que hace el reporte	NOT NULL	—	Numérico	FK hacia usuario.id_usuario	⚠ Antes decía "id_usuario"; el nombre real es id_usuario_reporta
id_usuario_reportado	INT	11		✓	Usuario que es reportado	Puede ser NULL	—	Numérico	FK hacia usuario.id_usuario	(Campo que faltaba en la versión anterior)
id_oferta	INT	11		✓	Oferta relacionada (si aplica)	Puede ser NULL	—	Numérico	FK hacia oferta.id_oferta	—
id_trueque	INT	11		✓	Trueque relacionado (si aplica)	Puede ser NULL	—	Numérico	FK hacia trueque.id_trueque	—

NORMALIZACIÓN DEL MODELO LÓGICO

La normalización es un proceso fundamental en el diseño de bases de datos relacionales, cuyo objetivo es eliminar redundancias, evitar anomalías en la manipulación de datos y garantizar la integridad estructural del sistema. En el modelo lógico de TRUEQUE MATCH se aplicaron las tres primeras formas normales (1FN, 2FN y 3FN), asegurando un diseño eficiente y escalable.

- **Primera Forma Normal (1FN)**

Todas las tablas del modelo cumplen con la 1FN, ya que:

- Cada campo contiene valores atómicos (sin listas ni agrupaciones).
- No existen repeticiones de grupos de datos dentro de una misma fila.
- Se definieron claves primarias únicas para identificar cada registro.

- **Segunda Forma Normal (2FN)**

Se garantiza la 2FN mediante:

- Eliminación de dependencias parciales en tablas con claves compuestas.
- Separación de entidades como Usuario, Oferta, Trueque y Evaluación, evitando que atributos dependan solo de parte de la clave.
- Uso de tablas puente como Usuario_oferta para representar relaciones muchos-a-muchos sin redundancia.

- **Tercera Forma Normal (3FN)**

El modelo cumple con la 3FN al:

- Eliminar dependencias transitivas: ningún campo depende de otro que no sea clave primaria.
- Separar atributos como clave acceso, comentario TDX, notificación BK en sus respectivas tablas, evitando duplicación de datos.
- Mantener la coherencia semántica entre entidades y sus atributos.

Este proceso de normalización asegura que el sistema TRUEQUE MATCH sea:

- Eficiente en almacenamiento y consultas.
- Escalable para futuras funcionalidades.
- Robusto frente a errores y duplicaciones.

5. POLÍTICAS DE SEGURIDAD

Para garantizar la protección de los datos en el sistema **TRUEQUE MATCH**, se han definido políticas que cubren los tres pilares fundamentales de la seguridad de la información: integridad, confidencialidad y disponibilidad. Estas políticas aseguran que el sistema sea confiable, trazable y resistente ante fallos o accesos no autorizados.

- **Integridad**

- Se implementan claves primarias y foráneas en todas las tablas para asegurar la consistencia relacional.
- Se aplican restricciones NOT NULL en campos críticos para evitar registros incompletos.
- Se utiliza AUTO_INCREMENT en identificadores para evitar duplicidad y garantizar unicidad.
- La normalización del modelo evita redundancias y mantiene la coherencia de los datos.

- **Confidencialidad**

- Las contraseñas (clave acceso) se almacenan de forma cifrada mediante algoritmos seguros como bcrypt o SHA-256.
- Se recomienda implementar roles de usuario (administrador, usuario estándar) con niveles de acceso diferenciados.
- El acceso a datos sensibles como correos, evaluaciones y reportes debe estar restringido por sesión y autenticación.

- **Disponibilidad**

- Se sugiere establecer copias de seguridad periódicas de la base de datos.
- El sistema debe contar con mecanismos de recuperación ante fallos, como replicación o restauración desde backups.
- Las notificaciones y reportes permiten detectar incidentes o anomalías que puedan afectar la disponibilidad del servicio.

Estas políticas forman parte del diseño estructural del sistema y deben complementarse con buenas prácticas de desarrollo, monitoreo y mantenimiento continuo. TRUEQUE MATCH no solo busca facilitar el intercambio, sino también proteger la confianza de sus usuarios.

2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.6. Scripts de instalación

Este tema se desarrolló en las evidencias:

Evidencia GA10-220501097-AA5-EV01. Configuración de servicios, bases de datos y software en el equipo del cliente.

Evidencia GA10-220501097-AA6-EV01. Archivos cargados en la plataforma de producción.

Se adiciono llevar Trueque Match a internet

Contenido.....	5
1. Introducción.....	6
2. Características de la aplicación a desplegar.....	7
3. Componentes de hardware a validar.....	7
4. Requisitos mínimos calculados para el servidor.....	8
4.1 Prerrequisitos de software.....	9
5. Consideraciones de licenciamiento de software.....	10
6. Plan de validación: cómo comprobar si un equipo cumple los mínimos.....	10
6.1 Método manual (verificación desde el propio sistema operativo).....	10
6.2 Método con herramienta de software.....	11
7. Aplicación práctica: verificación sobre un equipo real.....	11
8. Anexo: Entorno actual de desarrollo y pruebas (servidor local con XAMPP).....	12
8. Conclusiones.....	14
TABLA DE CONTENIDO.....	15
1. Introducción.....	17
2. Objetivo.....	18
2.1 Objetivo General.....	18
2.2 Objetivos Específicos.....	18
3. Estándares de Codificación.....	19
3.1 ¿Qué son los Estándares de Codificación?.....	19
3.2 ¿Por qué son Necesarios los Estándares de Codificación?.....	19
3.2.1 Legibilidad y Comprensión del Código.....	20
3.2.2 Reducción de Errores.....	20
3.2.3 Facilita la Revisión de Código (Code Review).....	20
3.2.4 Escalabilidad del Proyecto.....	20
3.2.5 Automatización y Calidad Continua.....	20
3.3 Prácticas Recomendadas para el Uso de Estándares de Codificación.....	21

3.3.1 Definir los Estándares al Inicio del Proyecto	22
3.3.2 Usar Herramientas de Análisis Estático (Linters)	22
3.3.3 Configurar Formateadores Automáticos de Código	22
3.3.4 Integrar los Estándares en el Control de Versiones	23
3.3.5 Documentar el Código con JSDoc	23
3.3.6 Convenciones de Nomenclatura	23
4. Herramientas de Versionamiento	24
4.1 ¿Qué es el Versionamiento de Software?	24
4.2 Tipos de Sistemas de Control de Versiones	24
4.2.1 Sistemas Locales	25
4.2.2 Sistemas Centralizados (CVCS)	25
4.2.3 Sistemas Distribuidos (DVCS)	25
4.3 Herramientas Principales de Versionamiento	25
4.3.1 Git	25
4.3.2 GitHub	26
4.3.5 Subversion (SVN)	27
4.4 Comparativa de Herramientas de Versionamiento	27
5. Selección de Herramientas para Trueque Match	28
5.1 Contexto del Proyecto	28
5.2 Herramientas Seleccionadas	29
5.2.1 Git como Sistema de Control de Versiones	29
5.2.2 GitHub como Plataforma de Repositorio Remoto	30
5.2.3 ESLint como Herramienta de Análisis Estático	30
5.2.4 Prettier como Formateador de Código	31
5.2.5 Conventional Commits como Estándar de Mensajes	31
5.3 Justificación Final de la Selección	32
6. Conclusiones	33
Contenido	36
1. Introducción	40
2. Objetivo	41
2.1 Objetivo General	41
2.2 Objetivos Específicos	41
3. Marco Teórico	42
3.1 Modelo Relacional	42
3.2 Conceptos Fundamentales	42
3.3 Cardinalidad de las Relaciones	43
3.4 Normalización	43
4. Modelo Relacional Trueque Match	43
4.1 Convenciones del Modelo	43

4.2 Tablas del Modelo.....	44
4.2.1 Tabla TIPO_USUARIO	44
4.2.2 Tabla USUARIO	45
4.2.3 Tabla ADMINISTRADOR	46
4.2.4 Tabla OFERTA.....	47
4.2.5 Tabla TRUEQUE.....	49
4.2.6 Tabla EVALUACION	50
4.2.7 Tabla SOLICITUD	52
4.2.8 Tabla NOTIFICACION	53
4.2.9 Tabla REPORTE	54
4.2.10 Tabla USUARIO_OFERTA (Favoritos).....	56
5. Cardinalidad de las Relaciones	57
6. Aplicación de la Normalización.....	58
6.2 Segunda Forma Normal (2FN).....	59
6.3 Tercera Forma Normal (3FN)	59
6.4 Resumen de Normalización por Tabla	60
1. Introducción.....	¡Error! Marcador no definido.
Introducción:	94
Instalación:	95
• Paso 1: Descargar e instalar VirtualBox.....	95
• Paso 2: Descargar la ISO de Ubuntu	97
• Paso 3: Crear la máquina virtual.....	98
• Paso 4: Instalar XAMPP para Linux	104
4.1 Descargar XAMPP para Linux (dentro de la máquina virtual)	104
4.2 Confirmar dónde quedó descargado	106
4.3 Abrir una terminal en la carpeta Descargas.....	107
4.4 Dar permisos de ejecución al archivo	107
4.5 Ejecutar el instalador	108
Poner una página de prueba en htdocs (dentro de Ubuntu).....	112
3.1 Descargar una plantilla (podemos reusar la misma, Clearwave).....	113
3.2 Mover el contenido a /opt/lampp/htdocs	115
3.3 Verlo funcionando	116
DOCKER Contenedores	118
4.1 Paso a Paso: Despliegue con Docker en Ubuntu.....	118
• PASO 1: Actualizar e instalar Docker:.....	118
• PASO 2: Lanzar Apache en contenedor Docker:	118
• PASO 3: Verificar que corre:	119

• PASO 4: Abrir Firefox en Ubuntu y entrar a:	119
Evidencia GA10-220501097-AA6-EV01. Archivos cargados en la plataforma de producción.	121
5.1. Plantilla web vs. sitio web (y qué cambia con la evidencia anterior).....	121
5.2 La idea con la analogía, para conectar con lo que ya sabés.....	122
Paso 1: Crear tu cuenta en InfinityFree	122
Paso 2: Crear mi primer sitio (subdominio gratis)	123
• ¿Por qué necesitamos esa contraseña FTP?	124
truequematch-gerson.great-site.net	125
Paso 3: Entrar al Administrador de archivos y subir la plantilla	126
Paso 4: Subir la plantilla Clearwave	128
Ahora sí, probemos la URL real	131
Repaso completo de las 5 evidencias visuales que ya tenés para esta parte	132
Paso 5: Adicionar HTTPS://	133
Vamos a comprobarlo directo:	134
Conclusiones	136
Llevar Trueque Match de verdad a Internet	137
Sobre guardar en “GitHub”	137
7.2 El plan completo, en 5 pasos.....	138
Paso 1: Preparar mi base de datos para exportar	138
Paso 2: Crear la base de datos en InfinityFree	139
Paso 3: Importar mi base de datos	143
Paso 4: Editar conexion.php para producción	146
Paso 5: Subir el proyecto completo al hosting	150
Paso 6: Probar el login en vivo (versión corregida y ya verificada)	157
Paso 7: Pruebas de funcionalidad:	159
Paso8: Actualización del README.md con el despliegue a producción	161
Paso 9: Trueque Match, en resumen, quedó en producción real:	162
10. Conclusiones de la evidencia llevar Trueque Match de verdad a Internet:	164
.....	168
Diagrama de componentes e identificación de capas	168

Introducción:

Como parte del componente formativo de sistemas operativos e implementación de servicios, en este informe documento dos ejercicios prácticos complementarios que juntos recorren el camino completo de un despliegue de aplicaciones: desde montar un servidor propio en un sistema operativo distinto (Linux, dentro de una máquina virtual) hasta publicar un sitio en un servidor real de Internet, accesible desde cualquier parte del mundo.

La primera parte (Evidencia GA10-220501097-AA5-EV01) muestra el montaje de un entorno de servidor sobre Ubuntu Linux, usando virtualización con VirtualBox para no afectar el sistema operativo principal del equipo de trabajo. Dentro de esa máquina virtual se instaló XAMPP para Linux, replicando el mismo stack (Apache y MySQL) que ya usa el proyecto Trueque Match en Windows, y se comprobó su funcionamiento desplegando una plantilla web de prueba de forma local.

La segunda parte (Evidencia GA10-220501097-AA6-EV01) da el paso siguiente: llevar esa misma lógica de despliegue a un entorno público real, usando InfinityFree como plataforma de hosting y dominio gratuito. Ahí se creó una cuenta de hosting, se generó un subdominio personalizado y se publicó una plantilla HTML gratuita, quedando accesible desde cualquier navegador conectado a Internet, sin depender de que el equipo propio esté encendido.

Ambos ejercicios, aunque usan plataformas distintas, comparten la misma lógica de fondo que ya se trabajó con XAMPP en Windows: un servidor web que recibe archivos en una carpeta específica (`htdocs`) y los sirve al mundo a través de una dirección. Documentar los dos procesos juntos permite comparar de forma directa las diferencias entre un entorno de desarrollo local, uno virtualizado en otro sistema operativo, y uno público en producción los tres extremos del ciclo real de vida de una aplicación web.

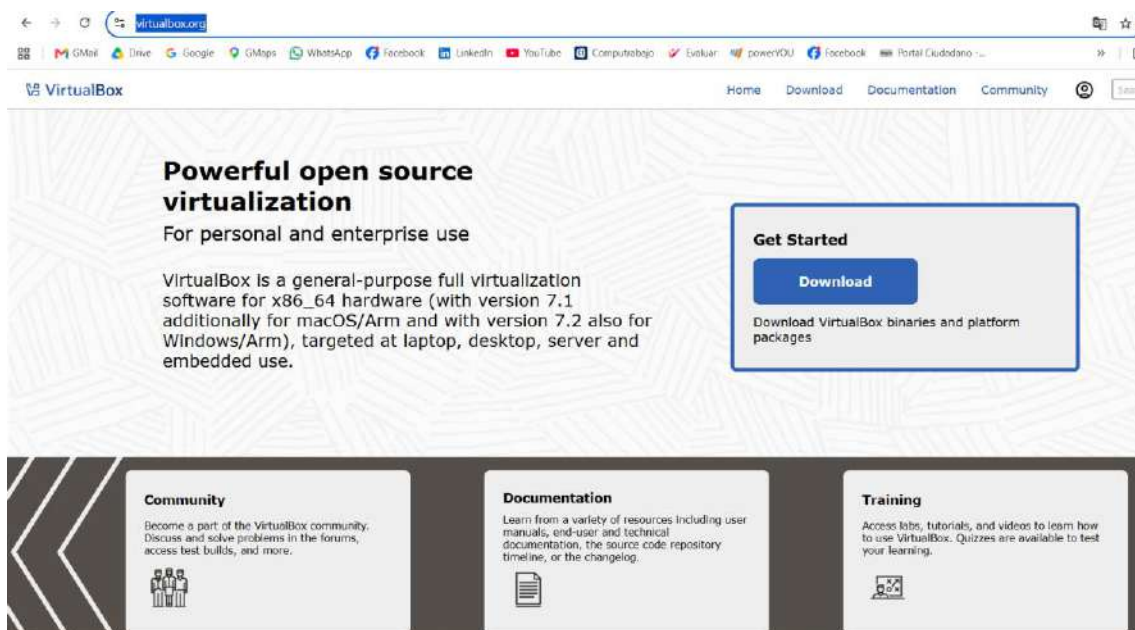
Instalación:

Esta evidencia tiene dos partes (**virtualización primero, contenedores Docker después**), y como acordamos, arrancamos solo con la primera: VirtualBox + Ubuntu + XAMPP para Linux.

- **Paso 1: Descargar e instalar VirtualBox**

Pensemos como un ejemplo simple: mi computador con Windows es como un edificio de un solo piso. VirtualBox es como construir una habitación separada, con su propia puerta, dentro de ese mismo edificio ahí adentro vamos a meter una segunda computadora completa (Ubuntu) sin tocar ni arriesgar nada de mi Windows actual.

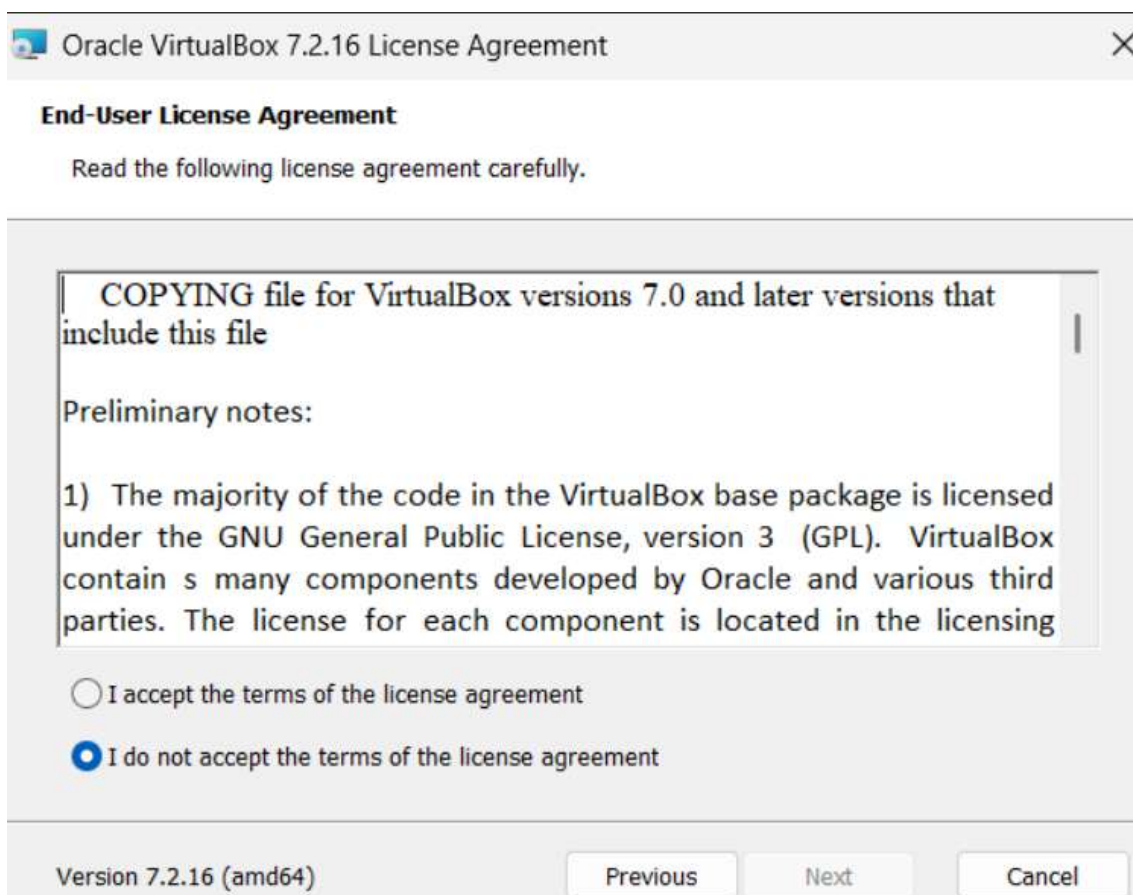
- Abrimos tu navegador y andá a: <https://www.virtualbox.org/>



- Vamos a ver un botón grande que dice "**Download VirtualBox**". Hacemos click ahí.
- En la siguiente página, buscá la sección "**VirtualBox for Windows hosts**" y hacé click para descargar el instalador (un archivo .exe).



- Cuando termine de descargar, abrí ese archivo .exe.
- Nos va a salir un asistente de instalación le damos “I accept”... **Next, Next, Next**, dejando todo por defecto.



- Al final, dale **Install**, y esperá a que termine.
- Cuando termine, marcá la casilla para abrir VirtualBox y dale **Finish**.



Perfecto VirtualBox instalado y abierto, tal cual.

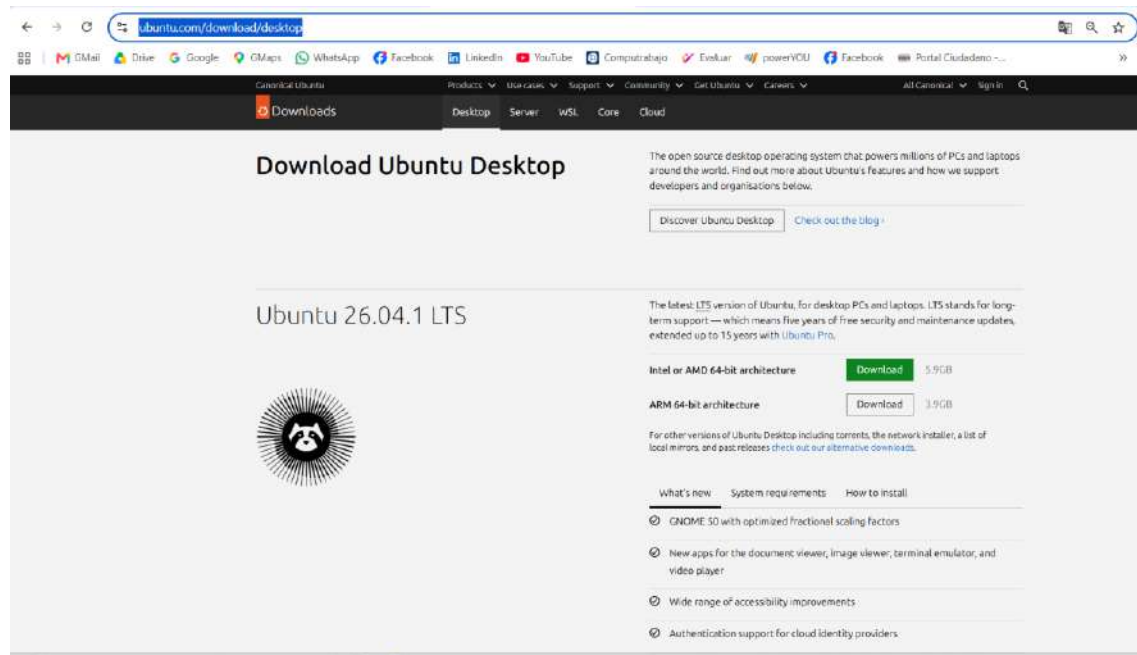
Justo nos está preguntando algo importante antes de seguir: "Basic Mode" o "Expert Mode". Elegimos Basic Mode, es como elegir entre el modo automático o manual de un carro: el modo experto nos muestra opciones avanzadas que no necesitamos para este ejercicio, mientras que el básico nos da justo lo suficiente para crear la máquina virtual sin complicarnos con configuraciones extra.

Hacemos click en "**Basic Mode**".

- **Paso 2: Descargar la ISO de Ubuntu**

Ahora necesitamos el "molde" con el que vamos a construir la computadora Linux de adentro. Eso es la ISO un archivo grande que contiene el sistema operativo completo, listo para instalar.

- Abrimos una pestaña nueva en mi navegador y nos dirigimos a:
<https://ubuntu.com/download/desktop>



- Vamos a ver un botón para descargar la versión de escritorio (Desktop) más reciente, normalmente dice algo como **"Download Ubuntu 24.04 LTS"** (o la versión que aparezca disponible en ese momento).
- Hacemos **click** en **descargar**. El archivo es grande (unos 4-6 GB), así que va a tardar un rato según depende de mi internet.
- Confirmamos que se está descargando en mi carpeta de Descargas, con un nombre como ubuntu-24.04-desktop-amd64.iso.

(Ojo: tenemos que fijarnos que diga "amd64" en el nombre, esa es la arquitectura que usa prácticamente cualquier PC con Windows normal, incluido el mío. No hace falta elegir nada especial, esa es la versión correcta.)

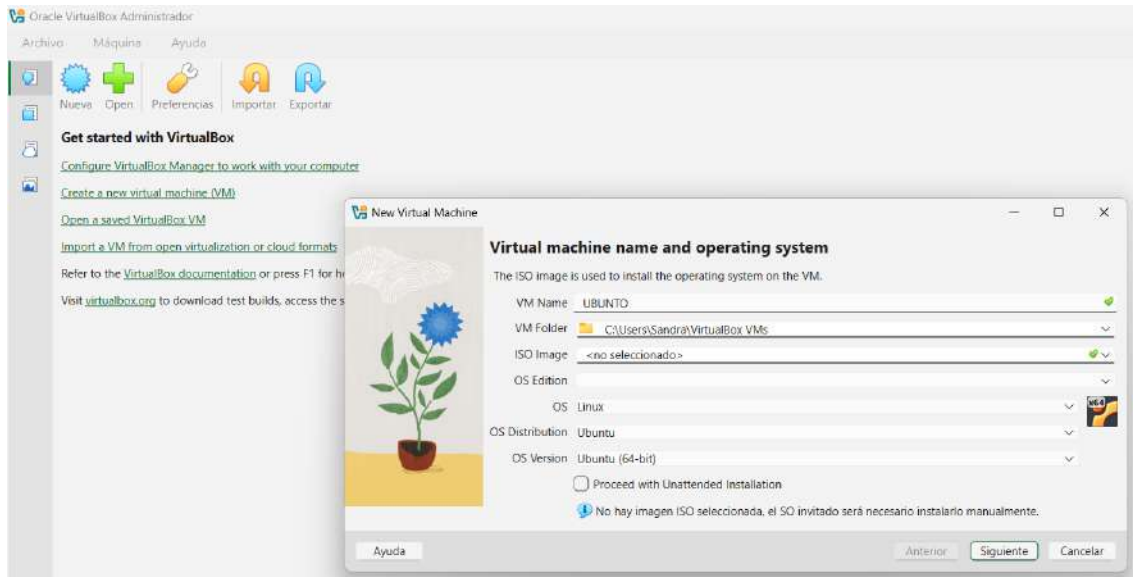
• Paso 3: Crear la máquina virtual

Ahora sí, vamos a "construir la habitación nueva" dentro de VirtualBox y meterle el molde de Ubuntu.

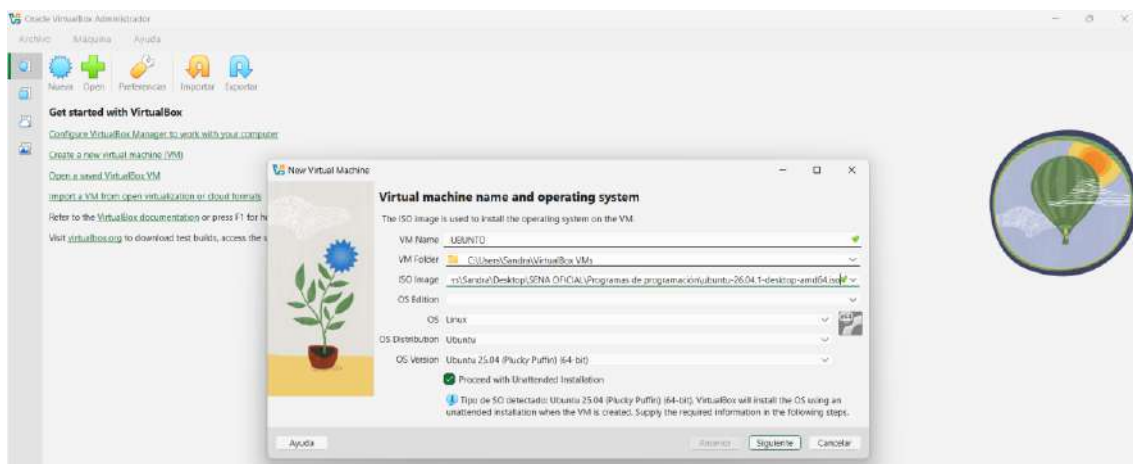
- Abrimos VirtualBox (el que ya instalamos).
- Arriba a la izquierda, hacemos **click** en el botón azul **"Nueva"**.
- Nos va aparecer un formulario. Lo llenamos así:

- ✓ **Nombre:** ponemos algo identificable, por ejemplo TruequeMatch_Ubuntu o simplemente Ubuntu.

- ✓ **Carpeta de la máquina:** dejamos como está por defecto, no hace falta cambiarla.
- ✓ **Tipo:** Linux (debería seleccionarse solo al escribir el nombre).
- ✓ **Versión:** Ubuntu (64-bit).



✓ **Imagen ISO:** acá es donde le decimos cuál es el "molde", hacemos **click** en el desplegable o en el botón de carpeta, y seleccioná el archivo ubuntu-26.04.1-desktop-amd64.iso que ya tenés descargado.

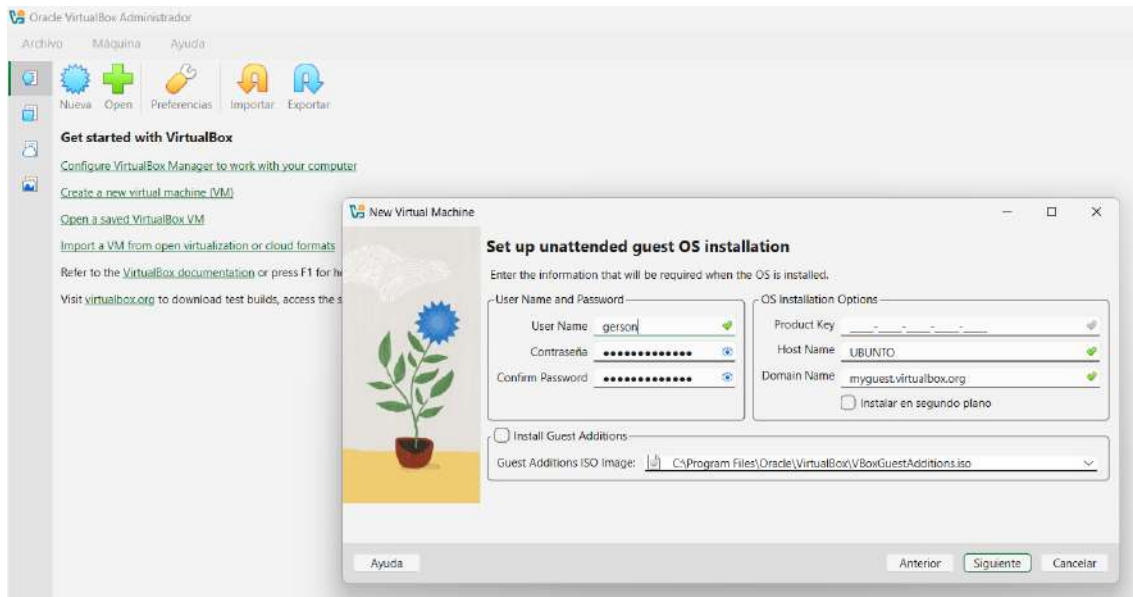


- Le damos **Siguiente**.

En la siguiente pantalla nos va a pedir usuario y contraseña para la cuenta de Ubuntu:

Nombre de usuario: el que quieras (por ejemplo **gerson**).

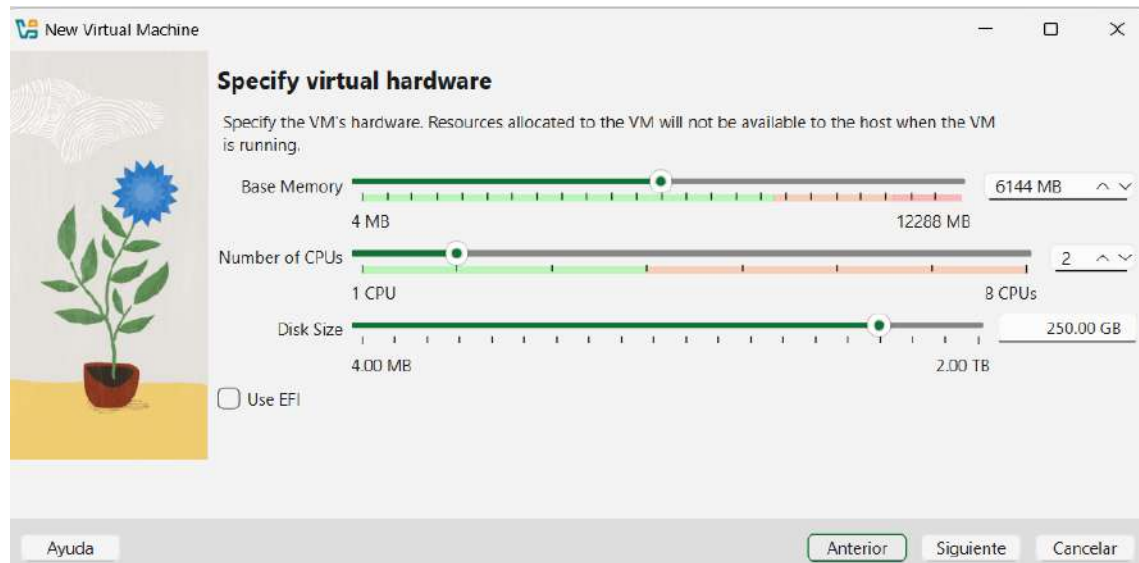
Contraseña: definimos una que podamos recordar fácil, la vamos a necesitar más adelante para instalar XAMPP, así que no debemos olvidarla.



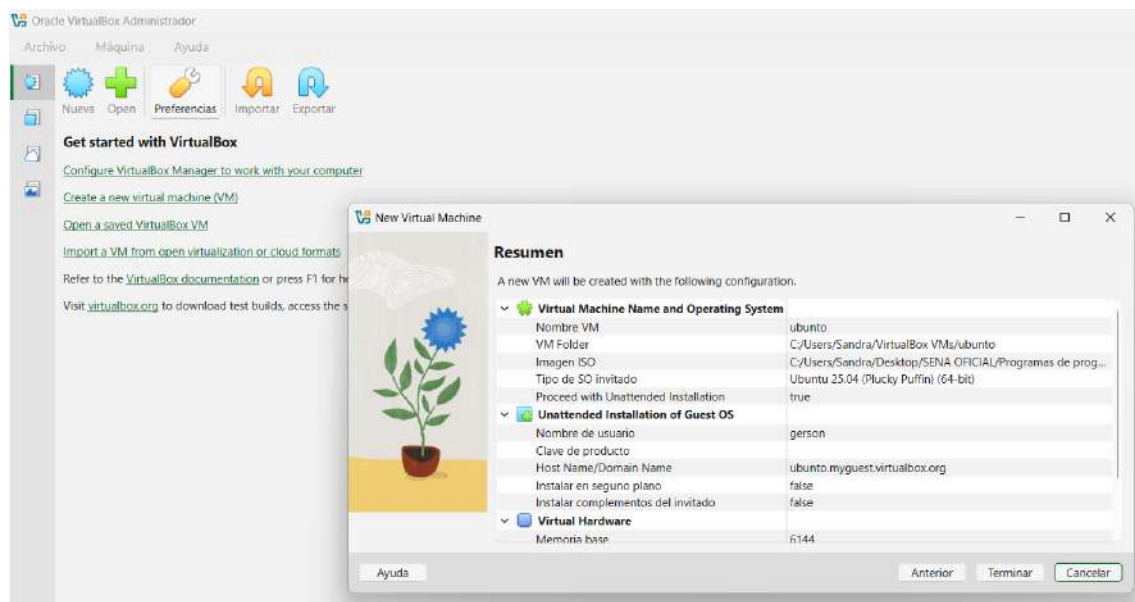
- Le damos **Siguiente**.

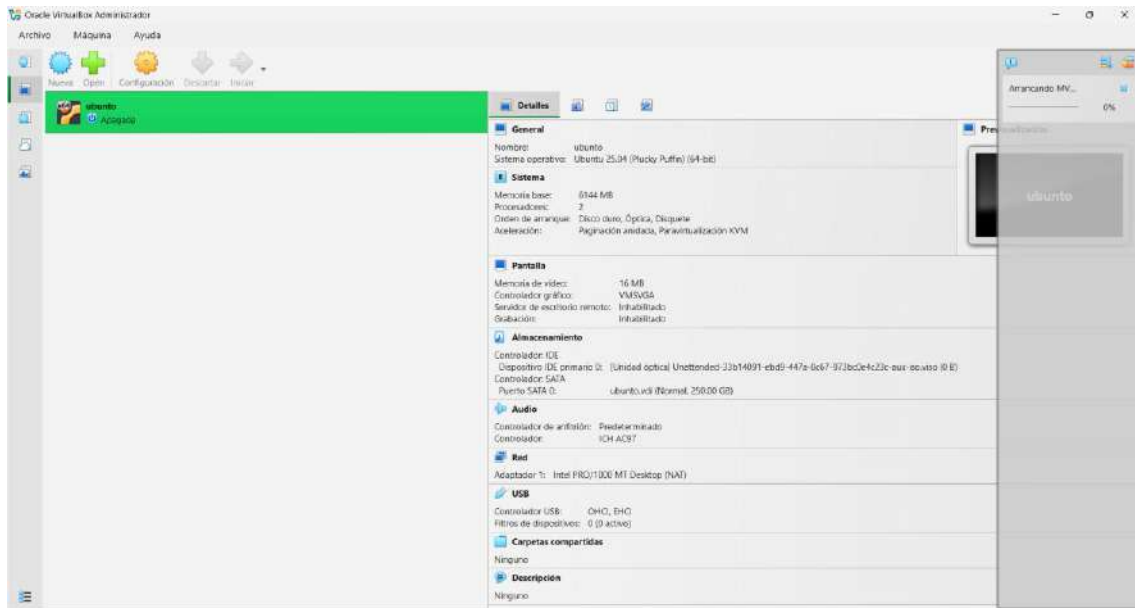
Ahora viene la parte de los recursos, acá es donde le decimos "cuánto de mi computador le prestamos a la máquina virtual":

- ✓ **Memoria base (RAM):** vamos a ver un control deslizante. Como mi Acer Aspire tiene 12 GB de RAM en total, no le demos más de la mitad, ponmos algo como **6144 MB**, que es suficiente para correr Ubuntu + XAMPP sin ahogar tu Windows mientras tanto.
- ✓ **Procesadores:** dejémoslo en 1 o 2 núcleos, no hace falta más para este ejercicio.
- ✓ **Disco duro:** acá el profe fue claro entre **250 y 300 GB**. Como mi SSD tiene 477 GB en total, y ya tenemos Trueque Match y todo mi software instalado ocupando espacio, lo recomendable seria ir por el número más bajo del rango: **250 GB**. Ojo: esto no ocupa 250 GB de una sola vez, es un límite máximo, el archivo crece solo a medida que instalamos cosas adentro.



- ✓ Le damos **Siguiente**, revisamos el resumen que muestra, y si todo está bien, le damos **Finalizar**.



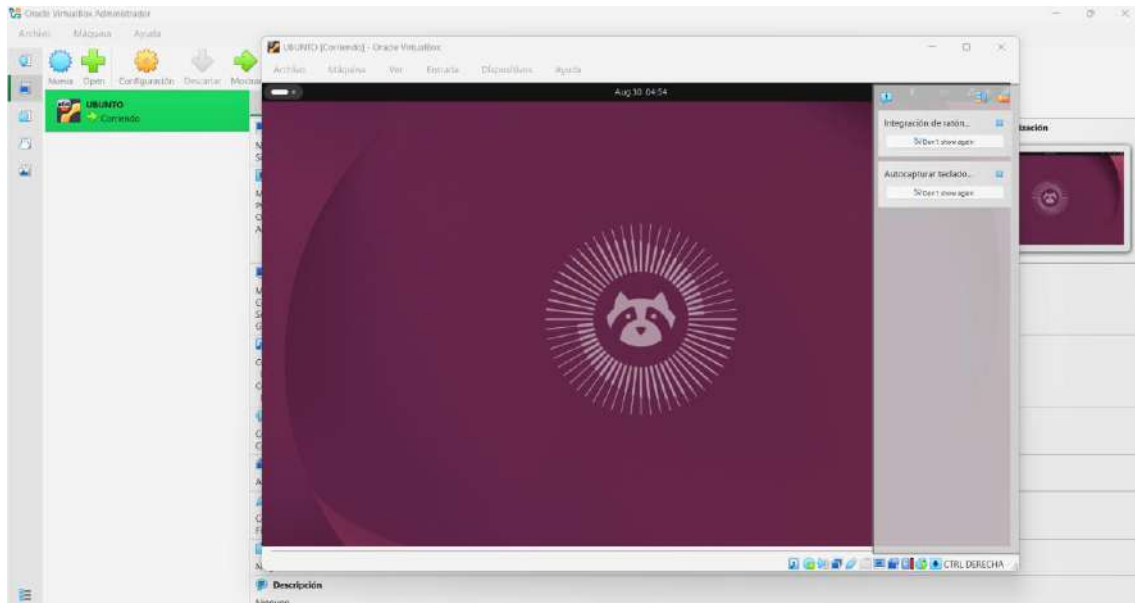


Excelente observamos el resumen: todo quedó configurado tal cual, 6144 MB de RAM, 2 procesadores, 250 GB de disco. Y en la segunda captura vemos **"Corriendo"** en verde, arrancando.

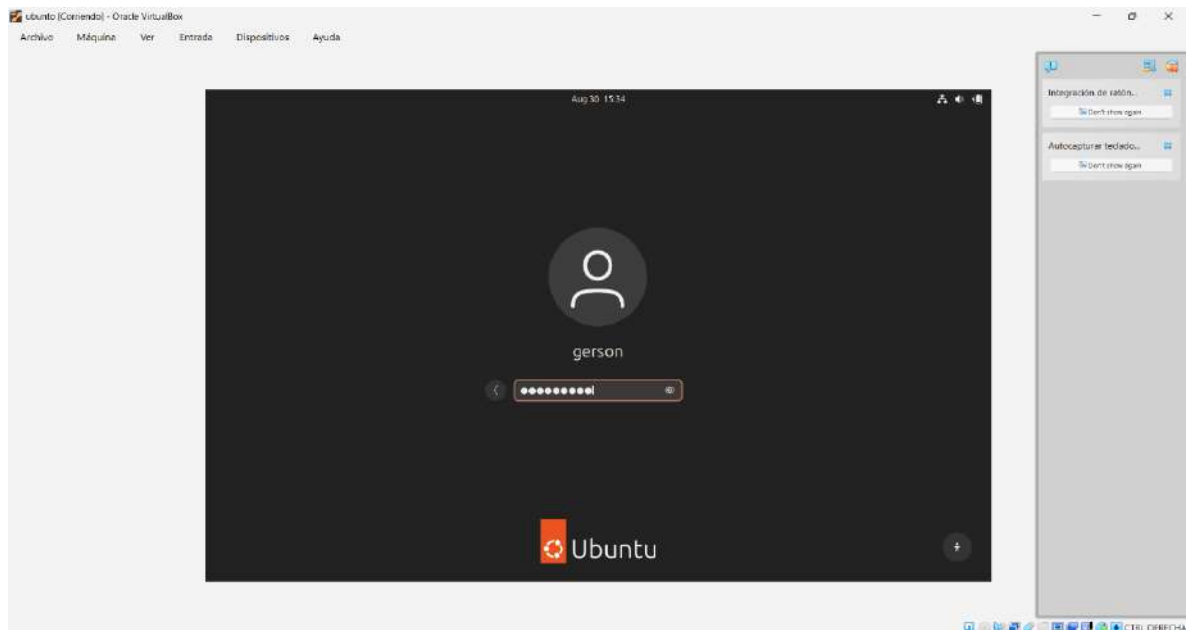
Como marcamos **"Proceed with Unattended Installation: true"**, VirtualBox va a instalar Ubuntu solo, automáticamente, usando el usuario y la contraseña que definiste no vamos a tener que responder preguntas del instalador manualmente. Es como cuando programamos la lavadora y nos va hacer otra cosa mientras termina el ciclo.

Qué hacer ahora?

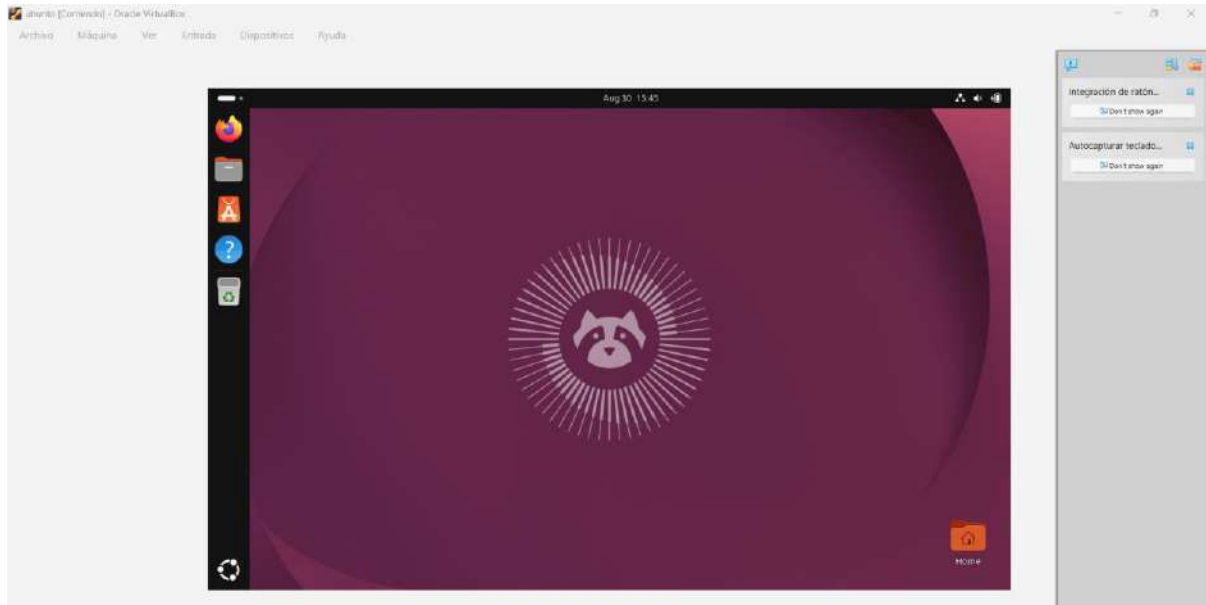
Hacemos doble click sobre el nombre "UBUNTU" en la lista de la izquierda (o click en el botón "Mostrar" arriba) para abrir la ventana completa de la máquina virtual.



- Vamos a ver una pantalla de texto negra corriendo comandos solos, eso es la instalación automática en progreso. **No toques nada, no cierres la ventana**, la dejamos trabajar.
- Este proceso puede tardar entre **10 y 25 minutos**, dependiendo de mi equipo
- La máquina se va a reiniciar sola una o dos veces durante el proceso. Es normal, no hay que asustarse si la pantalla se pone negra un momento o si vemos que reinicia.



- Vamos a saber que terminó cuando lleguemos a una **pantalla de escritorio de Ubuntu** (fondo de color, con íconos, muy similar a como se ve un Windows pero con otro estilo) o a una **pantalla de inicio de sesión** pidiéndote la contraseña que definiste.



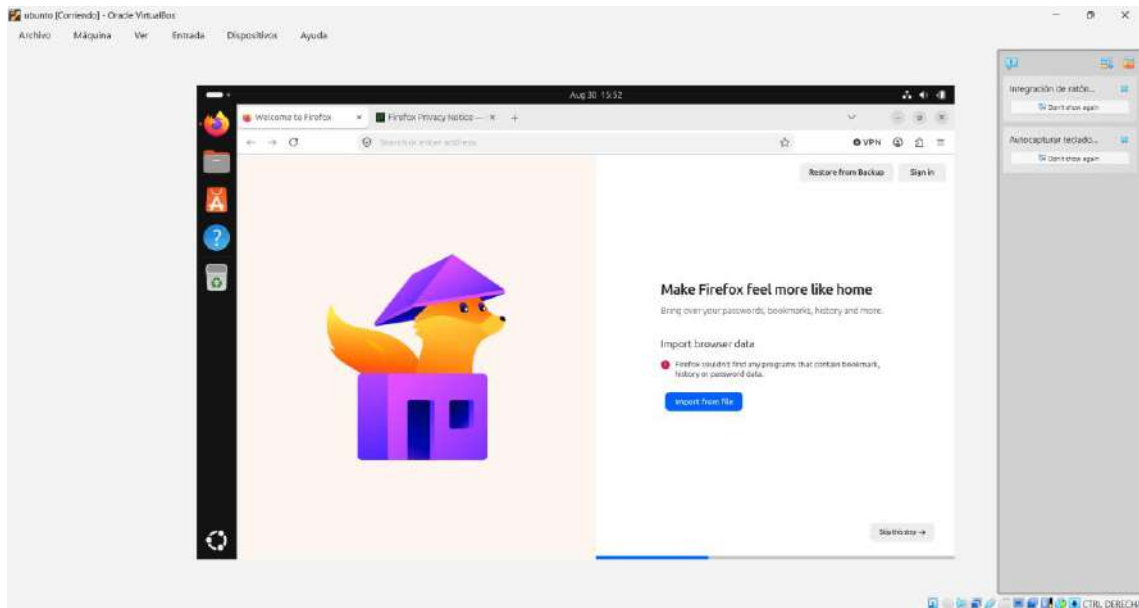
Bien ahí está el escritorio de Ubuntu 26.04 (Resolute Raccoon, el mapachito de la esquina lo confirma) cargando perfecto, con la barra de aplicaciones a la izquierda (Firefox, Archivos, una app naranja, Ayuda, Papelera

- **Paso 4: Instalar XAMPP para Linux**

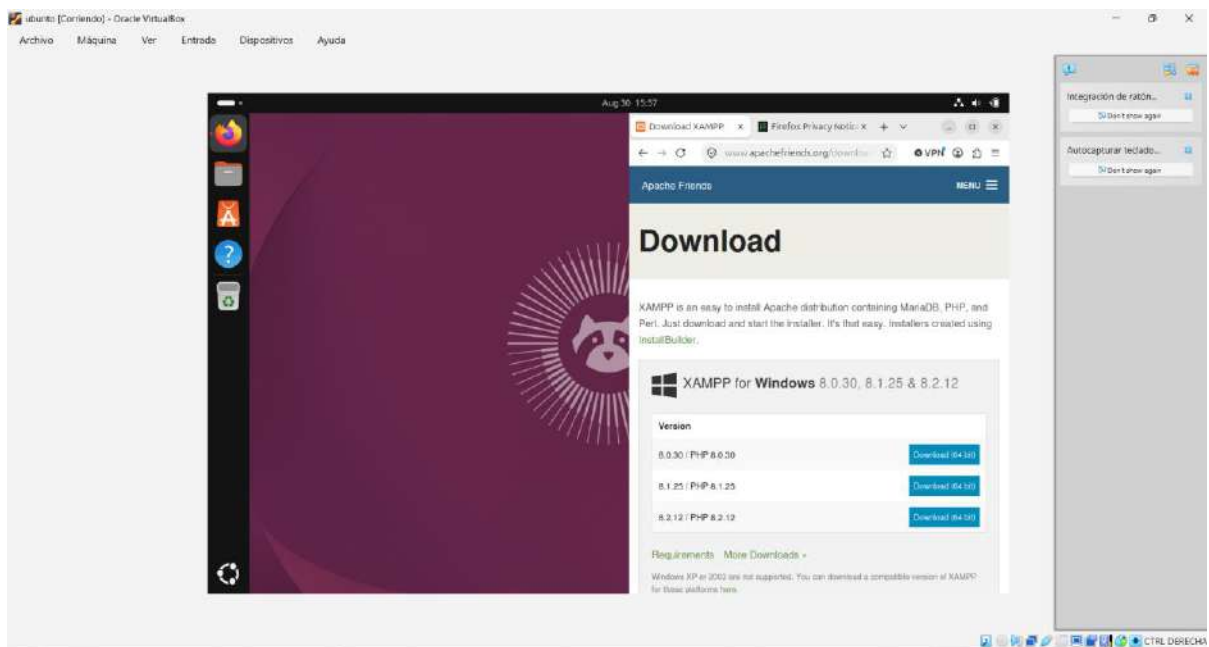
Ahora sí, "instalamos los servicios" (Apache + MySQL) dentro de esta nueva casa Linux.

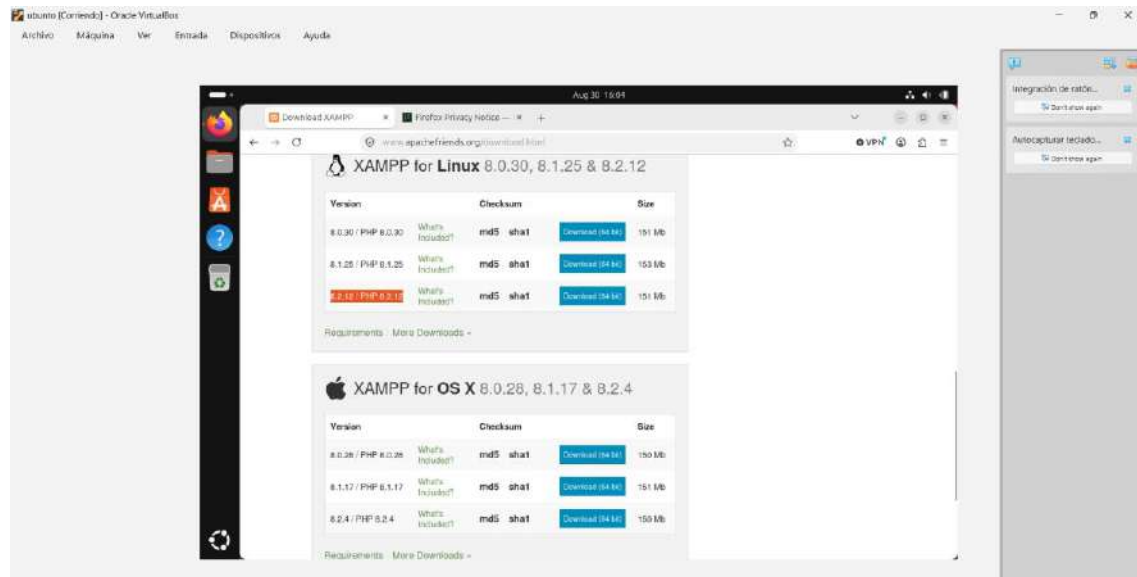
4.1 Descargar XAMPP para Linux (dentro de la máquina virtual)

- Adentro de la ventana de Ubuntu, hacemos **click** en el ícono de **Firefox** (el zorrillo naranja, arriba a la izquierda).



- En la barra de direcciones escribimos:
<https://www.apachefriends.org/download.html>
- Vas a ver tres botones de descarga: Windows, Linux, OS X. Hacé click en el de **Linux**.

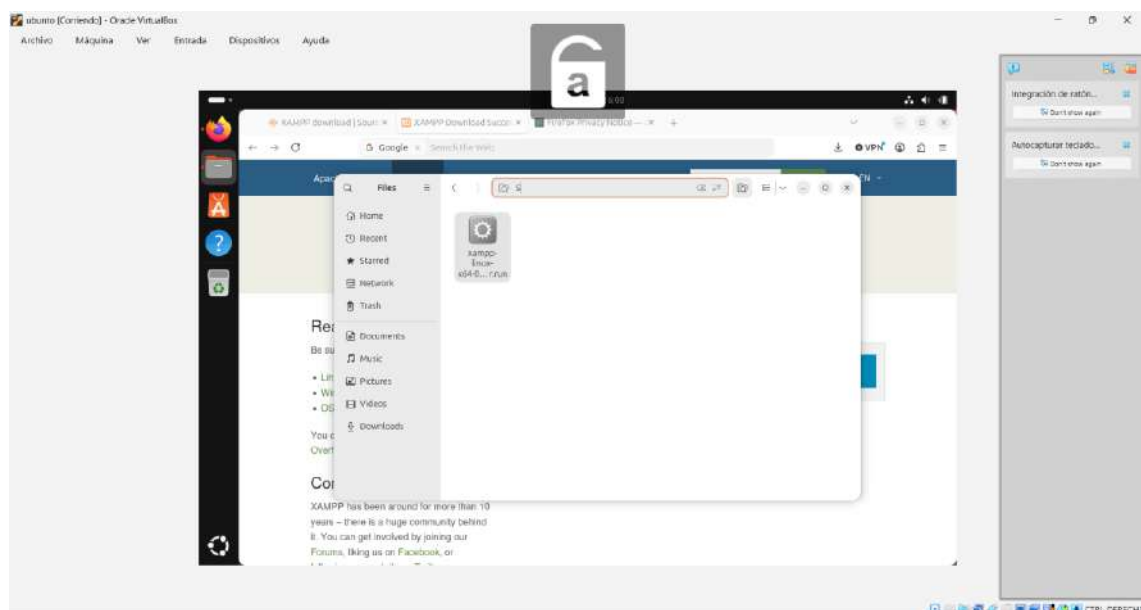




- Se va a descargar un archivo con un nombre como xampp-linux-x64-8.2.XX-**X-installer.run** lo dejamos descargar completo (puede tardar unos minutos).

4.2 Confirmar dónde quedó descargado

- Hacemos **click** en el ícono de **Archivos** (la carpeta gris, segundo ícono de la barra).
- Vamos a la carpeta **Descargas** (Downloads).
- Ahí vemos el archivo **.run** que acabamos de bajar.

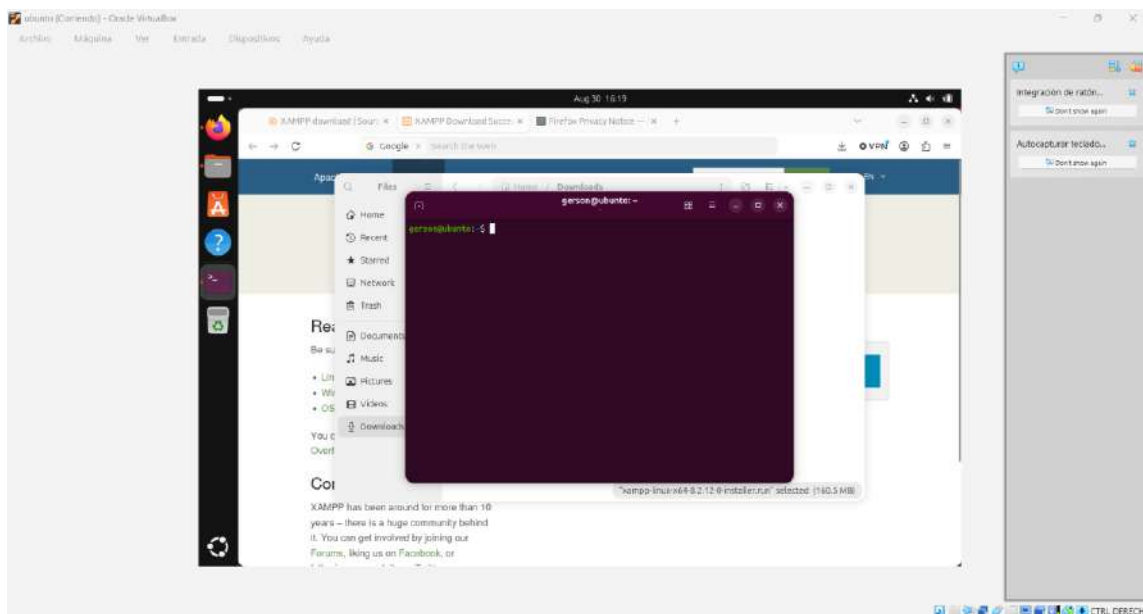


Ahora vamos con la parte de la terminal:

4.3 Abrir una terminal en la carpeta Descargas

Pensemos así: la terminal es como el mostrador de un taller donde le das instrucciones exactas y por escrito al técnico (Linux), en vez de señalar con el mouse como hacemos en Windows. Para instalar software con permisos especiales, casi siempre hay que pasar por ahí.

- Adentro de Ubuntu, abrimos la carpeta **Archivos, Descargas**, donde está tu archivo **.run**.
- Click derecho **dentro de esa carpeta** (en un espacio vacío, no sobre el archivo) buscamos la opción "**Abrir terminal aquí**" (Open Terminal Here). O presionamos **Ctrl + Alt + T**



*(Si no te aparece esa opción, abrimos la terminal desde el menú de aplicaciones y escribimos **cd Descargas** y **Enter**, que significa "muevete a la carpeta Descargas".)*

Vamos a ver una ventana negra con texto, esa es la terminal, ya ubicada en la misma carpeta donde está tu archivo.

4.4 Dar permisos de ejecución al archivo

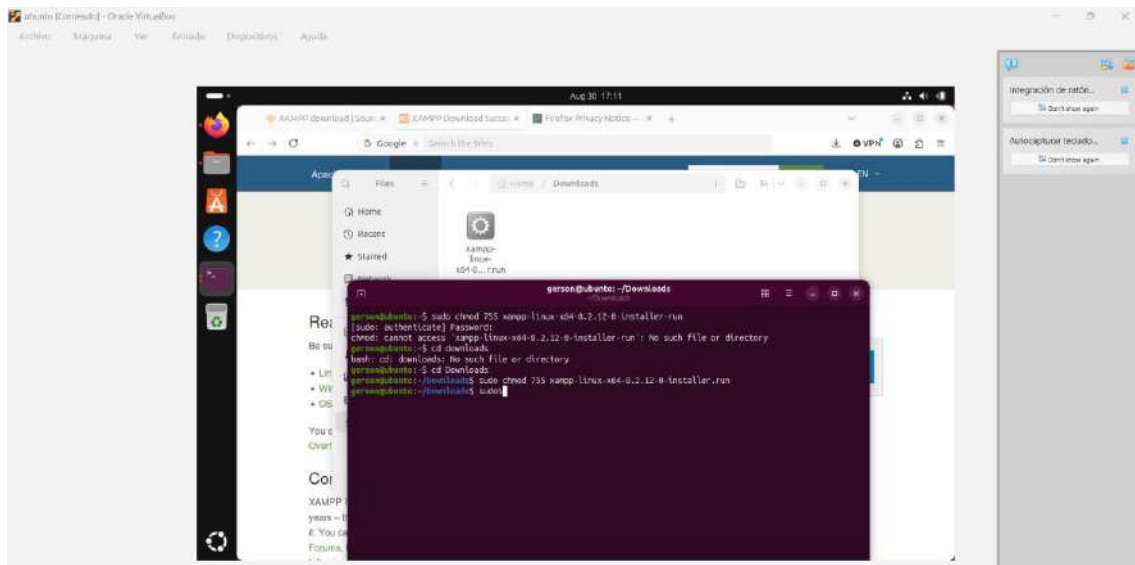
En Linux, por seguridad, un archivo recién descargado no tiene permiso de ejecutarse como programa hasta que se lo demos explícitamente es como si el archivo llegara "con el seguro puesto" y tuviéramos que quitárselo antes de poder usarlo.

Escribimos este comando exacto en la terminal (podés copiarlo y pegarlo con click derecho → Pegar, ya que Ctrl+V a veces no funciona igual dentro de la terminal de Linux):

`sudo chmod 755 xampp-linux-x64-8.2.12-0-installer.run`

Nos va a pedir la contraseña de tu usuario de Ubuntu (la que definiste al crear la máquina) la escribimos y damos Enter. Ojo: en la terminal de Linux, cuando escribimos la contraseña, no vamos a ver ni puntitos ni nada en pantalla, es normal, está registrando lo que escribimos igual, aunque parezca que no pasa nada. La escribimos completa y le damos Enter con confianza.

Si no nos tira ningún error después de eso, ese permiso ya quedó dado.



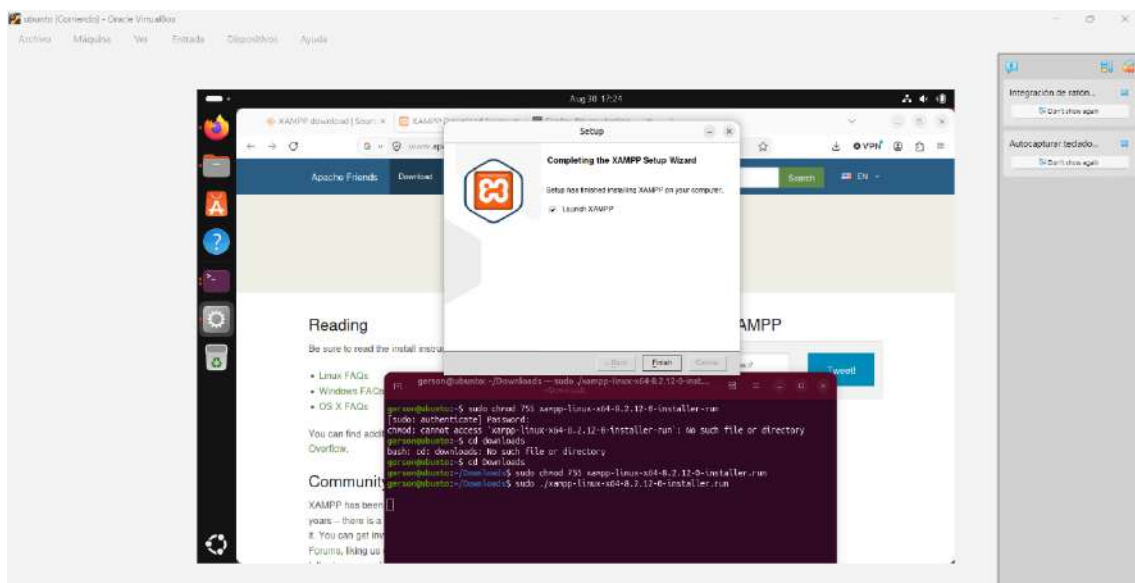
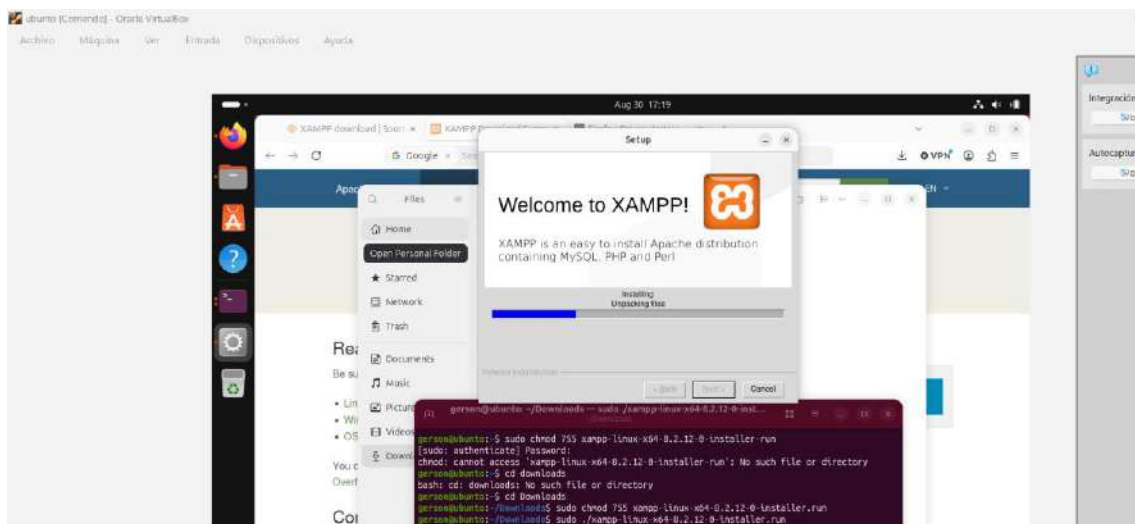
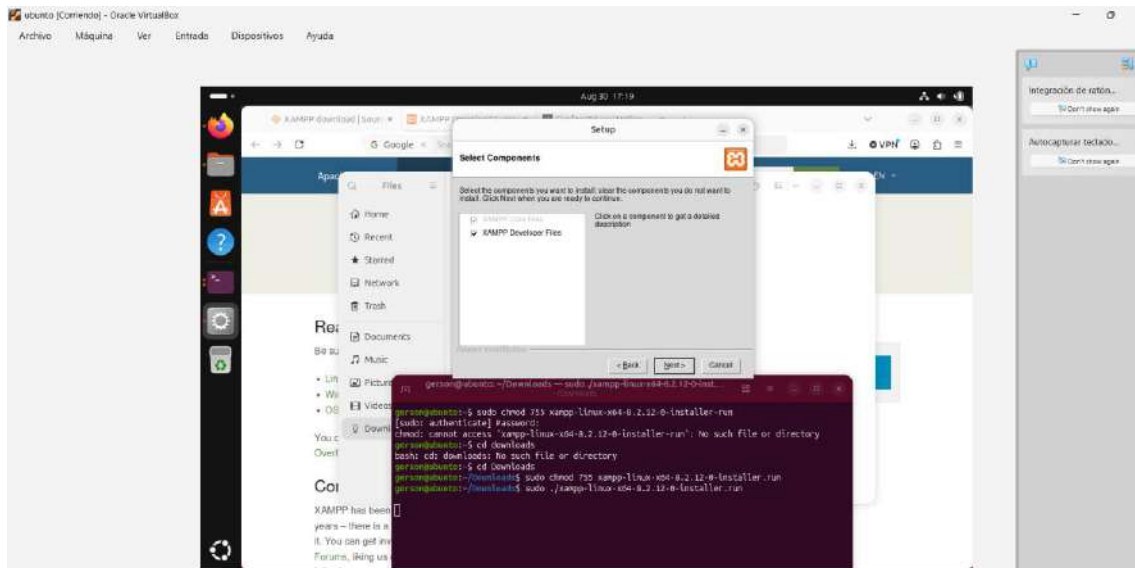
4.5 Ejecutar el instalador

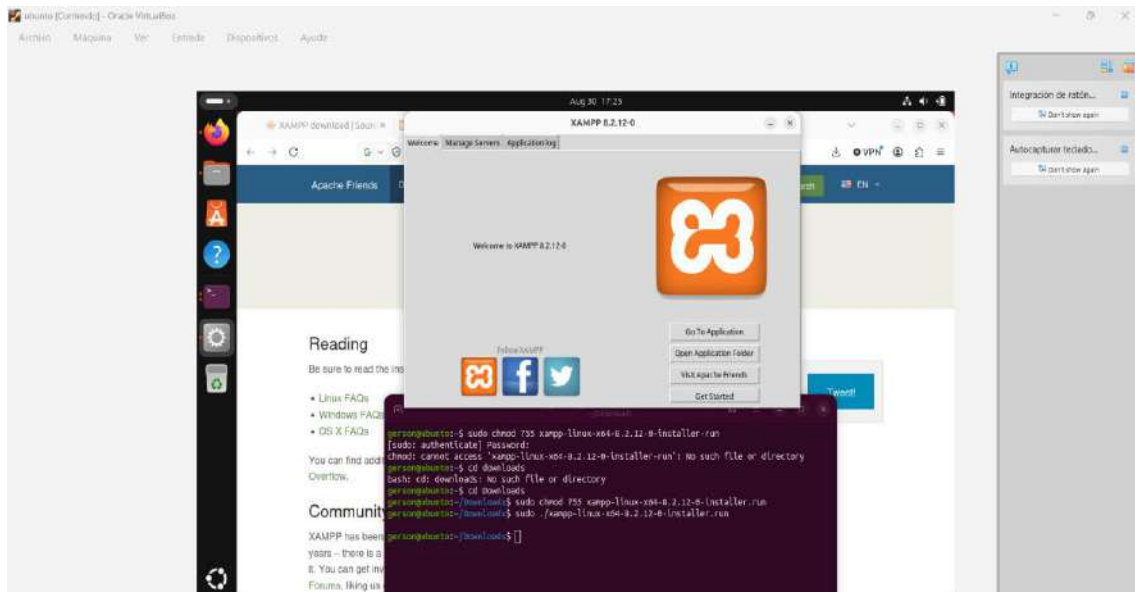
Ahora sí, el comando que arranca la instalación de verdad:

`sudo ./xampp-linux-x64-8.2.12-0-installer.run`

Damos Enter. Esto va a abrir una ventana gráfica (no todo es texto, tranquilo) con el asistente de instalación de XAMPP, muy parecido al que ya conocemos de Windows, Next, Next, Next.

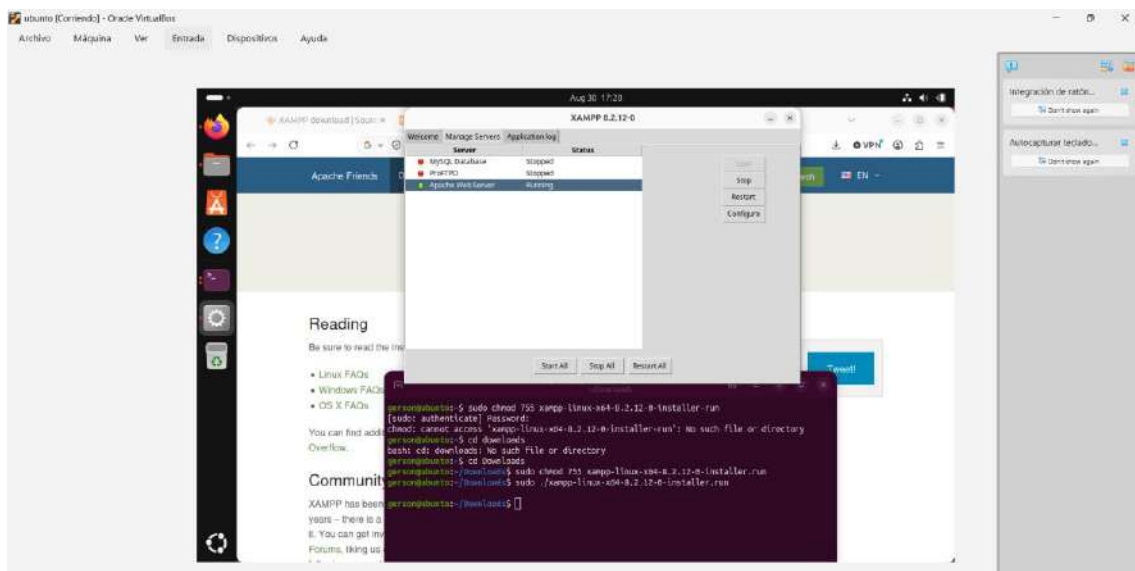
MODELO RELACIONAL TRUEQUE MATCH Y MODELO ENTIDAD RELACIÓN





Confirmar dos cosas antes de seguir:

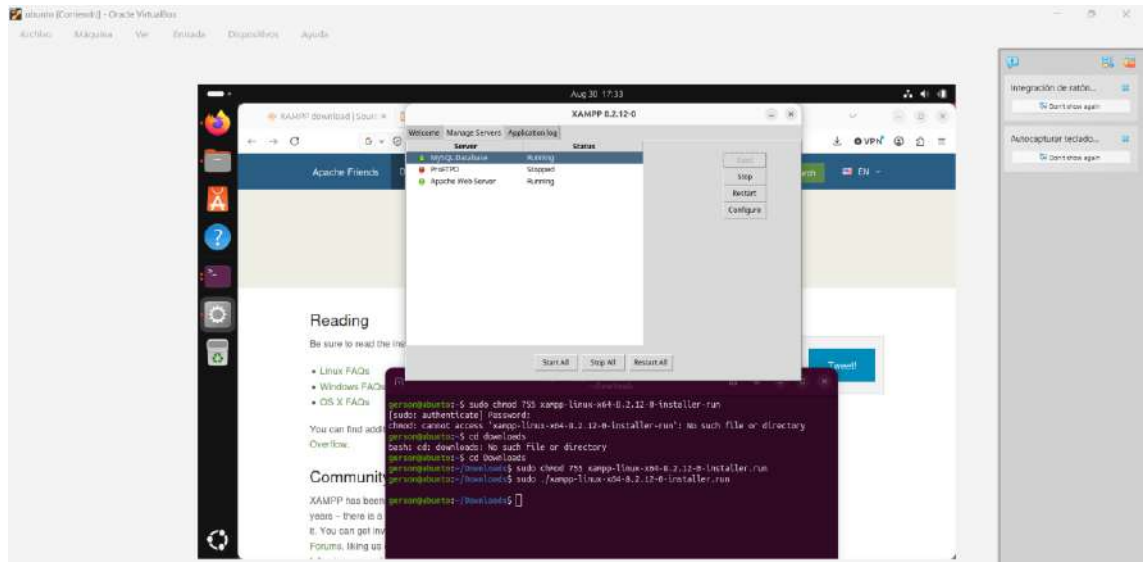
Que Apache y MySQL aparezcan ahí como servicios disponibles para iniciar (en Linux, XAMPP se instala en una carpeta que se llama /opt/lampp, así que el panel puede llamarse "LAMPP Manager" o algo parecido, no nos debemos sorprender si no dice "XAMPP" en el título, es el mismo programa).



- Que los servicios estén efectivamente **corriendo** (running / iniciado), no solo instalados.

Si el panel gráfico no se abrió solo, no hay problema, desde la misma terminal donde ya estabamos, este comando levanta los servicios directamente:

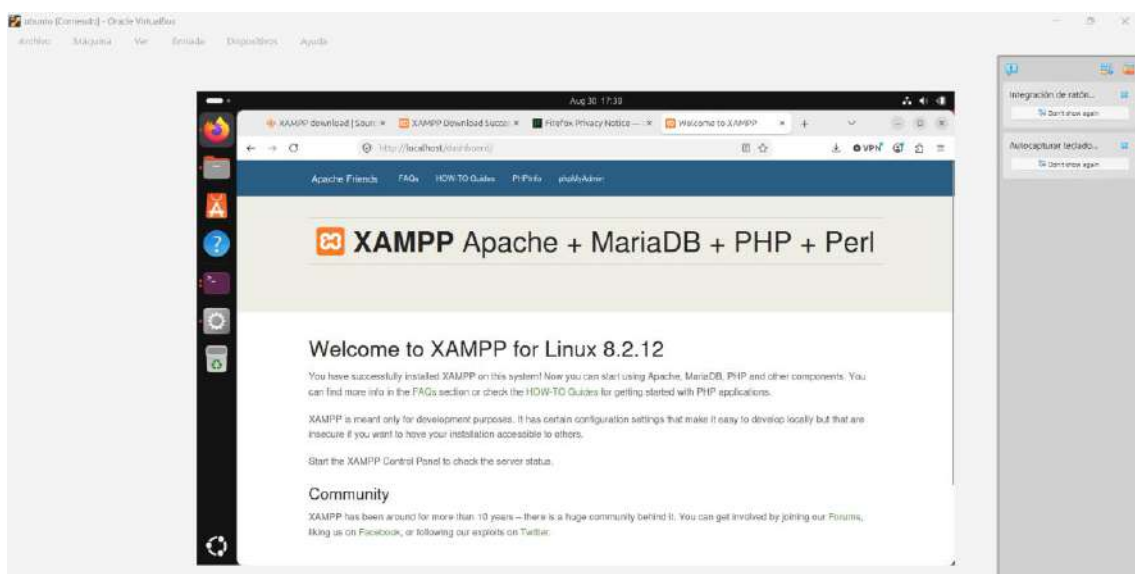
`sudo /opt/lampp/lampp start`



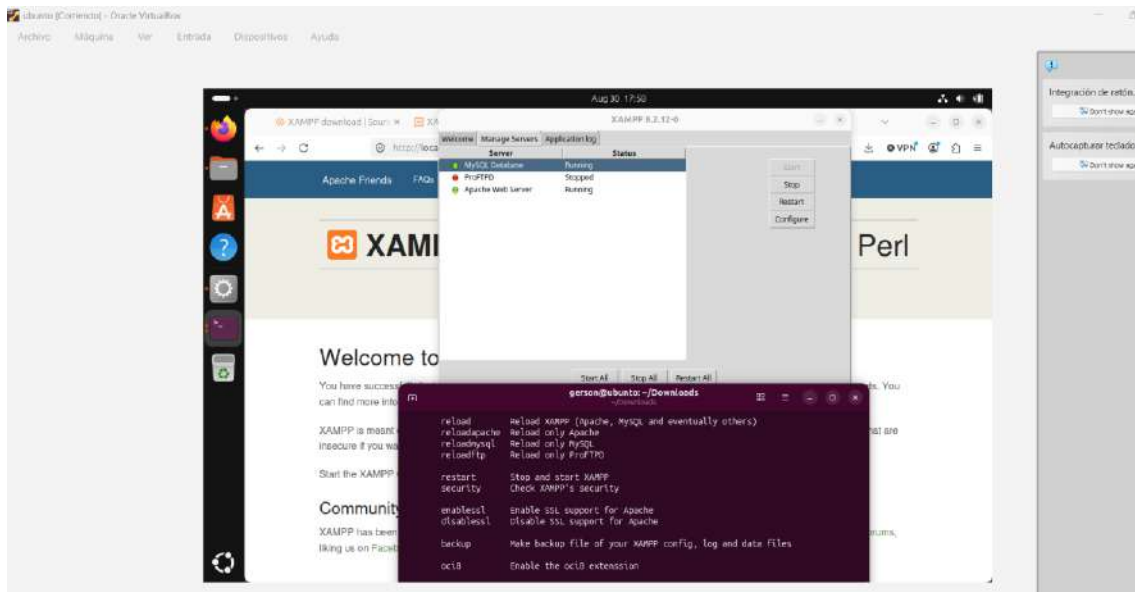
Ahora vamos a comprobar que el navegador de Ubuntu responde al servidor local y a dejar listo el terreno para tu proyecto.

1. Probar en el navegador

- Abrimos una nueva pestaña en el Firefox de Ubuntu.
- Escribimos `http://localhost` (o simplemente `localhost`) y le damos **Enter**.
- Nos va a que cargar la pantalla de bienvenida oficial de XAMPP (Dashboard).



- **Ubicar la carpeta del proyecto (htdocs)** En Linux, los archivos de mi página web no van en *Mis Documentos*, sino en la carpeta raíz que XAMPP administra:
/opt/lampp/htdocs.



Todo está impecable, Ese mensaje describe exactamente la situación en la que me encuentro: la infraestructura base está montada y funcionando al 100%.

Es la confirmación exacta de que pasamos la etapa crítica de configuración del entorno en Ubuntu. No hay errores ni bloqueos; estás listo para la fase de despliegue.

- ✓ **MySQL Database Running ok.**
- ✓ **Apache Web Server: Running ok.** (ProFTPD quedó "Stopped")
- ✓ El navegador Firefox ya está mostrando la página de bienvenida de XAMPP en <http://localhost>, eso confirma que Apache ya está sirviendo páginas de verdad. (panel de control + navegador confirmando)

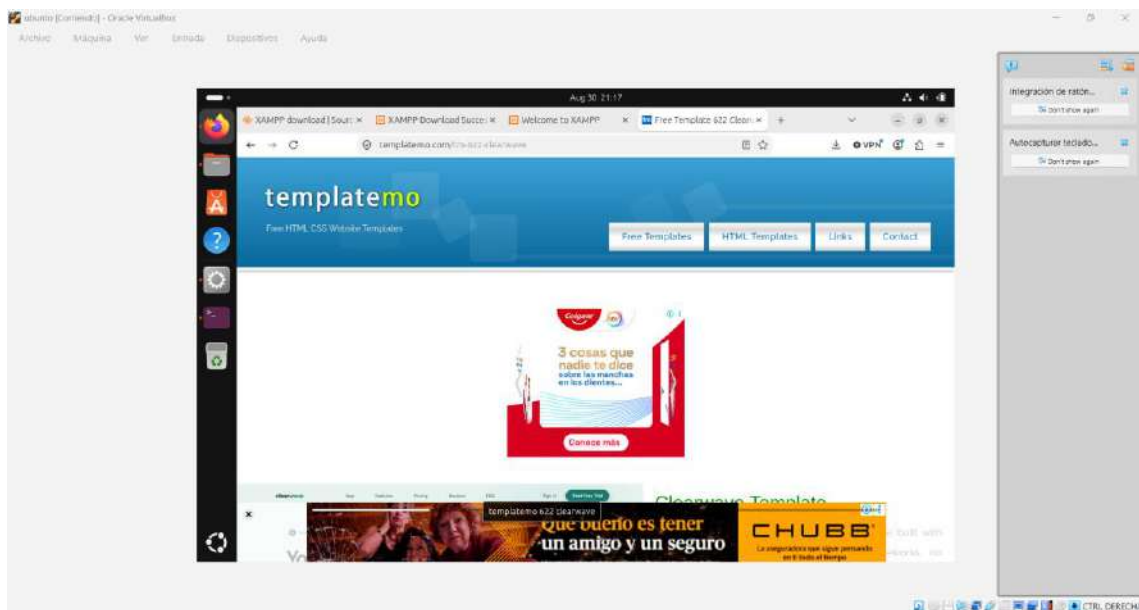
seguimos con el último paso: poner una página de prueba adentro de htdocs y verla cargar en el navegador de Ubuntu.

Poner una página de prueba en [htdocs](#) (dentro de Ubuntu)

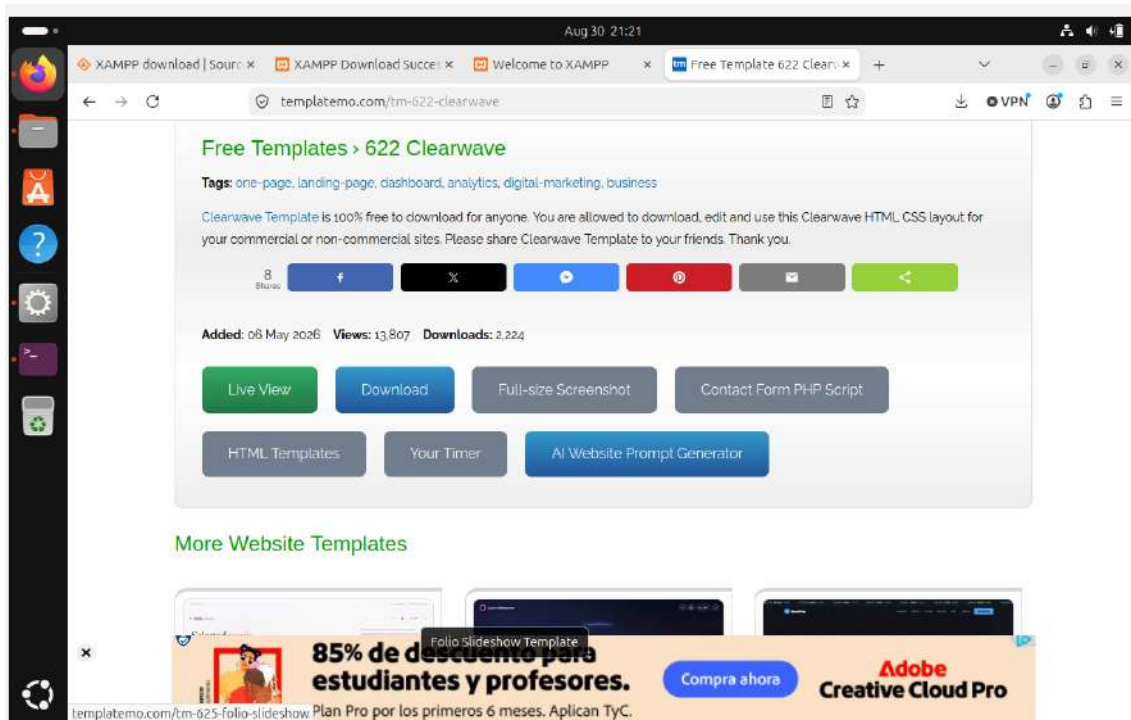
Ahora repetimos la misma idea que ya hicimos en Windows con la plantilla Clearwave, pero esta vez adentro de Linux. Acá la carpeta htdocs no está en "C:\xampp\htdocs" como en Windows, en Linux vive en /opt/lampp/htdocs.

3.1 Descargar una plantilla (podemos reusar la misma, Clearwave)

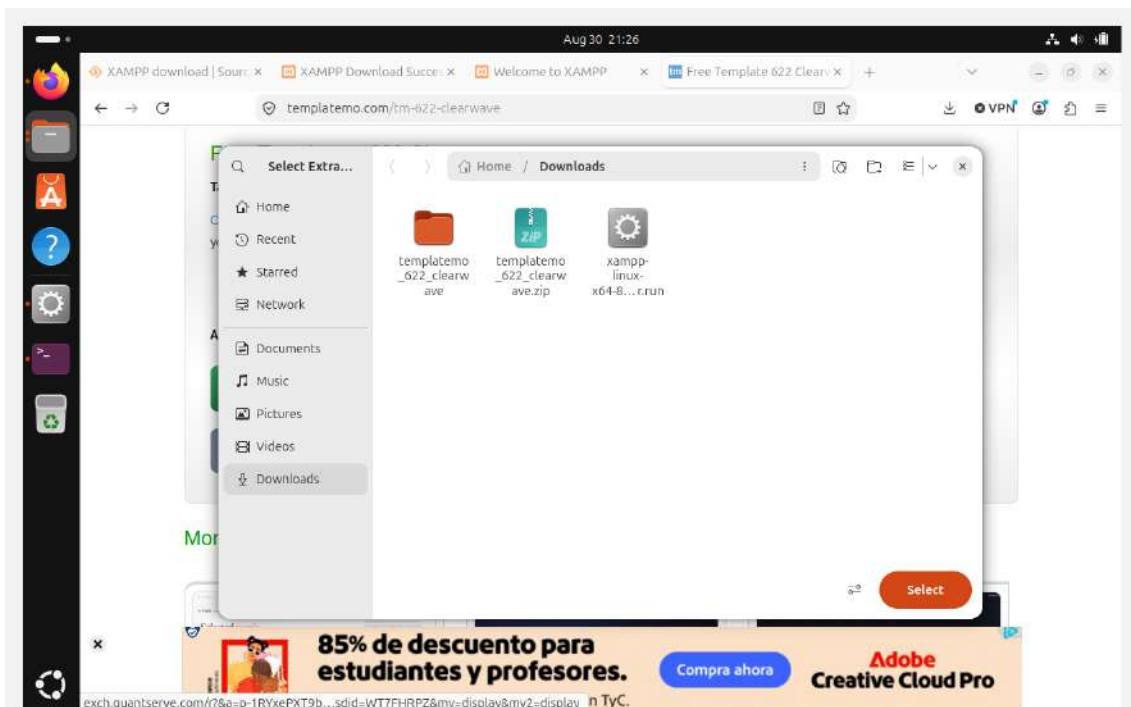
- En el Firefox de Ubuntu, vamos a <https://templatemo.com/tm-622-clearwave> (la misma que usamos en la otra evidencia).



- Bajamos hasta encontrar un botón verde que dice "Download". Hacemos **click** ahí.



- Descargamos él **.zip**.
- vamos a **Archivos, Descargas**, click derecho sobre él **.zip**, "Extraer aquí" (Extract Here).

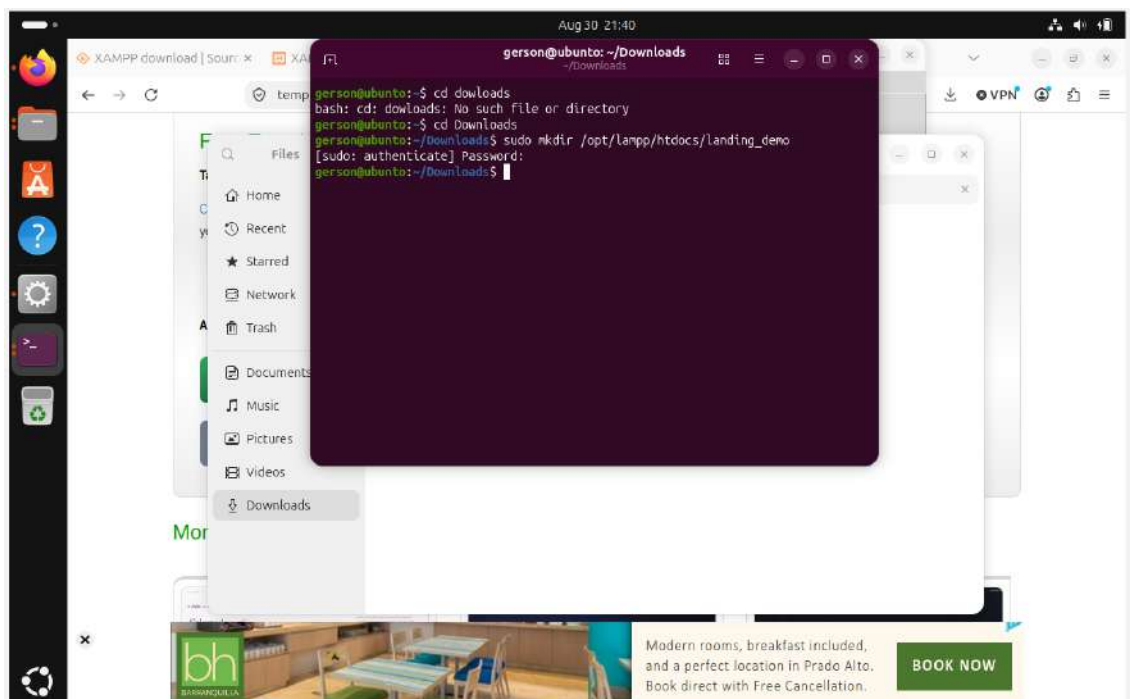


3.2 Mover el contenido a `/opt/lampp/htdocs`

Acá viene un detalle importante que mencionó el profe: **en Linux, la carpeta `htdocs` le pertenece al usuario "root" (el administrador del sistema)**, así que no voy a poder simplemente arrastrar y soltar los archivos como en Windows, hay que moverlos por la terminal, con permisos especiales, igual que hicimos con el instalador de XAMPP.

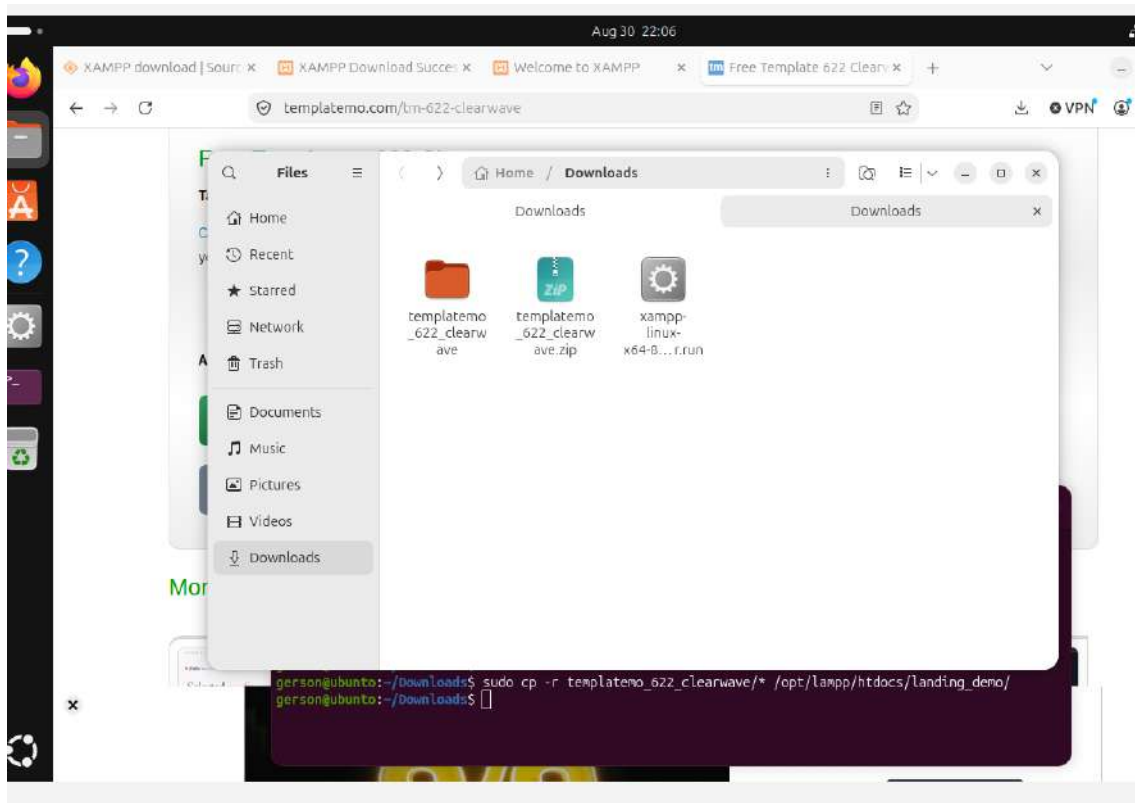
- Abrí una terminal en la carpeta Descargas (mismo truco de antes: click derecho "Abrir terminal aquí").
- Primero, creamos una carpeta nueva dentro de `htdocs` para nuestra plantilla, con este comando:

`sudo mkdir /opt/lampp/htdocs/landing_demo`



- Ahora copiamos todo el contenido de la carpeta que descomprimiste hacia esa carpeta nueva. Si la carpeta que se descomprimió se llama, por ejemplo, `templatemo_622_clearwave`, el comando sería:

`sudo cp -r templatemo_622_clearwave/* /opt/lampp/htdocs/landing_demo/`

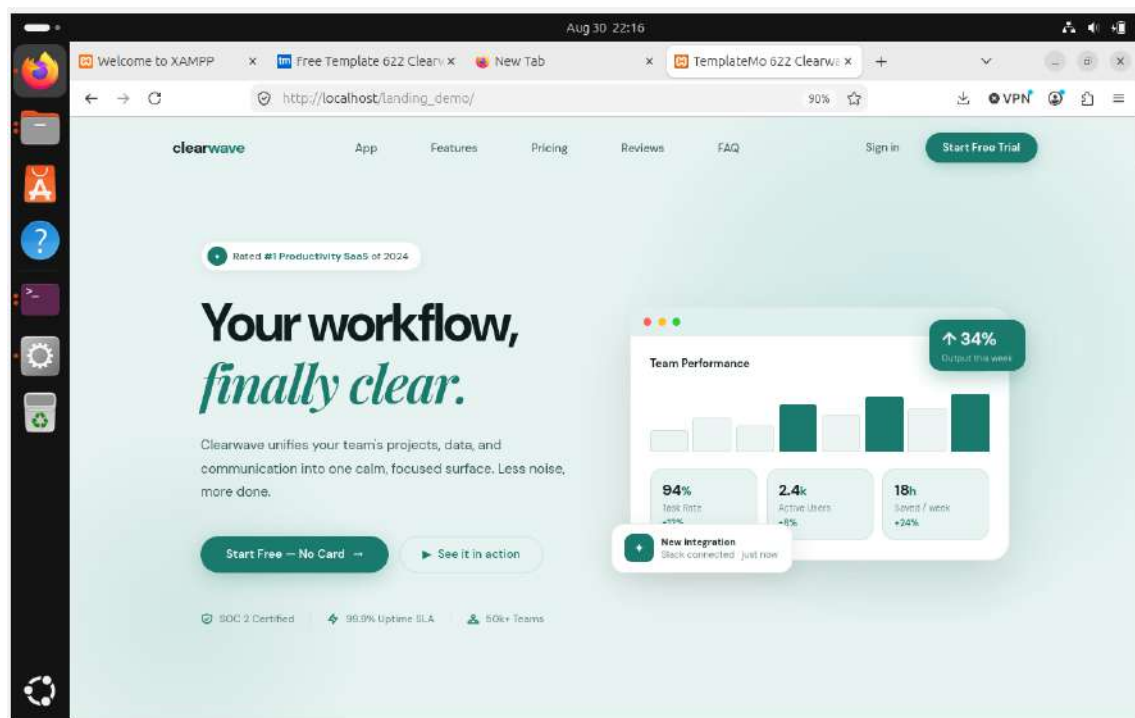
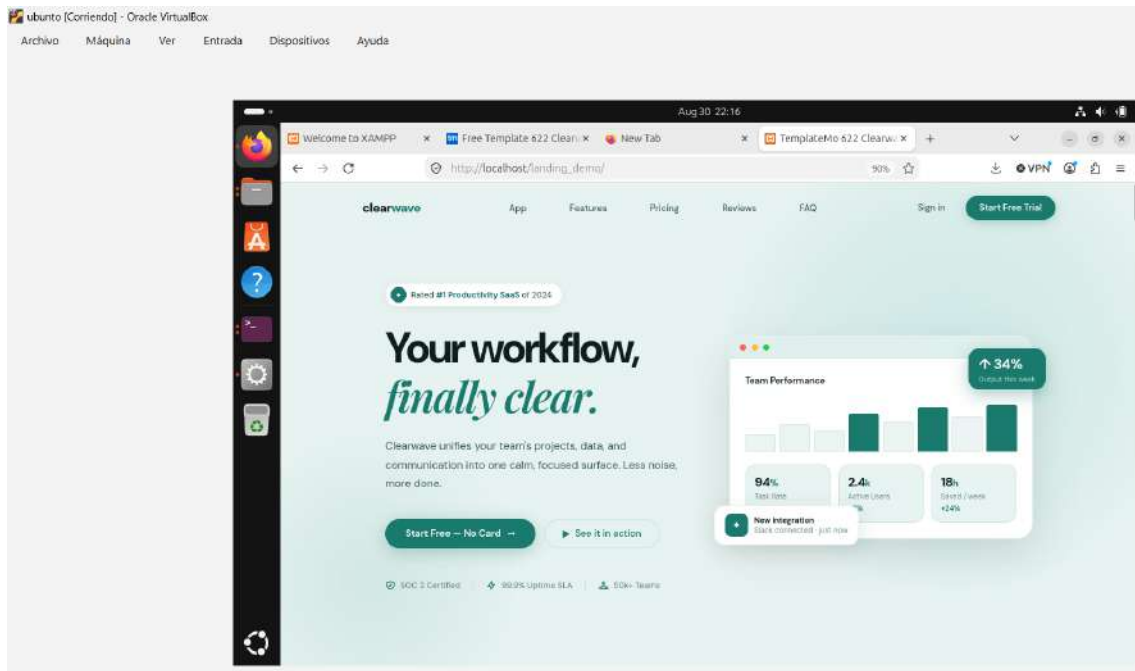


(Explicamos qué hace cada parte: **sudo** = "hacemos con permisos de administrador", **cp** = copiar, **-r** = "copiamos también las subcarpetas de adentro, no solo los archivos sueltos", el ***** al final del nombre de la carpeta significa "todo lo que está adentro".)

- Nos va a pedir la contraseña de nuevo, la escribimos (recordamos, no se ve nada en pantalla, es normal) y Enter.

3.3 Verlo funcionando

- Abrí Firefox (adentro de Ubuntu).
- Andá a: http://localhost/landing_demo/



Carga exitosa, cerramos el ciclo completo: sistema operativo **Linux** + **Apache** + **MySQL** corriendo + **plantilla web desplegada y visible**.

Cargó exitosamente [en http://localhost/landing_demo/](http://localhost/landing_demo/).

La plantilla Clearwave cargando completa, con todos los estilos, adentro de tu máquina Ubuntu, sirviendo desde Apache vía `/opt/lampp/htdocs/landing_demo/`. Esa

Conclusión: ya tenemos las 4 evidencias completas de la parte de virtualización

#	Evidencia	Estado
1	VirtualBox instalado	Ok.
2	Ubuntu 26.04 corriendo (escritorio visible)	Ok.
3	XAMPP para Linux instalado, Apache + MySQL "Running"	Ok.
4	Plantilla web desplegada y cargando en localhost	Ok.

DOCKER Contenedores

VirtualBox (Virtualización): Es como construir un edificio completo (con cimientos, tuberías, sistema eléctrico y vigilante) solo para abrir una tienda. Cada máquina virtual lleva su propio sistema operativo pesado cargado en memoria.

Docker (Contenedores): Es como ubicar un local prefabricado totalmente equipado dentro de un centro comercial existente. Utiliza los mismos recursos y el mismo motor del sistema operativo base (**Linux**), por lo que se enciende en un par de segundos y consume muy poca memoria RAM.

4.1 Paso a Paso: Despliegue con Docker en Ubuntu

Ejecutamos 3 pasos en orden en la terminal de **Ubuntu**:

- PASO 1: Actualizar e instalar Docker:**

```
sudo apt update && sudo apt install docker.io -y
```

Esto tarda unos minutos, espera que termine solo.

- PASO 2: Lanzar Apache en contenedor Docker:**

```
sudo docker run -d -p 8080:80 --name mi_servidor_docker httpd
```

- **PASO 3: Verificar que corre:**

`sudo docker ps`

Debemos observar `mi_servidor_docker` en estado **Up**.

```

person@ubuntu: ~/Downloads
debconf: unable to initialize frontend: Dialog
debconf: (Dialog frontend requires a screen at least 13 lines tall and 31 columns wide.)
debconf: falling back to frontend: Readline
Setting up pizr (2.0-1build1)...
Setting up git-nan (1:2.53.0-1ubuntu1)...
Setting up containerd (2.2.2-0ubuntu1.1)...
Created symlink '/etc/systemd/system/multi-user.target.wants/containerd.service' -> '/usr/lib/systemd/system/containerd.service'.
Setting up ubuntu-fan (0.12.17)...
Created symlink '/etc/systemd/system/multi-user.target.wants/ubuntu-fan.service' -> '/usr/lib/systemd/system/ubuntu-fan.service'.
Setting up docker.io (20.10.13-0ubuntu1.1)...
debconf: unable to initialize frontend: Dialog
debconf: (Dialog frontend requires a screen at least 13 lines tall and 31 columns wide.)
debconf: falling back to frontend: Readline
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' -> '/usr/lib/systemd/system/docker.service'.
Created symlink '/etc/systemd/system/sockets.target.wants/docker.socket' -> '/usr/lib/systemd/system/docker.socket'.
Setting up git (1:2.53.0-1ubuntu1)...
Processing triggers for non-db (2.13.1-1build1)...
person@ubuntu:~/Downloads$ sudo docker run -d -p 8080:80 --name mi_servidor_docker httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
0ff80e2b04b: Pull complete
0e0e0e0b7b17: Pull complete
af224c00a0b: Pull complete
4f4b700ef54: Pull complete
0de3897443b2: Pull complete
6310eb10bf42: Pull complete
d510e7b301a1: Download complete
2a40a000f0b5: Download complete
Digest: sha256:1979c38c228d28c2edf043c6e27dcee1c7b4a101a5526721a00e454609e99e
Status: Downloaded newer image for httpd:latest
2b92118d046e26e025e66504cae59e0407107c0ac5ecf20a396c0ecb9fe3ad
person@ubuntu:~/Downloads$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
2b92118d046e   httpd     "httpd -foreground"      About a minute ago    Up About a minute    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    mi_servidor_docker
  
```

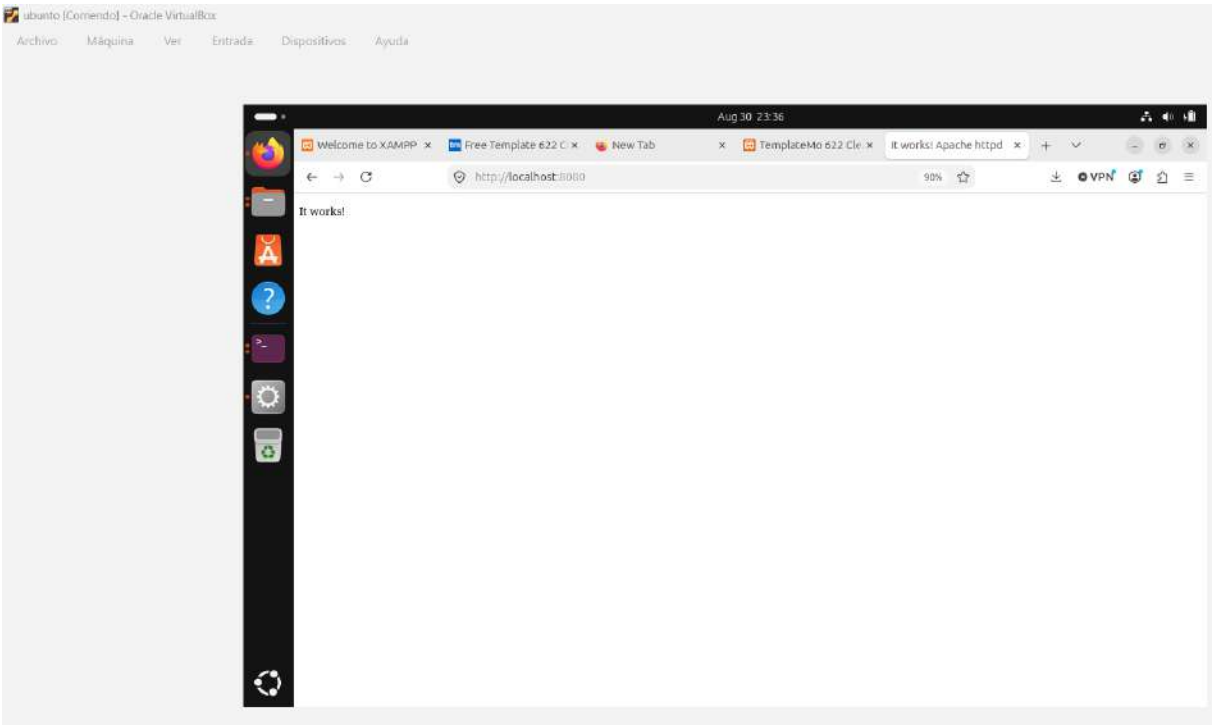
```

person@ubuntu:~/Downloads$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                               NAMES
2b92118d046e   httpd     "httpd -foreground"      About a minute ago    Up About a minute    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    mi_servidor_docker
person@ubuntu:~/Downloads$
  
```

- **PASO 4: Abrir Firefox en Ubuntu y entrar a:**

<http://localhost:8080>

Veremos como resultado: **"It works!"**



Con esto la evidencia **GA10-AA5-EV01** queda completa b:

#	Evidencia	Estado
1	VirtualBox instalado	Ok.
2	Ubuntu corriendo	Ok.
3	XAMPP Apache + MySQL Running	Ok.
4	Plantilla web en localhost	Ok.
5	Docker con Apache en puerto 8080	Ok.

Evidencia GA10-220501097-AA6-EV01. Archivos cargados en la plataforma de producción.

Plantilla web vs. sitio web (y qué cambia con la evidencia anterior)

La plantilla web es solo el "material de construcción", un conjunto de archivos HTML, CSS y JavaScript ya armados por otra persona (como Clearwave, la que ya usamos dos veces). Es como comprar los planos y los materiales prefabricados de una casa: existe, está completa, pero todavía no está parada en ningún terreno donde alguien pueda llegar y tocarle el timbre.

Un sitio web es esa misma plantilla, pero ya **instalada en un servidor real, conectado a Internet, con una dirección (dominio) a la que cualquier persona del mundo puede escribir en su navegador y entrar**, sin importar si esa persona está en la misma casa o del otro lado del planeta.

Ahí está la diferencia clave con lo que ya hicimos hasta ahora:

	Lo que ya hicimos (XAMPP Windows y XAMPP en la VM Linux)	Lo que vamos a hacer ahora (AA6-EV01)
¿Dónde vive la página?	Dentro de tu propio computador (localhost)	En un servidor de verdad, en Internet
¿Quién puede entrar?	Solo vos, desde esa misma máquina	Cualquier persona, desde cualquier lugar
¿Tiene una dirección web real?	No: localhost no significa nada fuera de tu PC	Sí: un dominio real tipo algo.netlify.app
¿Es un "sitio web" técnicamente?	No, es un despliegue local de prueba	Sí, ya es un sitio web publicado

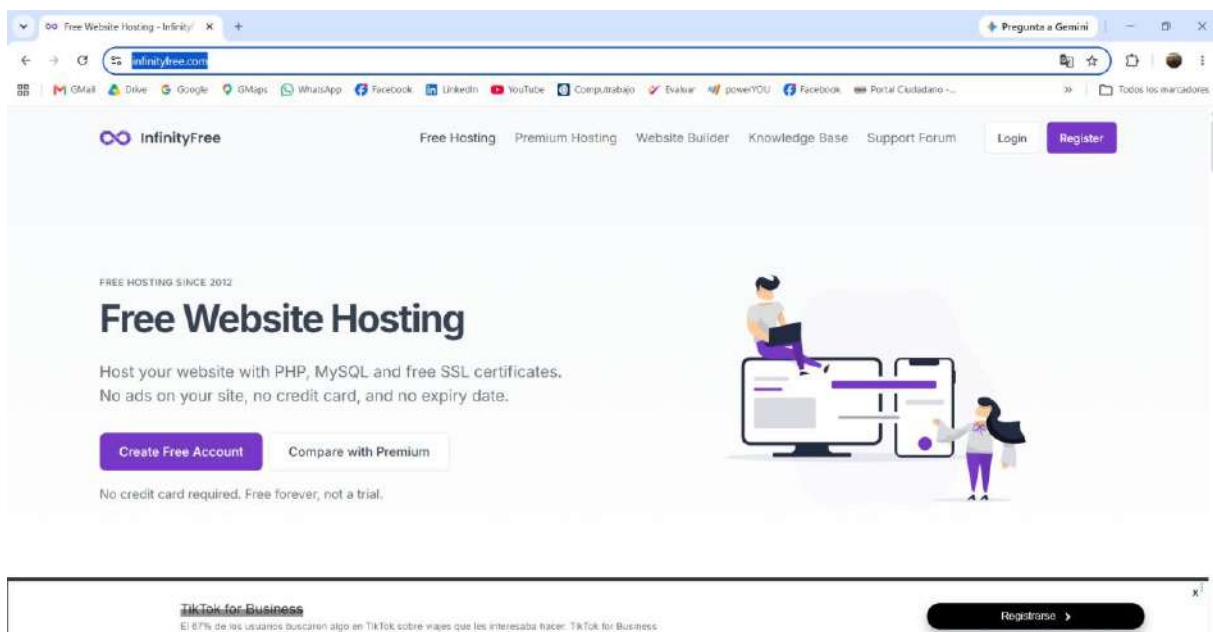
Es la diferencia entre ensayar una obra de teatro en tu living (nadie de afuera te ve) y presentarla en un teatro real con la puerta abierta al público.

La idea con la analogía, para conectar con lo que ya se conoce

Ya observamos la diferencia entre plantilla y sitio web. Ahora, XAMPP y VirtualBox eran como construir en **tu propio patio trasero**, nadie de afuera podía entrar. **InfinityFree es un terreno público real, en Internet**, con una dirección oficial (el dominio) a la que cualquiera puede llegar. Lo único que cambia es dónde ponemos la casa, no cómo la construimos, la carpeta sigue llamándose **htdocs**, igual que en XAMPP, así que ya sabemos cómo pensar esta parte.

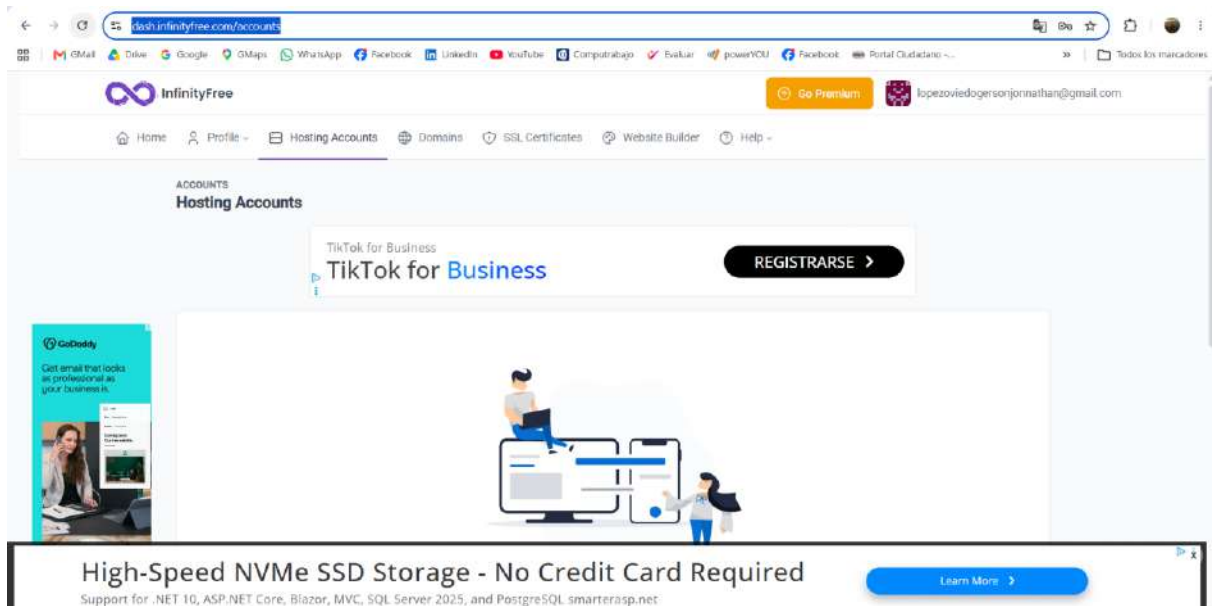
Paso 1: Crear tu cuenta en InfinityFree

- Nos dirigimos a <https://infinityfree.com/> (o **.net**, cualquiera de los dos te lleva al mismo sitio).



- Buscamos el botón de **registro** ("Sign Up" o "Register").
- Elegimos cómo registrarnos, podemos usar mi cuenta de **Google**, de **GitHub**, o simplemente un **correo + contraseña**. Cualquiera sirve, me decidí por **GitHub**.
- Aceptamos los términos y confirmamos el registro.

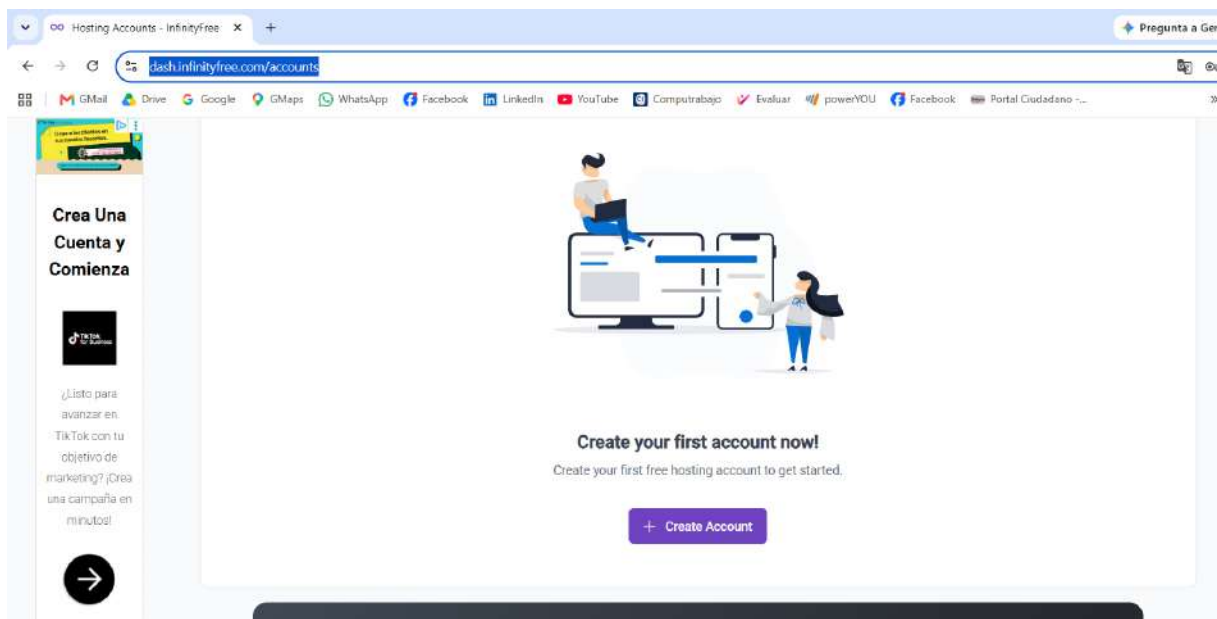
- Nos va a llegar un correo de confirmación lo abrimosy confirmá la cuenta desde ahí.
- Iniciá sesión con los datos que acabás de crear.



(Ojo con un límite: la cuenta gratuita te deja crear **máximo 3 sitios**)

Paso 2: Crear mi primer sitio (subdominio gratis)

1. En esa misma pantalla, buscamos un botón que diga algo como **"Create Account"** o **"Nueva cuenta"** (suele estar más abajo en la página, a veces hay que scrolllear un poco, porque arriba solo hay publicidad).



- Te va a aparecer un formulario pidiéndote:
 - ✓ **Nombre del subdominio:** acá escojo el nombre, por ejemplo **truequematch-gerson** o **landing-gerson3186647**. Esto va a ser parte de tu dirección web final (algo como **truequematch-gerson.infinityfreeapp.com**).
 - ✓ **Extensión del dominio:** dejala en la que venga por defecto (normalmente **.infinityfreeapp.com**).

- Nos va a pedir que confirmes que no soy un robot (un captcha simple, tipo clickear una casilla o resolver una imagen).
- Nos va a mostrar una **contraseña generada automáticamente** para esa cuenta de hosting **la guardamos en algún lado**, la vamos a necesitar más adelante si queremos entrar por FTP. (**File Transfer Protocol** o Protocolo de Transferencia de Archivos) como **un camión de mudanzas digital** que transporta las carpetas y archivos de mi página web desde mi computador hasta el servidor de InfinityFree en la nube.
- **¿Por qué necesitamos esa contraseña FTP?**
 - ✓ **La puerta de enlace:** Para subir archivos a un servidor privado en internet no basta con arrastrar y soltar en una web normal; necesitas ingresar por una conexión segura.

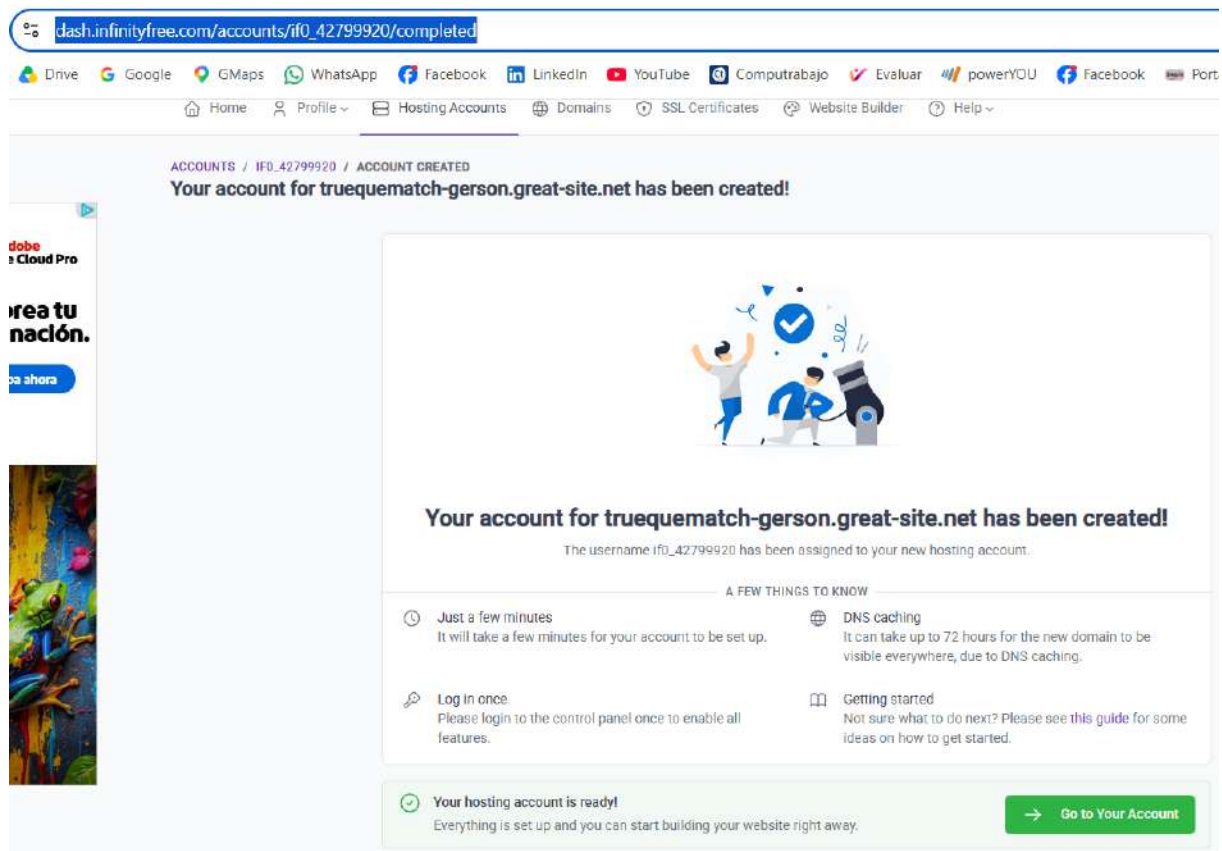
- ✓ **El programa de envío:** Para subir proyectos pesados o carpetas enteras (como los archivos de tu proyecto o plantilla) se utiliza un programa especial llamado **FileZilla**.
- ✓ **Las credenciales:** FileZilla te pedirá 3 datos clave para dejarte entrar al **servidor**:
 - **Servidor (Host):** La dirección IP o nombre del servidor de InfinityFree.
 - **Usuario (Username):** El usuario autogenerado que te acaba de asignar InfinityFree.
 - **Contraseña (Password):** La contraseña que anotaste o la que se generó en ese paso.

Contraseña: **tyEeOzzjEQ**

- Le damos **"Create Account"** (o como se llame el botón de confirmar).

truequematch-gerson.great-site.net

- Va a tardar un par de minutos en terminar de crear la cuenta, es normal, esperamos sin cerrar la pestaña.



Excelente, la cuenta quedó creada con el dominio **truequematch-gerson.great-site.net** un nombre bien identificable, se nota que es mío del proyecto.

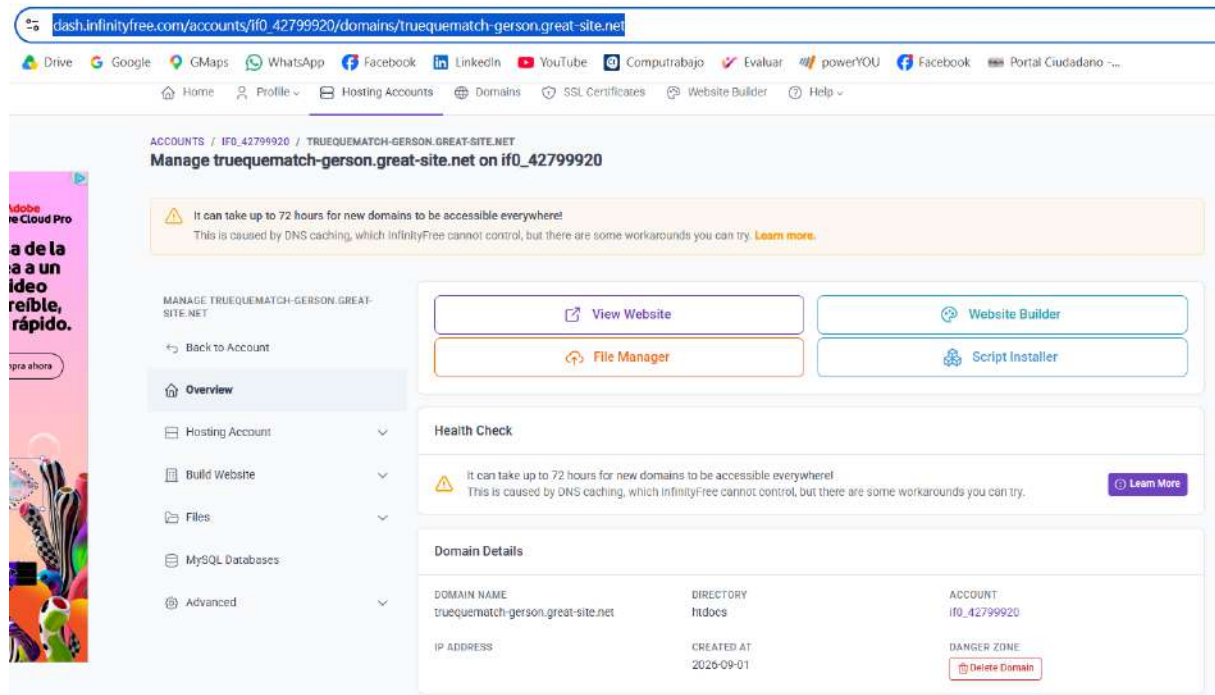
Ojo con dos avisos que trae la misma pantalla, para que no nos preocupemos si algo tarda:

- **"Just a few minutes"**, la cuenta tarda unos minutos en terminarse de activarse del todo por dentro.
- **"DNS caching: puede tardar hasta 72 horas en verse desde cualquier parte del mundo"**, esto es normal en cualquier dominio nuevo (hasta con InfinityFree). No nos alarmamos si la primera vez que la abrimos desde el equipo (o desde el celular) tarda un rato en cargar o no carga aún; normalmente en la práctica anda mucho antes de esas 72 horas, pero puede tardar.

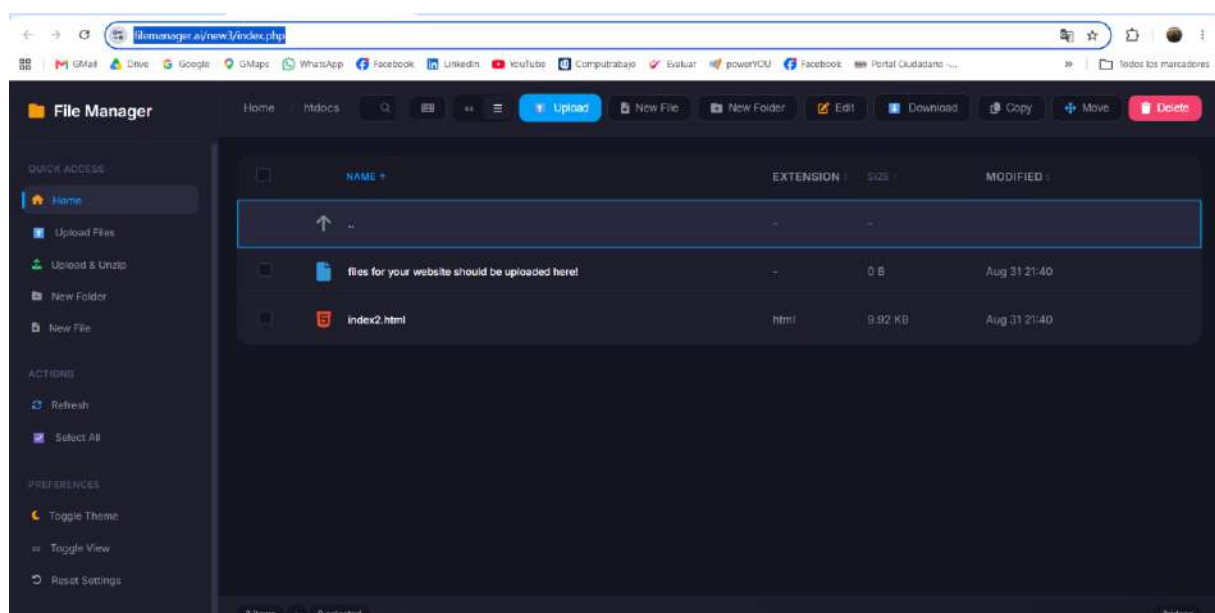
Paso 3: Entrar al Administrador de archivos y subir la plantilla

- Hacemos click en el botón verde **"Go to Your Account"**.

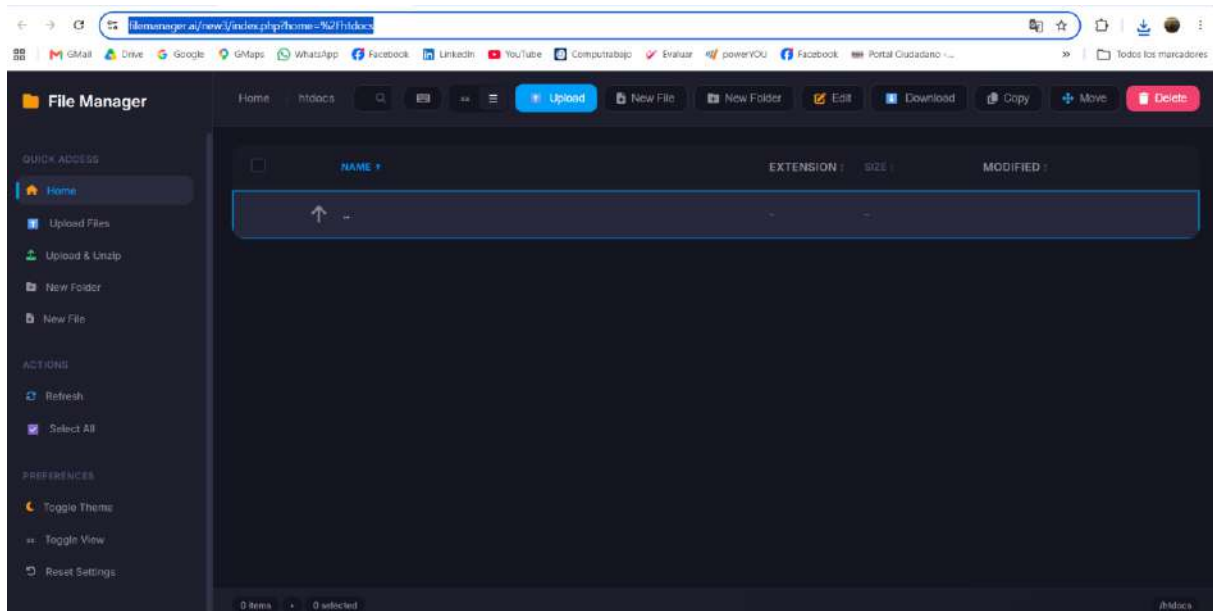
- Nos va a llevar al panel de control de mi cuenta de hosting (el "Manager" que mencionó el profe). Vamos a ver varias opciones: Administrador de archivos, MySQL Databases, FTP, etc.



- Buscamos y hacemos click en "Online File Manager" (Administrador de archivos en línea).

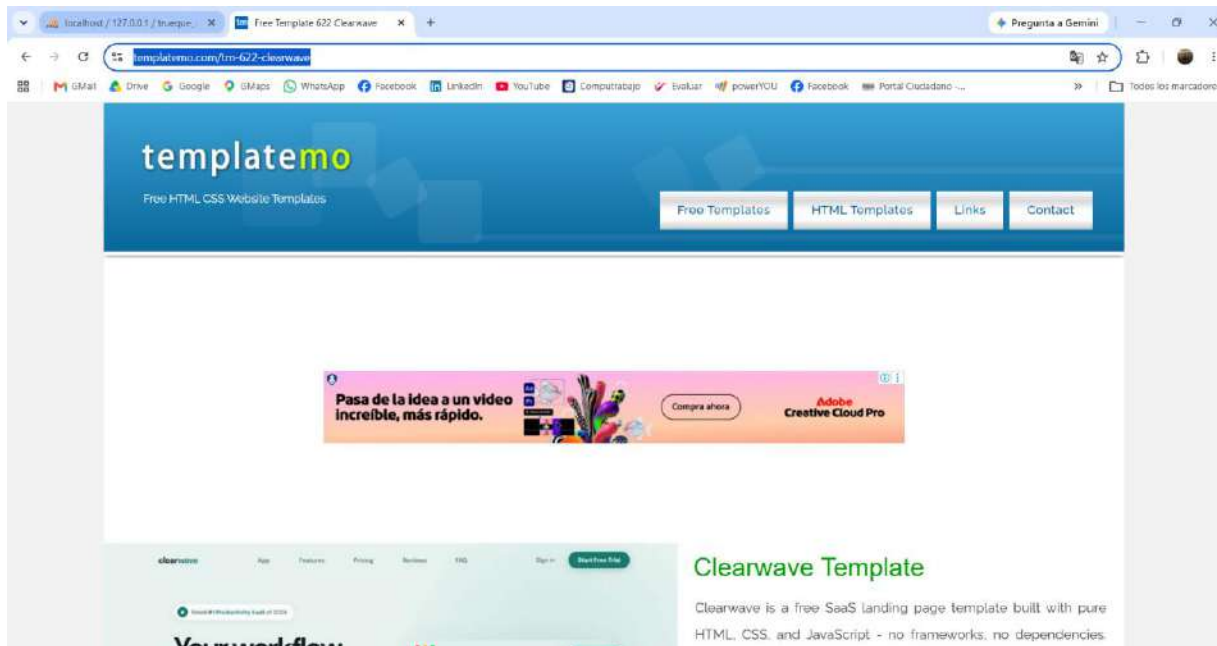


- Se va a abrir una vista parecida al Explorador de Windows, mostrando carpetas. Buscamos y entramos a la carpeta **htdocs**, es exactamente el mismo nombre que usamos en XAMPP, así que ya sabemos cómo pensarla: ahí es donde van mis páginas web.

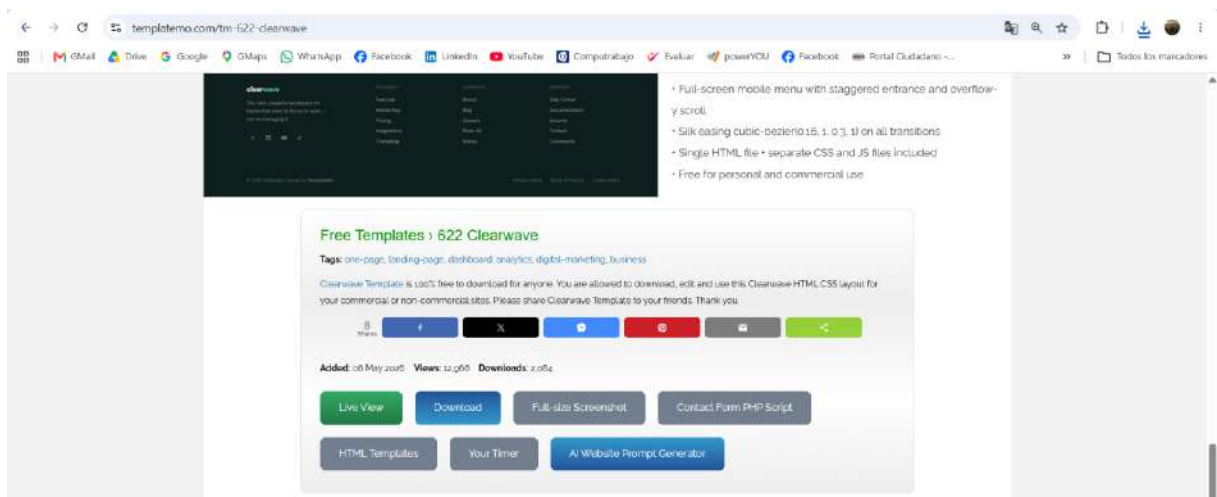


Paso 4: Subir la plantilla Clearwave

- Descargamos primero la plantilla, nos dirigimos a esta página en tu navegador:
<https://templatemo.com/tm-622-clearwave>



- Bajamos hasta encontrar un botón verde que dice "Download". Hacé click ahí.



- Se va a descargar un archivo comprimido (.zip) a tu carpeta de Descargas, algo como **templatemo_622_clearwave.zip**.

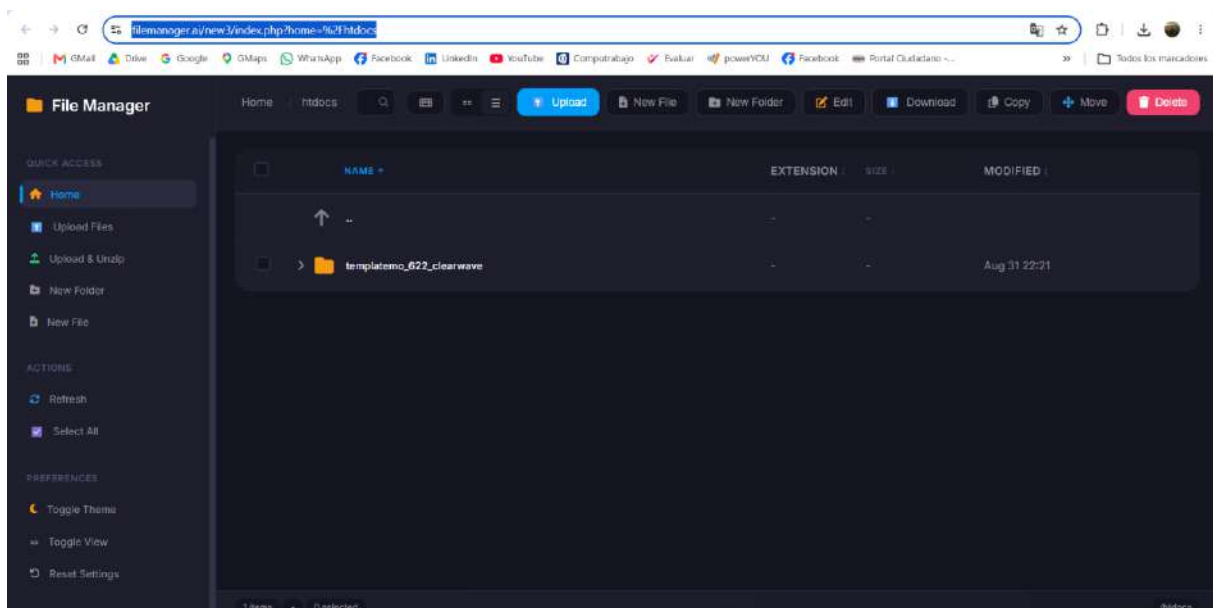
(Para que quede claro como ejemplo: esto es como comprar los planos y los materiales ya cortados de una casa prefabricada, todavía no está "armada" en el terreno, solo la tenemos guardada en una caja.)

- Vamos a la carpeta de Descargas y buscamos el .zip que bajamos.
- Click derecho sobre el **"Extraer todo"**
- Elegimos una ubicación temporal (puede ser la misma carpeta de Descargas o cualquiera) y damos Extraer.

- Se va a crear una carpeta con el contenido de la plantilla adentro, normalmente vamos a ver archivos como **index.html**, una carpeta **css**, una carpeta **js**, etc.

Continuamos con htdocs de InfinityFree:

- Arriba en la barra de botones, vamos a ver **"Upload"** (subir uno por uno) y **"Upload & Unzip"** (subir y descomprimir automático, en el menú de la izquierda), usamos esta segunda opción, nos ahorra un paso.
- Hacemos **click** en **"Upload & Unzip"**.
- Seleccionamos el archivo **templatemo_622_clearwave.zip** desde mi computador (el mismo que ya usaste para XAMPP y para la máquina virtual).
- Esperamos a que suba y se descomprima solo.



(Ojo con un detalle importante: cuando se descomprima, es probable que se cree una carpeta llamada **templatemo_622_clearwave** dentro de **htdocs**, en vez de que los archivos queden sueltos directo en **htdocs**. Si pasa eso, mi sitio no va a cargar en la dirección principal, vamos a revisarlo

Justo lo que te anticipamos: dentro de **htdocs** quedó una carpeta llamada **templatemo_622_clearwave**, en vez de que los archivos (**index.html**, **css**, **js**) queden sueltos directo ahí. Si probamos la URL principal ahora (**truequematch-gerson.great-site.net**), nos va

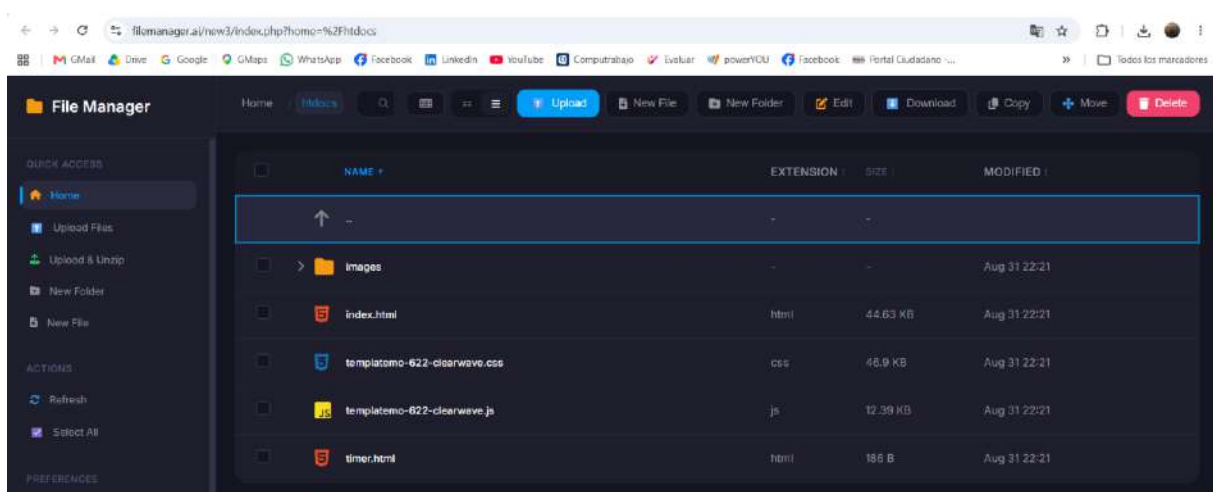
a dar error o página en blanco, porque el servidor busca el `index.html` directo dentro de `htdocs`, no adentro de una subcarpeta.

Corrección:

Opción simple: mover el contenido de adentro hacia afuera

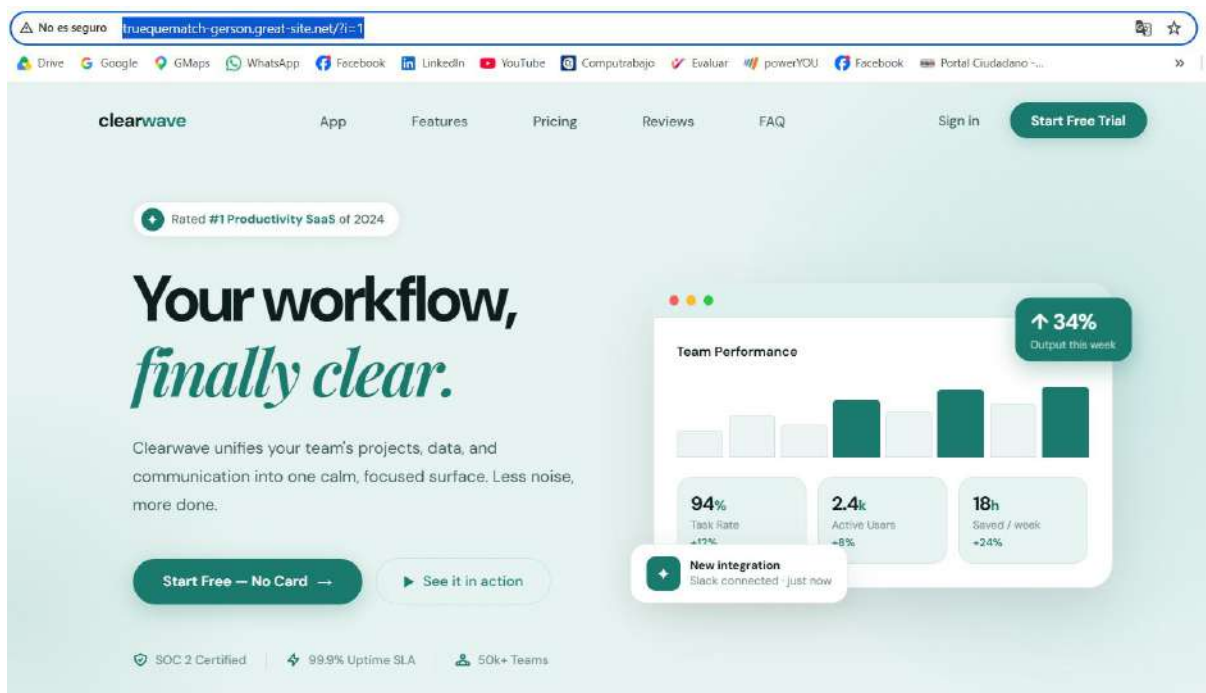
- Hacemos **click** en la flechita > al lado de la carpeta `templatemo_622_clearwave` para entrar y verla abierta (o hacemos doble **click** sobre el nombre).
- Adentro vamos a ver los archivos sueltos: `index.html`, la carpeta `css`, la carpeta `js`, etc.
- Marcamos el **check**, de **todos** esos archivos y carpetas (podemos usar "Select All" del menú izquierdo, pero fijémonos de estar parado *adentro* de `templatemo_622_clearwave` cuando lo hagamos, no en `htdocs`).
- Arriba, hacemos **click** en "Move".
- Nos va a pedir a dónde moverlos, elegimos la carpeta `htdocs` (un nivel arriba, la carpeta "padre").
- Confirmamos el movimiento.
- Una vez movidos, podemos borrar la carpeta `templatemo_622_clearwave` que quedó vacía (opcional, pero prolijo).

Al final, `htdocs` debería mostrar directamente `index.html`, `css`, `js` sin ninguna carpeta contenedora en el medio.



Ahora sí, probemos la URL real

- Abrimos una pestaña nueva.
- Escribimos: <http://truequematch-gerson.great-site.net/>
- Esperamos unos segundos, recordamos el aviso de "puede tardar unos minutos en activarse del todo".



Meta cumplida truequematch-gerson.great-site.net cargando la plantilla **Clearwave** completa, con todo su diseño, desde una dirección real de Internet.

Un detalle que noto, no bloqueante pero vale la pena mencionarlo: arriba en la barra dice "**No es seguro**" eso es porque el sitio está sirviendo por <http://> sin certificado SSL activado todavía (el famoso "candadito verde" con <https://>). No afecta que funcione, pero para dejarlo más prolijo, **InfinityFree** nos deja activar un certificado SSL gratis (Let's Encrypt) desde la sección "**SSL Certificates**" del panel.

Repaso completo de las 5 evidencias visuales que ya tenés para esta parte

#	Captura	Estado
1	Cuenta InfinityFree creada e ingresada	OK.

#	Captura	Estado
2	Sitio/dominio truequematch-gerson.great-site.net creado	OK.
3	Carpeta htdocs vacía en el Administrador de archivos	OK.
4	Archivos de Clearwave subidos y reubicados correctamente	OK.
5	Sitio cargando en vivo desde la URL pública	OK.

Paso 5: Adicionar [HTTPS://](#)

The screenshot shows the InfinityFree website interface. The top navigation bar includes links for Home, Profile, Hosting Accounts, Domains, SSL Certificates, Website Builder, and Help. The main content area is titled 'Order Free SSL Certificate' and features a message: 'Custom SSL certificates are no longer supported for free subdomains. All free subdomains have SSL by default.' Below this, there is a form to 'Order Free SSL Certificate' with a text input field for 'Domain Name' containing 'truequematch-gerson.great-site.net'. A note states 'The www. subdomain is automatically included.' and there is an 'Advanced Options' link. At the bottom, there are 'Back' and 'Create Order' buttons.

Excelente miremos el aviso rosado que dice: "Custom SSL certificates are no longer supported for free subdomains. All free subdomains have SSL by default."

Traducido: como mi dominio es un subdominio gratis de InfinityFree (truequematch-gerson.great-site.net), el SSL ya viene incluido automáticamente, no tenemos que pedirlo ni configurar nada, esta pantalla de "Order Free SSL Certificate" es para

cuando alguien conecta su propio dominio comprado aparte (tipo .com propio), no para los subdominios gratis como el mio.

No hace falta que le demos a "Create Order", no vamos a lograr nada nuevo ahí, porque ya lo tenemos activado de fábrica.

Vamos a comprobarlo directo:

- Cerramos esa pantalla (o dale **"Back"**).
- Abrí una pestaña nueva y escribí la dirección con <https://> al principio, así, explícito:

<https://truequematch-gerson.great-site.net/>

- Fijémonos si ahora aparece el candadito, en vez del **"No es seguro"** que vimos antes.

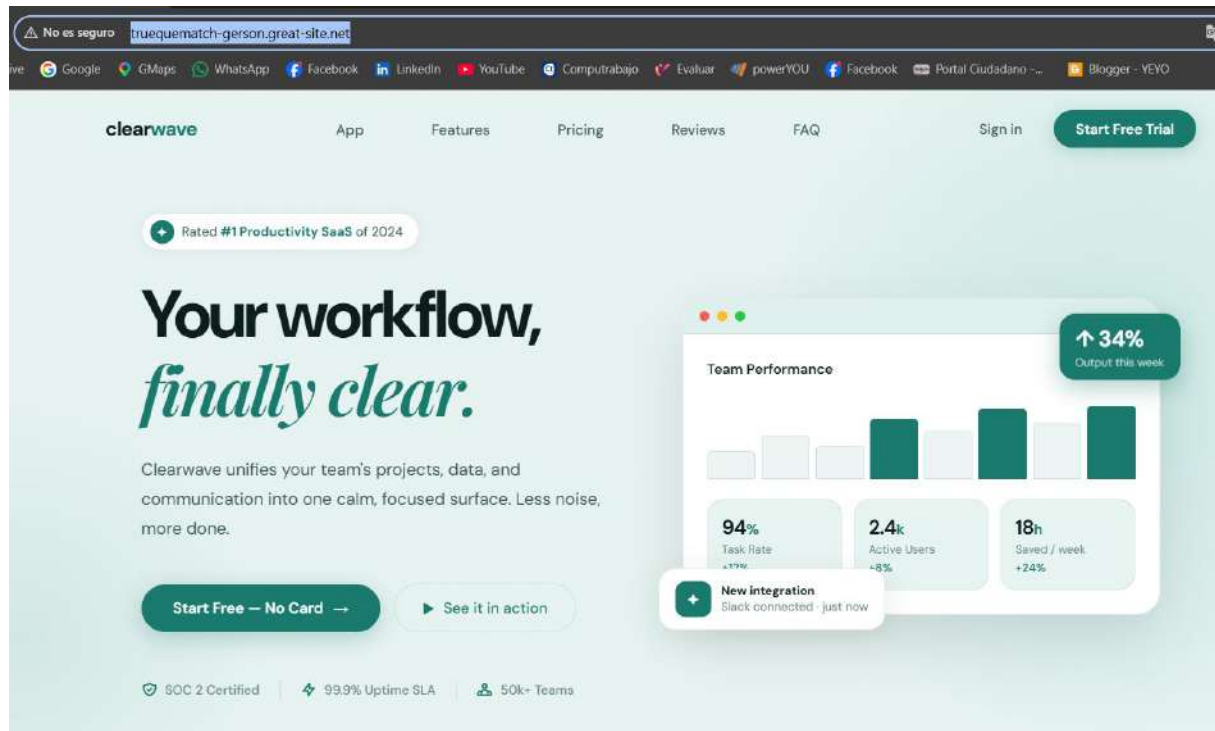
(Ojo: puede que tarde un ratito en propagarse el certificado la primera vez, sobre todo si el sitio recién se creó hoy, si te sigue saliendo "No es seguro" ahora mismo, no nos preocupamos, probamos de nuevo en 15-20 minutos.)

Saldrá **ERR_CONNECTION_REFUSED**, que quiere decir "todavía no hay nadie atendiendo en la puerta de **HTTPS** (el puerto 443)". Es justo lo que analizamos anteriormente que podía pasar: el certificado SSL está incluido, pero recién se está terminando de activar/propagar en el servidor, no significa que se rompió nada de lo que ya armamos.

Confirmemos que lo importante sigue funcionando: probamos de nuevo la versión sin la "s" (<http://>, la que ya nos había funcionado perfecto hace un rato):

<http://truequematch-gerson.great-site.net/>

Si el HTTPS igual para que quede más prolijo, lo más simple es dejarlo descansar unas horas (a veces hasta un día, por eso el aviso de "DNS caching hasta 72 horas" que vimos en la creación de la cuenta) y probarlo de nuevo más tarde, no hay nada más que podamos hacer de mi lado para acelerarlo, es un proceso automático del lado del servidor.



Todo perfecto miremos que la pestaña dice **"TemplateMo 622 Clearwave"** y la página cargó completa con estilos, botones, todo. El <http://> no aparece escrito en la barra porque Chrome lo oculta visualmente cuando escribís sin <https://> (es un comportamiento normal del navegador, no significa que no haya cargado por http).

El aviso **"No es seguro"** es exactamente lo que esperábamos: es solo la falta del candadito [HTTPS](https://) que ya sabemos que va a tardar un rato en propagarse. No afecta que el sitio funcione ni invalida la evidencia, la plantilla está online, accesible, y cargando perfecto desde una URL pública real.

Conclusiones

- Montar un servidor no depende de un solo sistema operativo ni de una sola herramienta: el mismo par Apache más MySQL que corre en Windows con XAMPP se pudo replicar sin problema dentro de una máquina virtual con Ubuntu Linux, usando prácticamente la misma lógica de carpetas (htdocs) y el mismo tipo de instalador.
- La virtualización con VirtualBox demostró ser una forma segura de experimentar con un sistema operativo distinto sin arriesgar el equipo principal cualquier error durante la instalación (como el que se presentó por falta de RAM asignada) se resuelve ajustando la máquina virtual, sin ningún impacto sobre Windows ni sobre el resto de los proyectos que ya corren en el equipo real.
- Existe una diferencia clara entre un despliegue local y uno en producción: mientras que XAMPP (tanto en Windows como en la máquina virtual Linux) solo es accesible desde el propio equipo a través de localhost, una plataforma de hosting como InfinityFree convierte esa misma plantilla en un sitio web real, con una dirección pública que cualquier persona puede visitar desde Internet.
- El proceso de publicar en un hosting gratuito replica los mismos pasos conceptuales que ya se habían practicado en local: descargar una plantilla, ubicarla en la carpeta correcta del servidor (htdocs) y verificar que cargue correctamente en el navegador lo que cambia es la escala y el alcance, no la lógica de fondo.
- Se identificaron y resolvieron dos inconvenientes reales durante la práctica: la instalación de Ubuntu que se trababa por RAM insuficiente (resuelto subiendo la memoria asignada de 2 GB a 6 GB, el mínimo oficial que exige Ubuntu 26.04), y la estructura de carpetas incorrecta al subir la plantilla al hosting (resuelto moviendo los archivos al nivel correcto dentro de htdocs). Ambos hallazgos, junto con su solución, quedan documentados como parte del valor real de la evidencia no solo mostrando que el resultado final funciona, sino el proceso de diagnóstico para llegar hasta ahí.
- En conjunto, ambas evidencias cierran el ciclo completo de un despliegue de aplicaciones: desarrollo local (Windows), servidor alternativo virtualizado (Linux) y publicación real en Internet (hosting gratuito) los tres escenarios que un desarrollador necesita dominar antes de llevar un proyecto real, como Trueque Match, a producción.

Llevar Trueque Match de verdad a Internet

Antes de empezar confirmamos los límites de mi base mi base de datos (`trueque_match_db.sql`) pesa apenas unos **20 KB**, y el límite de **InfinityFree** por base de datos es de **50 MB**, le sobra espacio de lejos. Y de yapa, permite hasta 400 bases de datos por cuenta, así que no nos acercamos a ningún límite.

Sobre guardar en “GitHub”

Voy a subir el resultado final a GitHub como siempre, pero **hay una excepción importante**: Mi repo `proyecto_sena` es **público**, cualquiera puede. Si edito `conexion.php` con las credenciales reales de la base de datos de **InfinityFree** (usuario, contraseña, servidor) y lo subimos tal cual a ese repo público, **cualquier persona en el mundo podría copiar esas credenciales y conectarse directo a mi base de datos real**, sin permiso.

Es como pegar la clave de mi casa nueva en un cartel en la vía pública para que "quede documentado que me mudaste" la intención es buena (dejar registro), pero el lugar donde lo público es el problema, no la idea en sí.

Por eso voy a hacer esto distinto a como vengo trabajando:

1. El `conexion.php` que ya tengo en GitHub (con 127.0.0.1:3307) **se queda tal cual está**, esas son credenciales locales, inofensivas si alguien las ve, porque solo funcionan dentro de tu propia laptop.
2. Para InfinityFree, voy a crear un **archivo nuevo y separado** con las credenciales de producción, y ese archivo **nunca se sube a GitHub**, se queda únicamente adentro del hosting de **InfinityFree**, subido directo por el Administrador de archivos (no por `git push`).
3. Lo que sí subiré a GitHub, cuando termine, es la **documentación del proceso** (capturas, el informe) igual que con las evidencias anteriores pero sin ninguna clave real adentro.

Sobre la cuenta de InfinityFree con GitHub

Eso no tiene relación con este tema, usamos GitHub solo como **método de login** (como cuando entramos a una app con tu cuenta de Google), no conectaste tu repositorio ni le di acceso a mis archivos de código. Es completamente independiente.

¿Cuándo hacemos el git push entonces?

Al final, como siempre, pero documentando el proceso (capturas + informe), nunca las credenciales reales de producción.

7.2 El plan completo, en 5 pasos

Para no chocar con la evidencia que ya entregue (**Clearwave**, que se queda tal cual en **htdocs**), vamos a meter Trueque Match en una carpeta aparte dentro del mismo hosting, así: truequematch-gerson.great-site.net/truequematch/

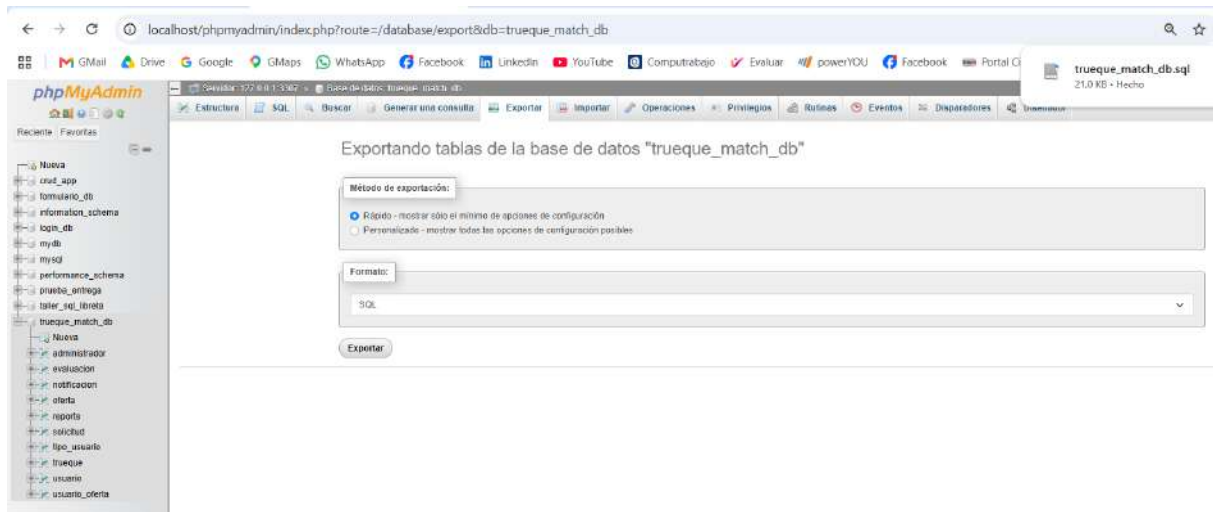
- **Crear la base de datos** en el panel de **InfinityFree** (con su propio usuario, contraseña y nombre de servidor, distintos a los de mi **XAMPP** local).
- **Importar mi base de datos** (**trueque_match_db.sql**) a esa base nueva, usando el **phpMyAdmin** que trae **InfinityFree**.
- **Ajustar `conexion.php`** para que apunte al servidor de **InfinityFree** en vez de a tu **localhost:3307**.
- **Subir todos los archivos del proyecto** (no solo la plantilla, sino **app_trueque_match/** completa lo más archivos de la raíz) a una carpeta nueva dentro de **htdocs**.
- **Probar en vivo**, entrar con mi usuario real y confirmar que el **login**, el **dashboard** y las ofertas cargan con datos reales desde el otro lado del mundo (bueno, desde el hosting).

Paso 1: Preparar mi base de datos para exportar

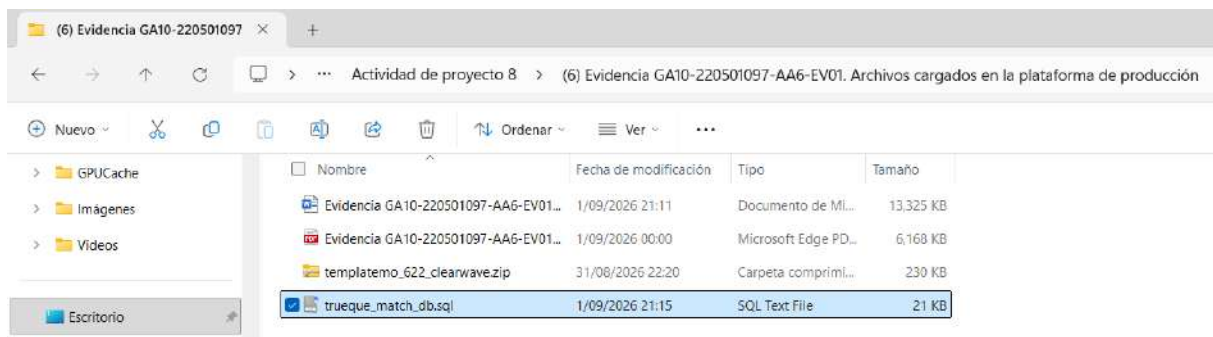
Antes de tocar InfinityFree, aseguremos que él **.sql** que vamos a subir esté actualizado y limpio (sin la basura de pruebas de Postman que ya deberíamos haber borrado).

- Abrimos **phpMyAdmin** en tu **XAMPP** local (<http://localhost/phpmyadmin/>).
- Seleccionamos la base **trueque_match_db** en el panel izquierdo.
- Nos dirigimos a la pestaña **"Exportar"** arriba.

- Dejamos el método "**Rápido**" y el formato "**SQL**" (son los valores por defecto).
- Le damos click a "**Continuar**" (o "Go").

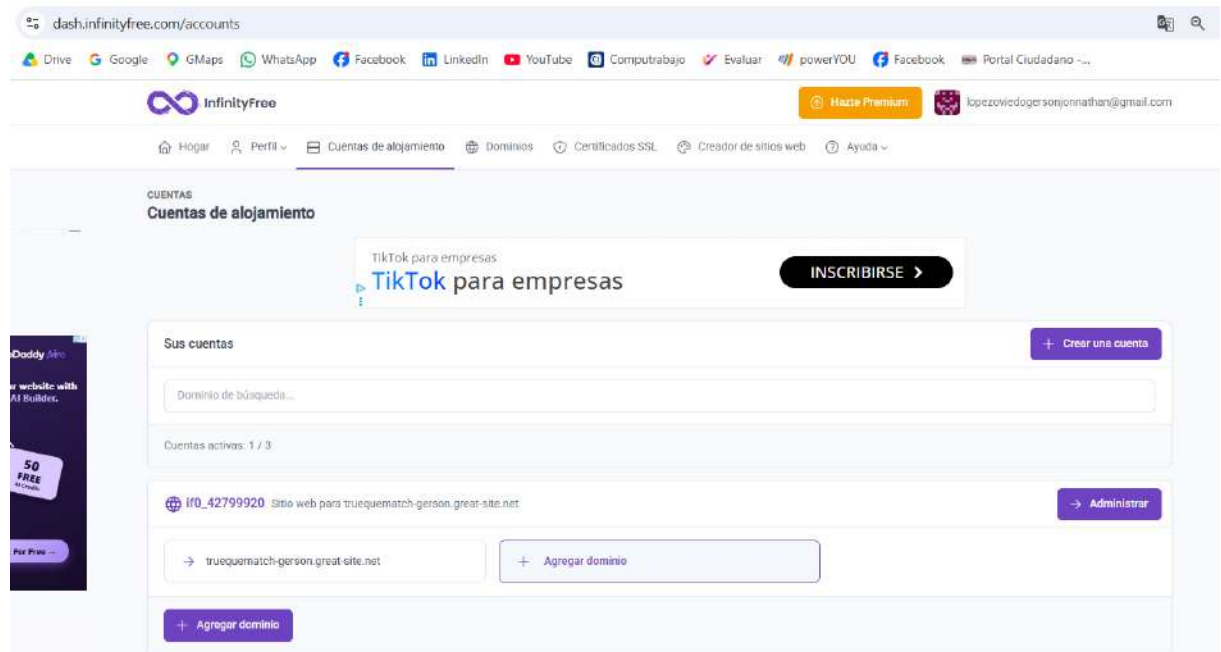


- Se descarga un archivo **.sql** nuevo, con el nombre **trueque_match_db.sql**, a mi carpeta de Descargas, este es el que vamos a subir a **InfinityFree**. Lo podemos guardar en cualquier carpeta si deseamos.

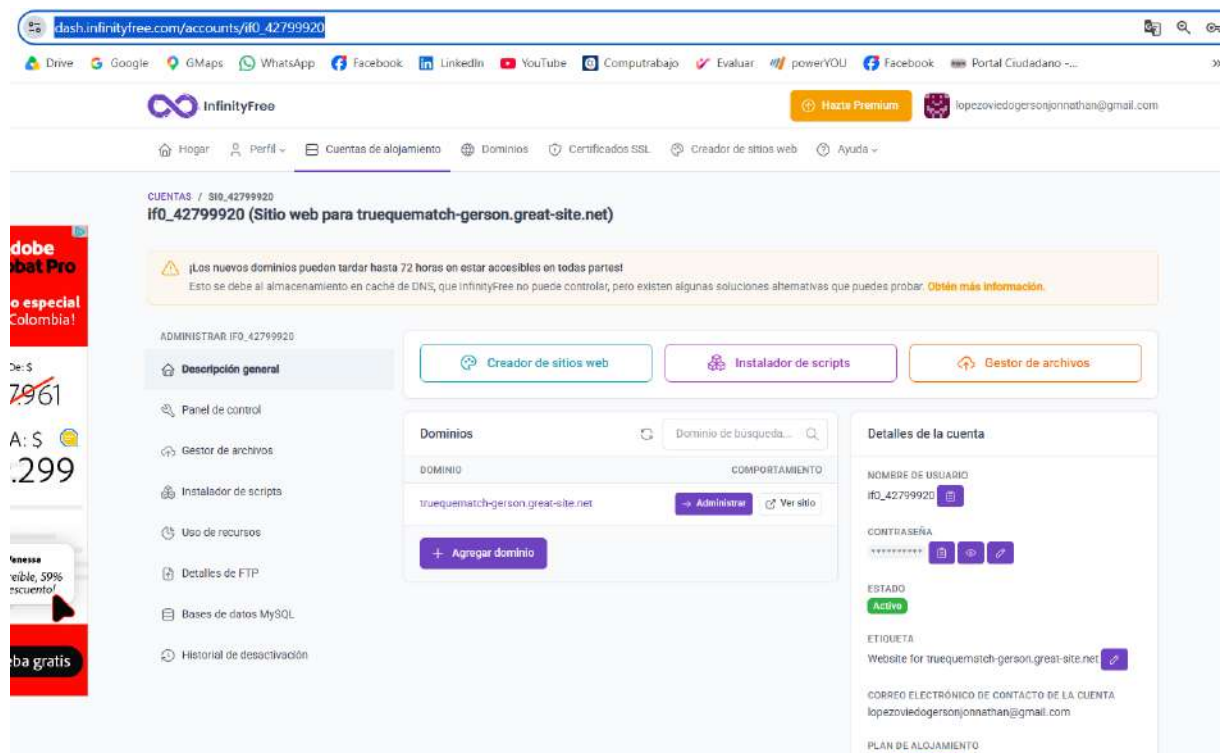


Paso 2: Crear la base de datos en InfinityFree

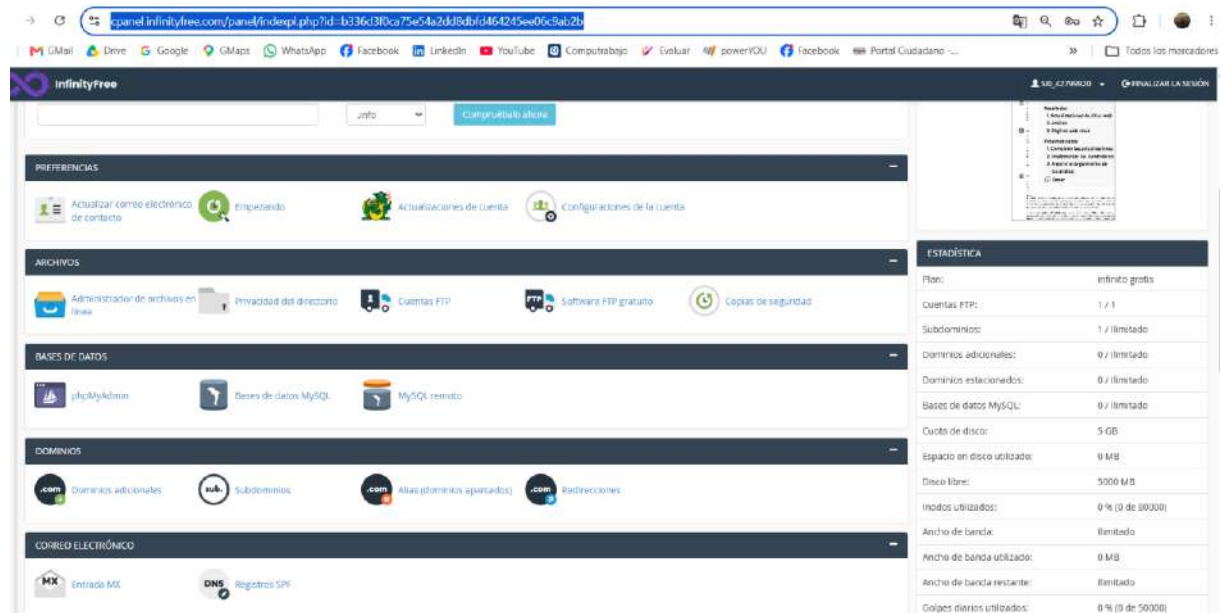
1. Nos dirigimos al panel principal de **InfinityFree** (dash.infinityfree.com) y entramos a "**Hosting Accounts**".



- Buscamos el botón morado que dice **Administrar** a la derecha de mi cuenta (if0_42799920). Hacemos clic ahí.
- Sobre mi cuenta (truequematch-gerson.great-site.net), hacemos click en "**Control Panel**" (o el ícono de engranaje/configuración).

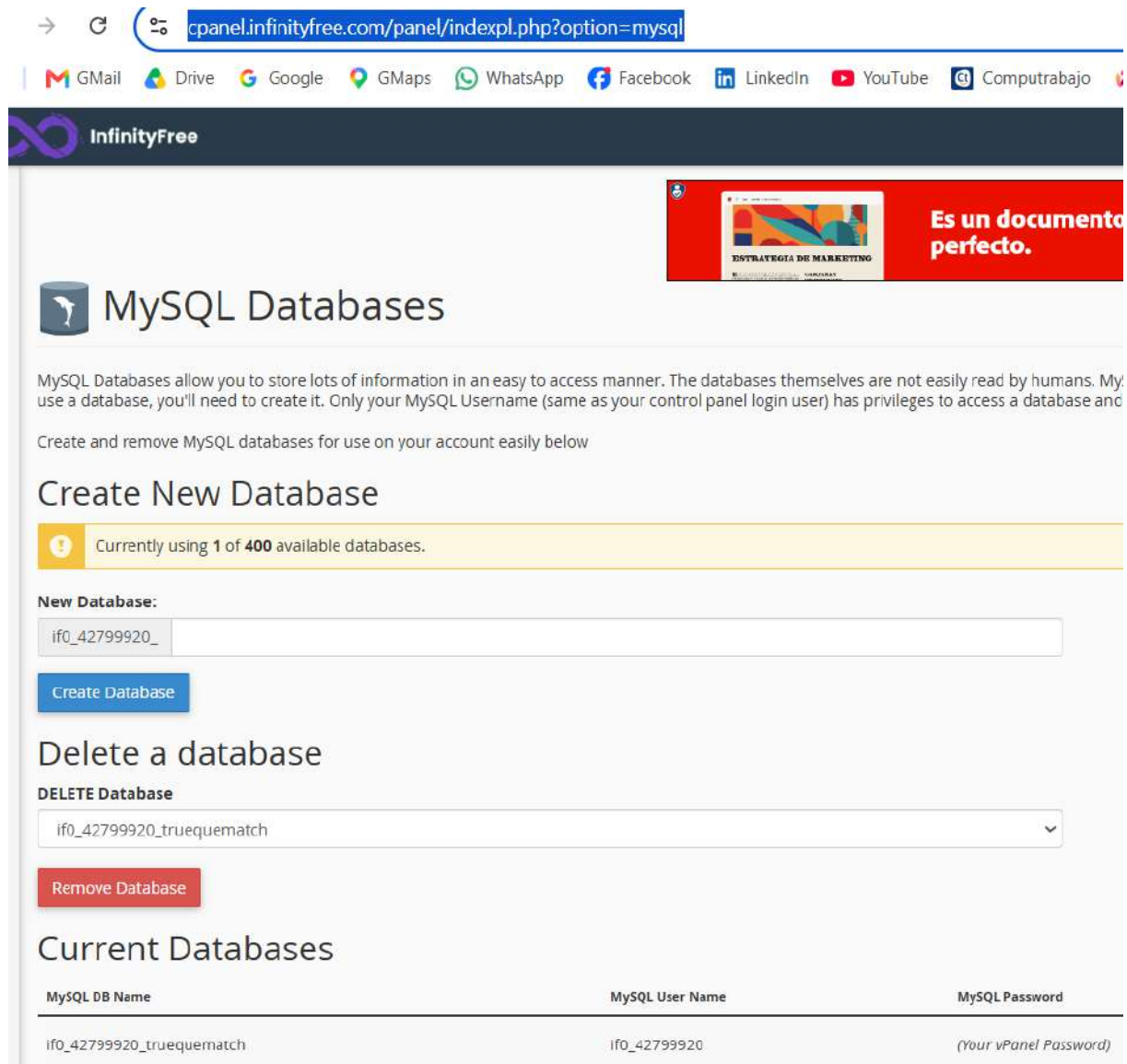


- Nos va a llevar a un panel completo con varias secciones, buscamos "**MySQL Databases**" (a veces está bajo una categoría llamada "**Databases**").



- Ahí vamos a ver un formulario para crear una base nueva. Escribimos un nombre por ejemplo **truequematch** (InfinityFree le va a pegar un prefijo automático a mi cuenta adelante, algo como **if0_XXXXXXXX_truequematch**, así identifica que es tuya).
- Le damos **click** en “**Create**” o “**Crear**”.
- Cuando la termine de crear, nos va a mostrar (o vamos a tener que entrar a verla después) **4 datos clave** los escribimos todos:

Dato	Ejemplo de cómo se ve
Nombre de la base	if0_XXXXXXXX_truequematch
Usuario	(normalmente el mismo nombre que la base)
Contraseña	la que te genere o la que definas vos
Servidor (hostname)	Algo como sqlXXX.infinityfree.com este dato es clave, cada cuenta tiene uno distinto, no es siempre el mismo



→ ↻ 🔍 cpanel.infinityfree.com/panel/indexpl.php?option=mysql

GMail Drive Google GMaps WhatsApp Facebook LinkedIn YouTube Computrabajo

InfinityFree

MySQL Databases

MySQL Databases allow you to store lots of information in an easy to access manner. The databases themselves are not easily read by humans. My use a database, you'll need to create it. Only your MySQL Username (same as your control panel login user) has privileges to access a database and

Create and remove MySQL databases for use on your account easily below

Create New Database

Currently using 1 of 400 available databases.

New Database:

ifo_42799920_

Create Database

Delete a database

DELETE Database

ifo_42799920_truequematch

Remove Database

Current Databases

MySQL DB Name	MySQL User Name	MySQL Password
ifo_42799920_truequematch	ifo_42799920	(Your vPanel Password)

La clave en la captura: donde dice "MySQL Password", **InfinityFree** no me muestra la clave ahí dice literalmente "(Your vPanel Password)". Eso significa que **la contraseña de mi base de datos es la misma que la contraseña de mi cuenta de hosting** (la que **InfinityFree** generó automáticamente cuando cree truequematch-gerson.great-site.net, la que anote en aquel momento para el **FTP**). No es una clave separada.

Entonces si [tyEeOzzjEQ](#) es esa contraseña que anote en aquel momento, **sí, esa es la correcta** para conectar a la base también.

Ya tengo los 4 datos completos:

Dato	Valor
Nombre de la base	if0_42799920_truequematch
Usuario	if0_42799920
Contraseña	tyEeOzzjEQ (a confirmar en el próximo paso)
Servidor (host)	sql309.infinityfree.com

Paso 3: Importar mi base de datos

- En ese mismo panel de **InfinityFree**, buscamos el enlace o botón "**phpMyAdmin**" (suele estar en el menú principal del Control Panel, en la sección de bases de datos).
- Se va a abrir un **phpMyAdmin**, muy similar al que ya usaba en mi **XAMPP** local.



- En el panel izquierdo, hacemos **click** sobre **if0_42799920_truequematch** (mi base recién creada, todavía vacía).
- Arriba, nos dirigimos a la pestaña "**Importar**".

← → ↻ php-mysqladmin.net/db_import.php?db=if0_42799920_truequematch

[GMail](#)
[Drive](#)
[Google](#)
[GMaps](#)
[WhatsApp](#)
[Facebook](#)
[LinkedIn](#)
[YouTube](#)
[Computrabajo](#)
[Evaluar](#)
[powerYOU](#)
[Facebook](#)
[Porta](#)

[Servidor: sql209.byethost1.com](#)
[Base de datos: if0_42799920_truequematch](#)

[Estructura](#)
[SQL](#)
[Buscar](#)
[Generar una consulta](#)
[Exportar](#)
[Importar](#)
[Operaciones](#)
[Rutinas](#)
[Diseñador](#)

Importando en la base de datos "if0_42799920_truequematch"

Archivo a importar:

El archivo puede ser comprimido (gzip, bzip2, zip) o descomprimido.
 Un archivo comprimido tiene que terminar en `[formato].[compresión]`. Por ejemplo: `.sql.zip`

Buscar en su ordenador: [Seleccionar archivo](#) Ningún archivo seleccionado (Máximo: 300MB)

También puede arrastrar un archivo en cualquier página.

Conjunto de caracteres del archivo:

Importación parcial:

☒ Permitir la interrupción de una importación en caso que el script detecte que se ha acercado al límite de tiempo PHP. (Esto podría ser un buen método para importar archivos grandes; sin embargo, puede dañar las transacciones.)

Omitir esta cantidad de consultas (en SQL) desde la primera:

Otras opciones:

☒ Habilite la revisión de las claves foráneas

Formato:

Opciones específicas al formato:

Modalidad SQL compatible:

☒ No utilizar AUTO_INCREMENT con el valor 0

[Continuar](#)

- Click en "**Seleccionar archivo**" y elegimos mi [trueque_match_db.sql](#) (el que exporte).

Importando en la base de datos "if0_42799920_truequematch"

Archivo a importar:

El archivo puede ser comprimido (gzip, bzip2, zip) o descomprimido.
 Un archivo comprimido tiene que terminar en .{formato}.{compresión} Por ejemplo: .sql.zip
 Buscar en su ordenador trueque_match_db.sql (Máximo: 300MB)
 También puede arrastrar un archivo en cualquier página.
 Conjunto de caracteres del archivo:

Importación parcial:

☒ Permitir la interrupción de una importación en caso que el script detecte que se ha acercado al límite de tiempo PHP. (Esto podría ser un buen método para importar archivos grandes, sin embargo, puede dañar las transacciones.)
 Omitir esta cantidad de consultas (en SQL) desde la primera:

Otras opciones:

☒ Habilitar la revisión de las claves foráneas

Formato:

Opciones específicas al formato:

Modalidad SQL compatible:
☒ No utilizar AUTO_INCREMENT con el valor 0

- Dejamos el resto de las opciones por defecto.
- Le doy en **"Continuar"** (o "Go") al final de la página.



Excelente, base importada de verdad, 58 consultas ejecutadas (las 10 tablas, sus restricciones de llaves foráneas, y todos los datos que tenías cargados). Con eso confirmamos también que la contraseña era correcta.

Lista de tablas dentro de **if0_42799920_truequematch** (click en el nombre de la base en el panel izquierdo de **phpMyAdmin**), para que quede la evidencia visual de las 10 tablas.

The screenshot shows the phpMyAdmin interface for a database named 'truequematch'. The URL in the browser is 'php-myadmin.net/db_structure.php?db=if0_42799920_truequematch'. The interface includes a top navigation bar with icons for GMail, Drive, Google, GMaps, WhatsApp, Facebook, LinkedIn, YouTube, and a 'Com' icon. Below this is a status bar showing 'Servidor: sql309.byetcluster.com' and 'Base de datos: if0_42799920_truequematch'. A main menu bar contains 'Estructura', 'SQL', 'Buscar', 'Generar una consulta', 'Exportar', 'Importar', 'Operaciones', 'Rutinas', and a 'Filtros' button. The 'Filtros' section has a search box labeled 'Que contengan la palabra:'. The main table lists 10 tables with columns for 'Tabla', 'Acción', 'Filas', 'Tipo', and 'Cotejamiento'. The tables are: administrador, evaluacion, notificacion, oferta, reporte, solicitud, tipo_usuario, trueque, usuario, and usuario_oferta. Each table has icons for 'Examinar', 'Estructura', 'Buscar', 'Insertar', 'Vaciar', and 'Eliminar'. The 'Filas' column shows the number of rows for each table. The 'Tipo' column shows the database engine (InnoDB or MyISAM). The 'Cotejamiento' column shows the character set and collation. At the bottom, there is a 'Crear tabla' button and a form to create a new table with fields for 'Nombre' and 'Número de columnas' (set to 4). There are also links for 'Imprimir' and 'Diccionario de datos'.

Tabla	Acción	Filas	Tipo	Cotejamiento
administrador	Examinar Estructura Buscar Insertar Vaciar Eliminar	3	InnoDB	utf8mb4_general_ci
evaluacion	Examinar Estructura Buscar Insertar Vaciar Eliminar	7	MyISAM	latin1_swedish_ci
notificacion	Examinar Estructura Buscar Insertar Vaciar Eliminar	6	InnoDB	utf8mb4_general_ci
oferta	Examinar Estructura Buscar Insertar Vaciar Eliminar	11	InnoDB	utf8mb4_general_ci
reporte	Examinar Estructura Buscar Insertar Vaciar Eliminar	4	InnoDB	utf8mb4_general_ci
solicitud	Examinar Estructura Buscar Insertar Vaciar Eliminar	4	InnoDB	utf8mb4_general_ci
tipo_usuario	Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_general_ci
trueque	Examinar Estructura Buscar Insertar Vaciar Eliminar	6	InnoDB	utf8mb4_general_ci
usuario	Examinar Estructura Buscar Insertar Vaciar Eliminar	9	InnoDB	utf8mb4_general_ci
usuario_oferta	Examinar Estructura Buscar Insertar Vaciar Eliminar	6	InnoDB	utf8mb4_general_ci
10 tablas	Número de filas	58	MyISAM	latin1_swedish_ci

Paso 4: Editar conexion.php para producción

Ahora armamos el archivo de conexión nuevo, el que **solo va a vivir en InfinityFree, nunca en GitHub** como mencione. Voy a partir del **conexion.php** real que ya tengo, cambiando solo los 4 datos de conexión:

```
<?php

/*

* =====

* TRUEQUE MATCH — conexion.php (PRODUCCIÓN)

* Archivo de conexión para el hosting InfinityFree
```

```

* Gerson Jonnathan López Oviedo | Ficha 3186647

* =====

*/

// En producción usamos el servidor de InfinityFree, no localhost

// Ojo: acá NO va el puerto 3307 pegado con ':' — eso era solo para tu XAMPP local

$conexion = mysqli_connect("sql309.infinityfree.com", "if0_42799920", "tyEeOzzjEQ",
"if0_42799920_truequematch");

// Verificamos si la conexión falló

if (!$conexion) {

    die("ERROR: No se pudo conectar a la base de datos. " . mysqli_connect_error());

}

// Configuramos el juego de caracteres a UTF-8

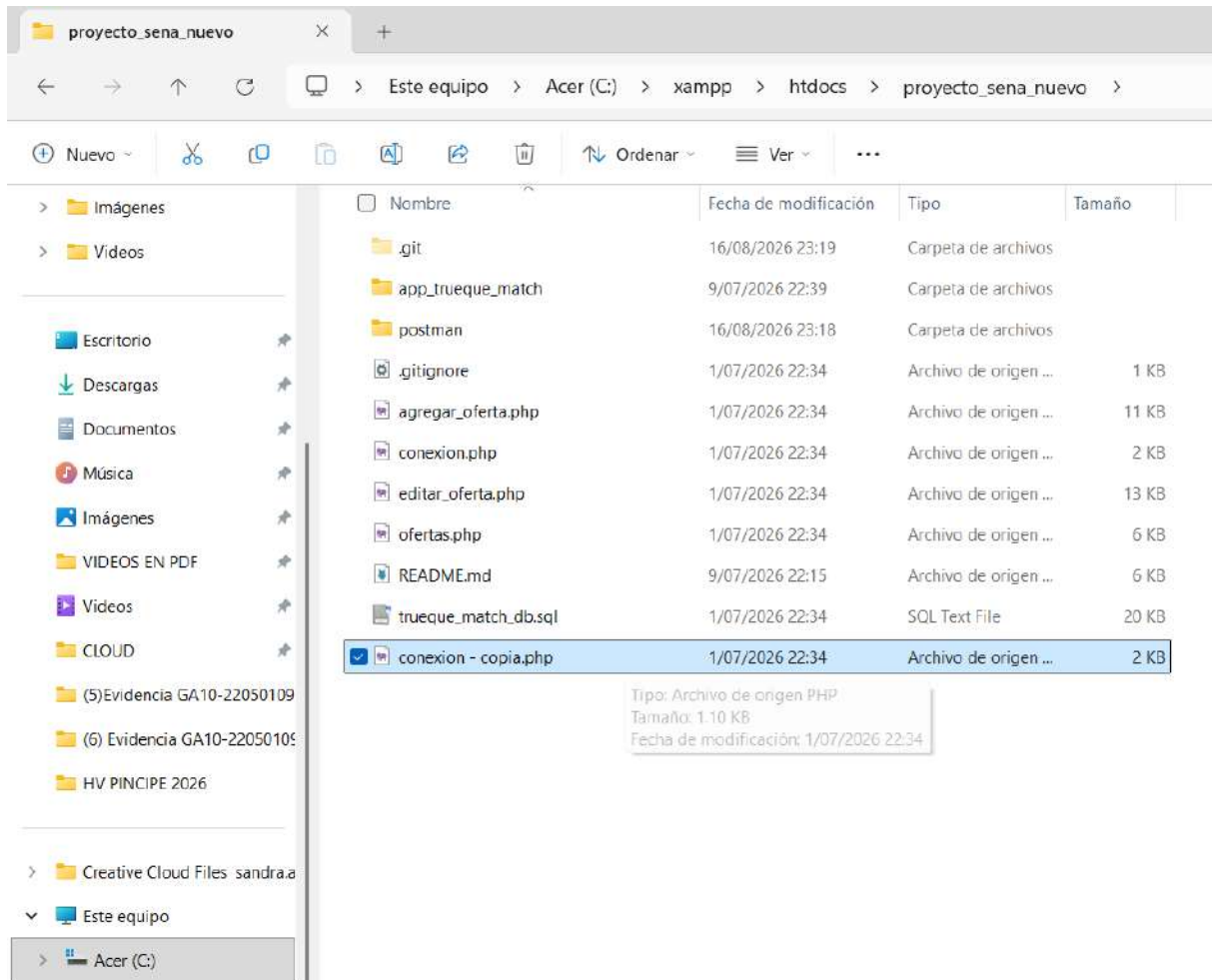
mysqli_set_charset($conexion, "utf8");

?>

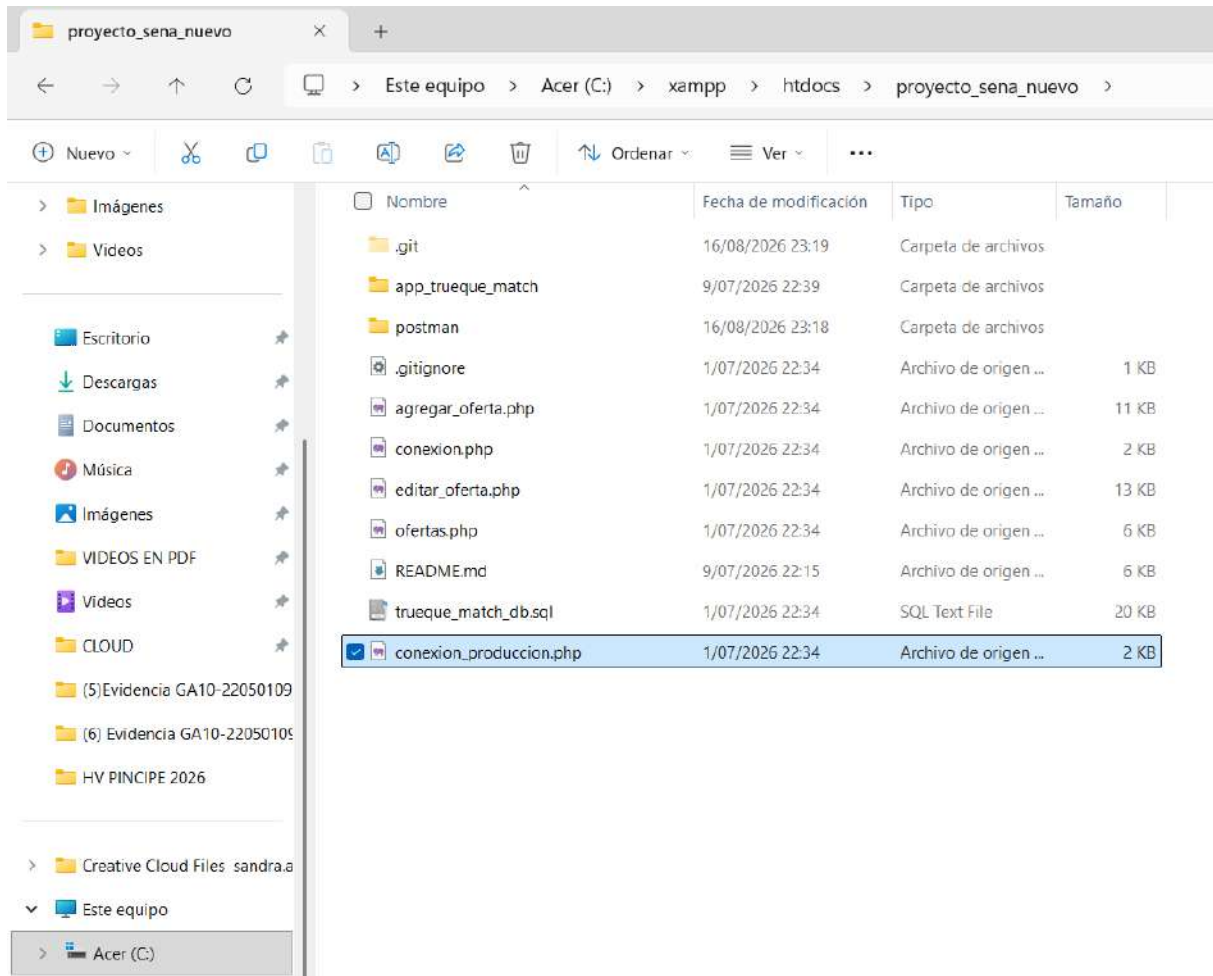
```

Cómo creo este archivo en mi computador, sin tocar el del repo:

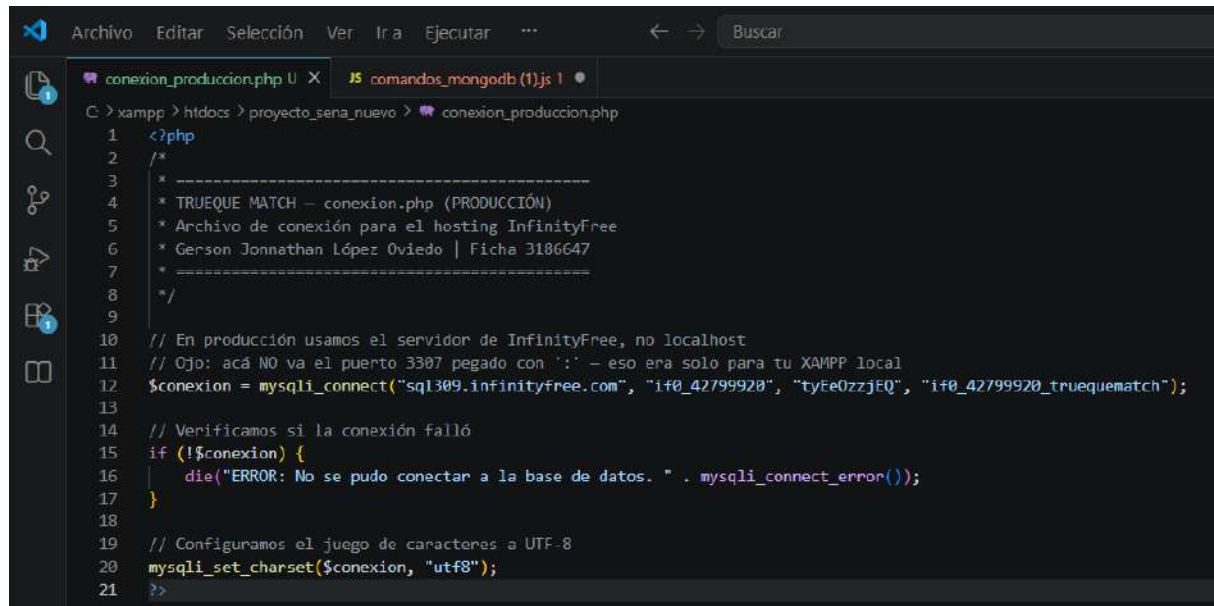
- ✓ Me dirijo a mi carpeta **C:\xampp\htdocs\proyecto_sena_nuevo**
- ✓ Copio el archivo **conexion.php** que ya está ahí (Ctrl+C, Ctrl+V), me va a quedar algo como **conexion - copia.php**.



- ✓ Renombramos a **conexion_produccion.php** (el nombre no importa mucho, es solo para mí, para no confundirlo con el original).



- ✓ Abrimos con VS Code y reemplazamos el contenido completo por el bloque de arriba.
- ✓ Guardamos.



```

1  <?php
2  /*
3   * -----
4   * TRUEQUE MATCH – conexion.php (PRODUCCIÓN)
5   * Archivo de conexión para el hosting InfinityFree
6   * Gerson Jonnathan López Oviedo | Ficha 3186647
7   * -----
8   */
9
10 // En producción usamos el servidor de InfinityFree, no localhost
11 // Ojo: acá NO va el puerto 3307 pegado con ':' – eso era solo para tu XAMPP local
12 $conexion = mysqli_connect("sql309.infinityfree.com", "if0_42799920", "tyEe0zzjEQ", "if0_42799920_truequematch");
13
14 // Verificamos si la conexión falló
15 if (!$conexion) {
16     die("ERROR: No se pudo conectar a la base de datos. " . mysqli_connect_error());
17 }
18
19 // Configuramos el juego de caracteres a UTF-8
20 mysqli_set_charset($conexion, "utf8");
21 >>

```

*(Ojo: este archivo **conexion_produccion.php** no lo voy a mover a la carpeta real del proyecto ni a hacer git add de él, se queda aparte, listo para subirlo directo a **InfinityFree** con otro nombre en el próximo paso.)*

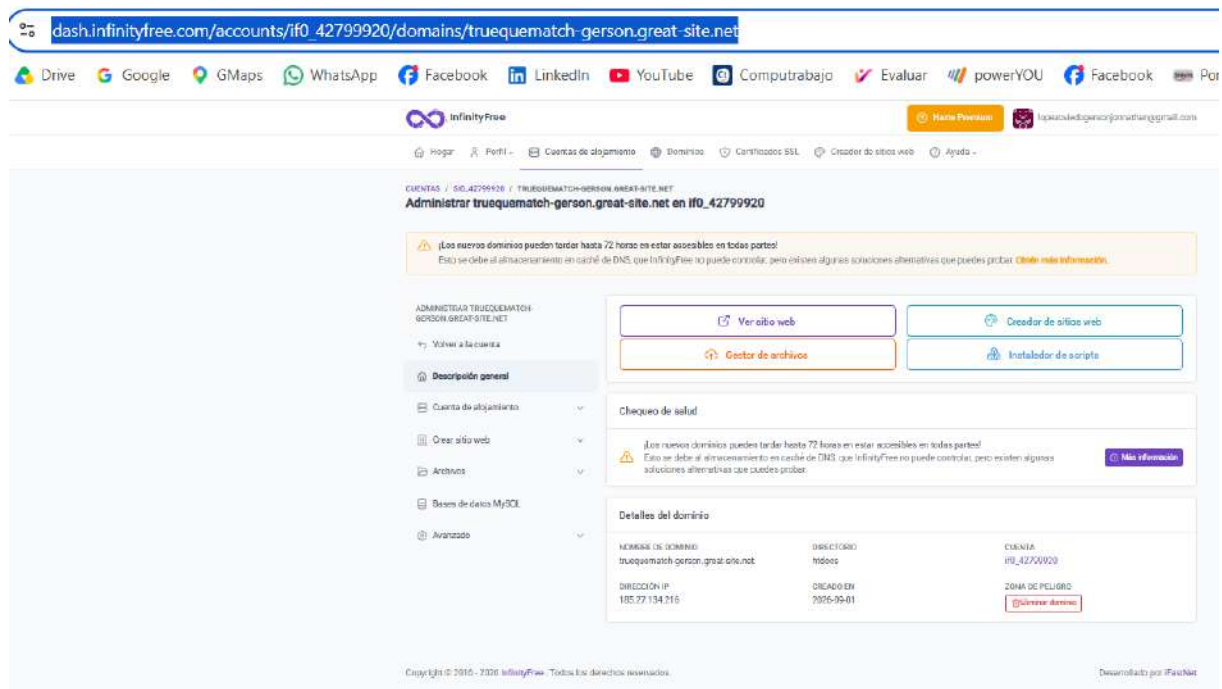
Paso 5: Subir el proyecto completo al hosting

- Volvemos al **Administrador de archivos de InfinityFree** (**filemanager.ai**, el mismo que usaste con **Clearwave**).

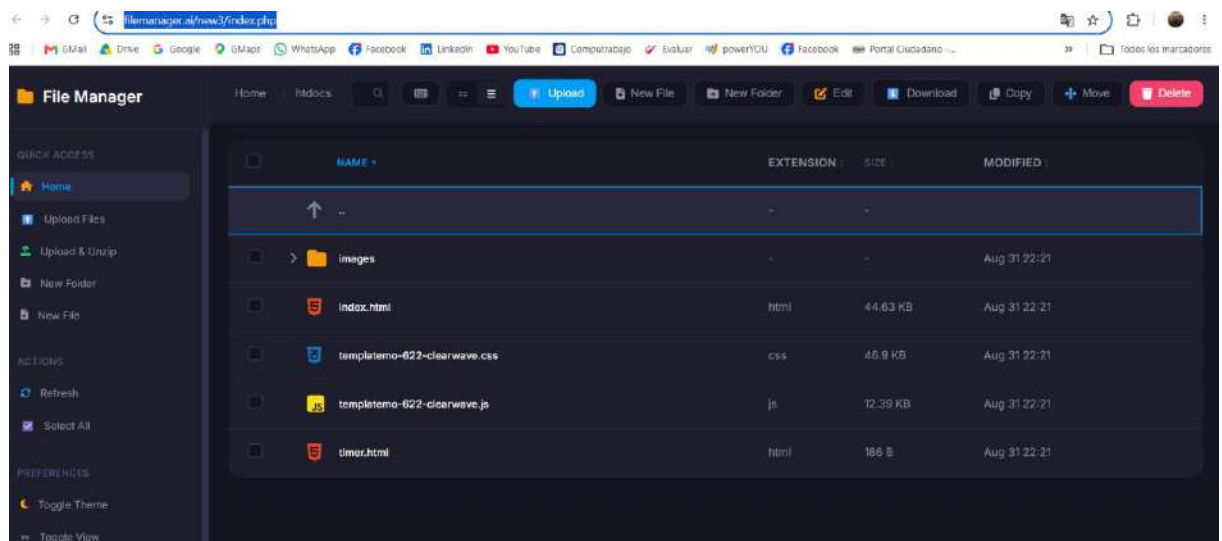
The screenshot shows the InfinityFree account management interface. At the top, there's a navigation bar with links to Home, Profile, Accounts, Domains, SSL Certificates, Website Creator, and Help. The main content area is divided into several sections:

- Left Sidebar:** Contains a list of account management options: Descripción general, Panel de control, Gestor de archivos, Instalador de scripts, Uso de recursos, Detalles de FTP, Bases de datos MySQL, and Historial de desactivación.
- Top Right:** Displays the account name 'if0_42799920' and a warning message about domain accessibility.
- Center:** Features a 'Dominios' section with a search bar and a list of domains. The domain 'truequematch-gerson-great-site.net' is listed with an 'Administrar' button.
- Right Panel:** Contains 'Detalles de la cuenta' (Account Details) with fields for Username, Password, Email, and other account information.

- Hacemos clic en el botón morado de la derecha que dice, **Administrar**.
Ese botón me llevará al panel general de mi cuenta de hosting, donde vere los accesos directos al **File Manager** (Administrador de archivos) y al **Control Panel**.
- Apenas entro a la siguiente pantalla, busco el botón que dice **File Manager** o **Administrador de archivos** para entrar directamente a la carpeta **htdocs**.



- En la pantalla actual vemos el botón con el ícono de nube naranja que dice **Gestor de archivos**. Hacemos clic ahí y se abrirá el administrador de archivos (*File Manager*) en una pestaña nueva.

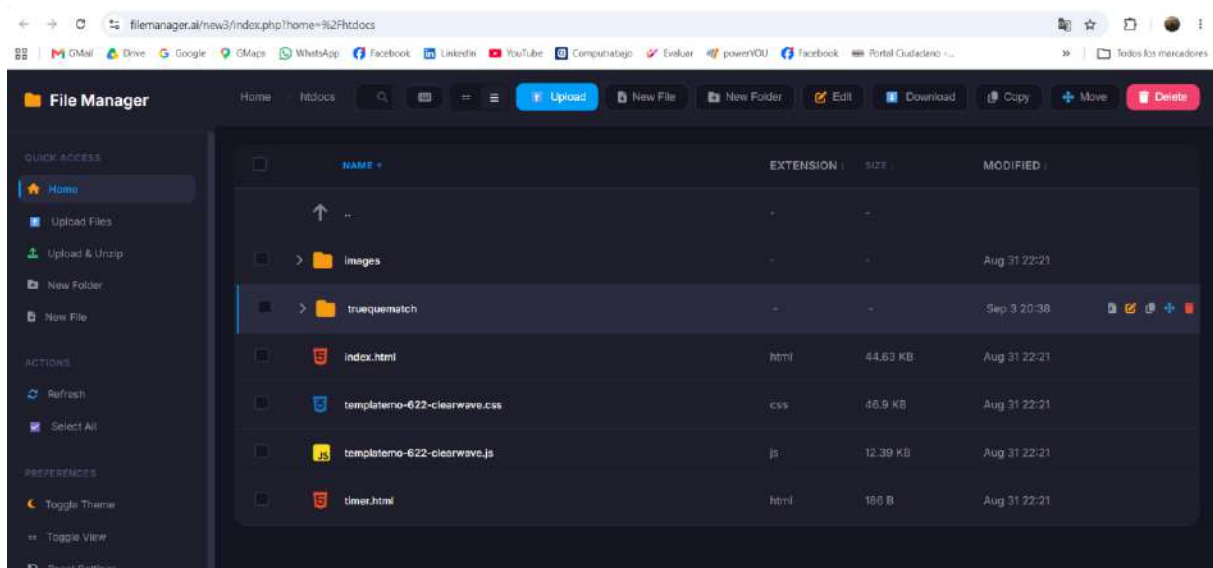


- Creamos una carpeta nueva: **truequematch**.

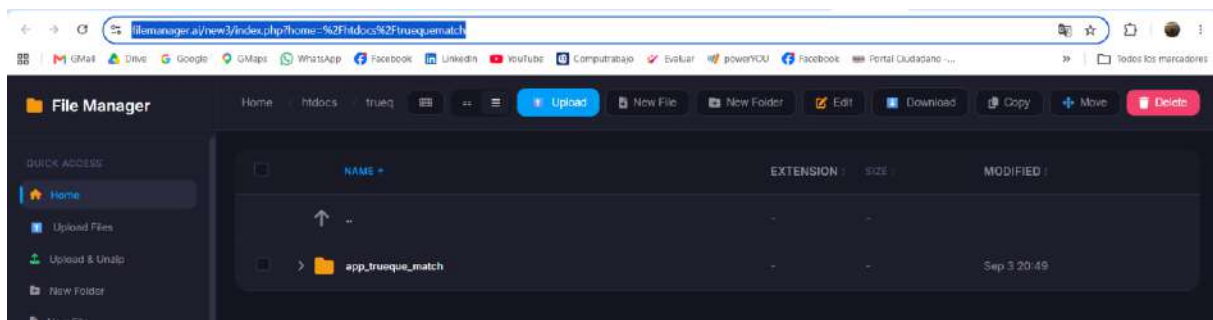
Pasos exactos a seguir dentro del Gestor de archivos:

- ✓ Hacemos doble clic en la carpeta **htdocs** para abrirla.
- ✓ Hacemos clic en **New Folder** (Nueva carpeta) en la barra superior.
- ✓ Escribimos **truequematch** como nombre y le damos en **OK**.

- ✓ Hacemos doble clic sobre la carpeta **truequematch** recién creada para entrar en ella.

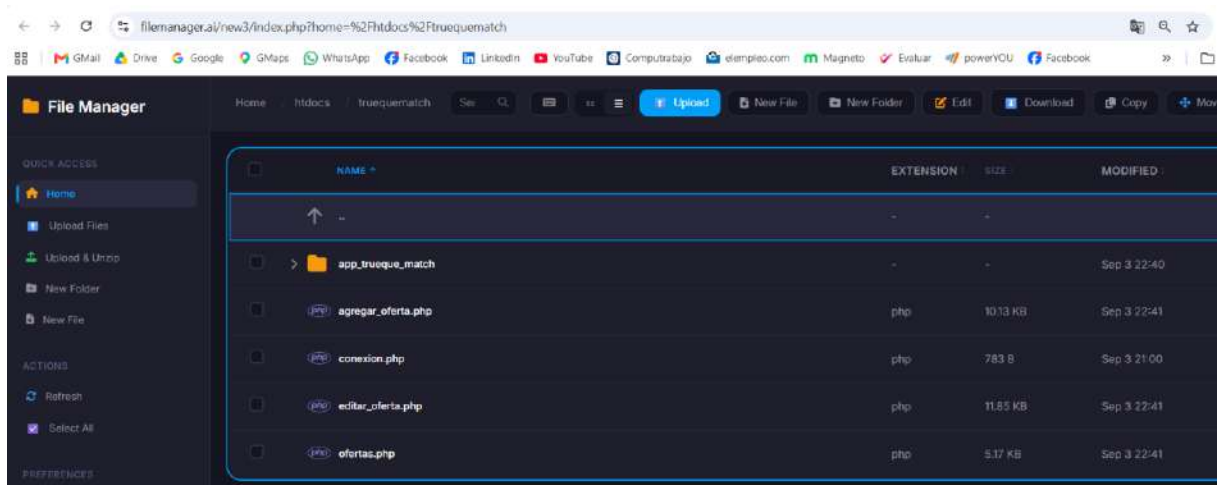


- ✓ Dentro de /htdocs/truequematch/, volvemos a hacer clic en **New Folder**, crea la carpeta **app_trueque_match** y presiona **OK**.

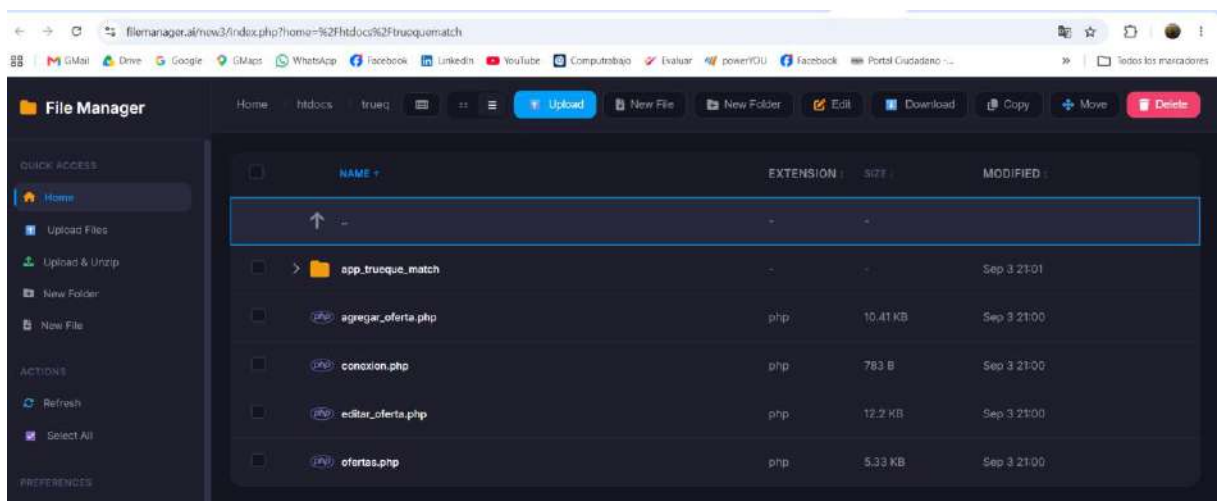


- Entramos a esa carpeta (htdocs/truequematch/).
- Subimos ahí estos archivos **de la raíz del proyecto** (desde mi carpeta local **C:\xampp\htdocs\proyecto_sena_nuevo**):
 - ✓ **agregar_oferta.php**
 - ✓ **editar_oferta.php**
 - ✓ **ofertas.php**
 - ✓ Y el archivo **conexion_produccion.php** que acabamos de mover, pero al subirlo, lo renombramos a **conexion.php** (el administrador de archivos me deja hacer eso, o lo subimos y después usamos "Edit"/renombrar ahí mismo).

*(No hace falta subir **trueque_match_db.sql** ni **README.md**, esos son solo para el repo, no los necesita el servidor para funcionar.)*



- Dentro de htdocs/truequematch/, creá **otra carpeta llamada app_trueque_match**.



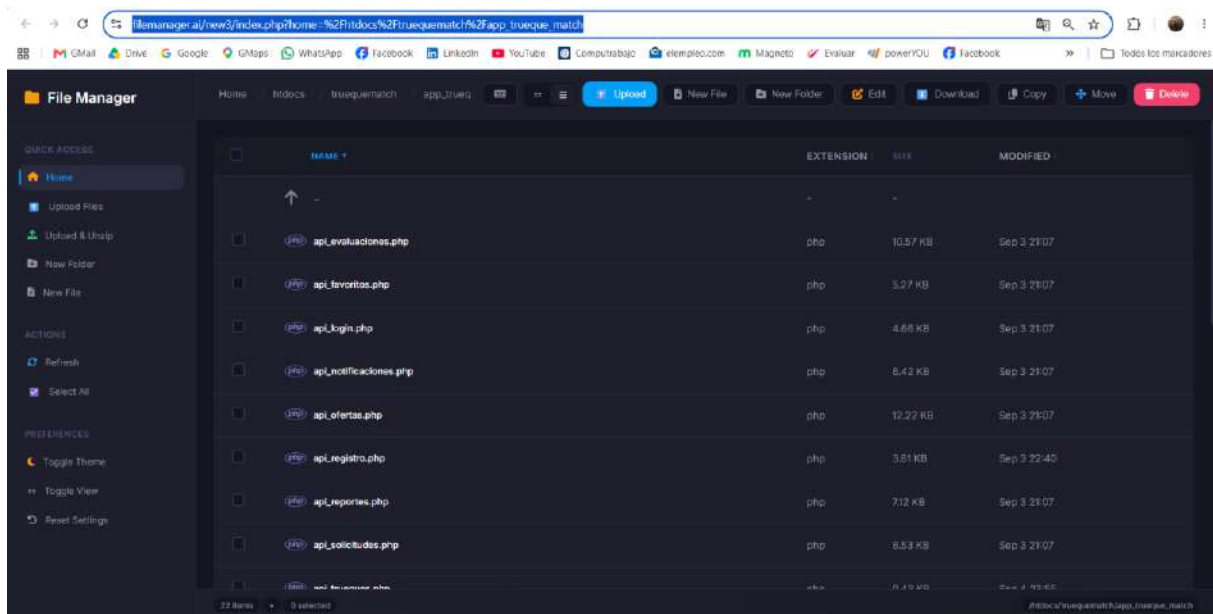
- Entramos ahí y subimos **algunos ítems del contenido** de tu carpeta local **app_trueque_match/** 22 archivos reales de **app_trueque_match/**, ya sin **.gitattributes** (que es un archivo de configuración de Git sin función en el servidor) y sin los mockups eliminados (login.html, registro.html):

#	Archivo	Función
1	index.html	Página de inicio pública
2	como-funciona.html	Página informativa "Cómo funciona"
3	ofertas.html	Listado público de ofertas
4	registro.php	Registro de usuario (real, conectado a BD)
5	login.php	Inicio de sesión (real, conectado a BD)
6	dashboard.php	Panel principal del usuario logueado
7	cerrar_sesion.php	Cierre de sesión
8	guardar_oferta.php	Crear una oferta nueva
9	eliminar_oferta.php	Eliminar una oferta
10	mis_ofertas_ajax.php	Carga dinámica de "Mis Ofertas"
11	api_login.php	API de autenticación
12	api_registro.php	API de registro de usuarios
13	api_ofertas.php	API única de ofertas (GET/POST/PUT/DELETE)
14	api_solicitudes.php	API de solicitudes de trueque
15	api_trueques.php	API de trueques (aceptar/completar/cancelar), <i>el que faltaba</i>
16	api_evaluaciones.php	API de evaluaciones y reputación
17	api_favoritos.php	API de ofertas favoritas
18	api_notificaciones.php	API de notificaciones
19	api_reportes.php	API de reportes de usuarios/ofertas

#	Archivo	Función
20	app.js	Lógica JavaScript del Frontend
21	styles.css	Estilos del sitio
22	LOGO_FINAL.png	Logo de Trueque Match

- Un dato: técnicamente también existe **conexion.php**, pero **no vive dentro de app_trueque_match/**, está un nivel arriba, en la raíz del proyecto (**htdocs/truequematch/conexion.php**), y en InfinityFree es una versión distinta con las credenciales reales de producción, no la misma que en local. Por eso no se contó entre los 22 de esta carpeta.

(Truco: la mayoría de estos administradores de archivos web te dejan seleccionar varios archivos a la vez desde el cuadro de diálogo de "Subir", mantenemos presionado Ctrl y hacemos click en cada uno, o Ctrl+A si sesamos seleccionarlos todos de una carpeta.)

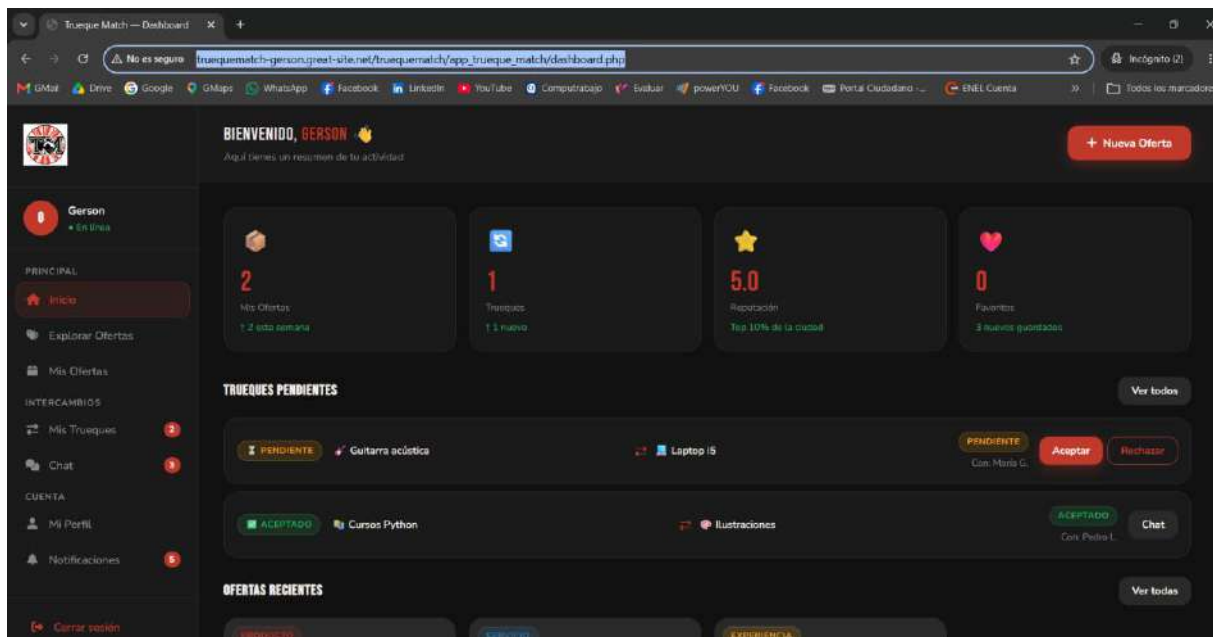


Quedó exactamente como tenía que quedar:

En `htdocs/truequematch/`: `agregar_oferta.php`, `conexion.php` (783 B, justo el tamaño chico que esperábamos del archivo de producción, no el original), `editar_oferta.php`, `ofertas.php`, y la carpeta `app_trueque_match`.

En `htdocs/truequematch/app_trueque_match/`: confirmamos abajo del todo, dice "22 items".

Con esto ya está subido el proyecto completo. Ahora sí, el momento de la verdad.

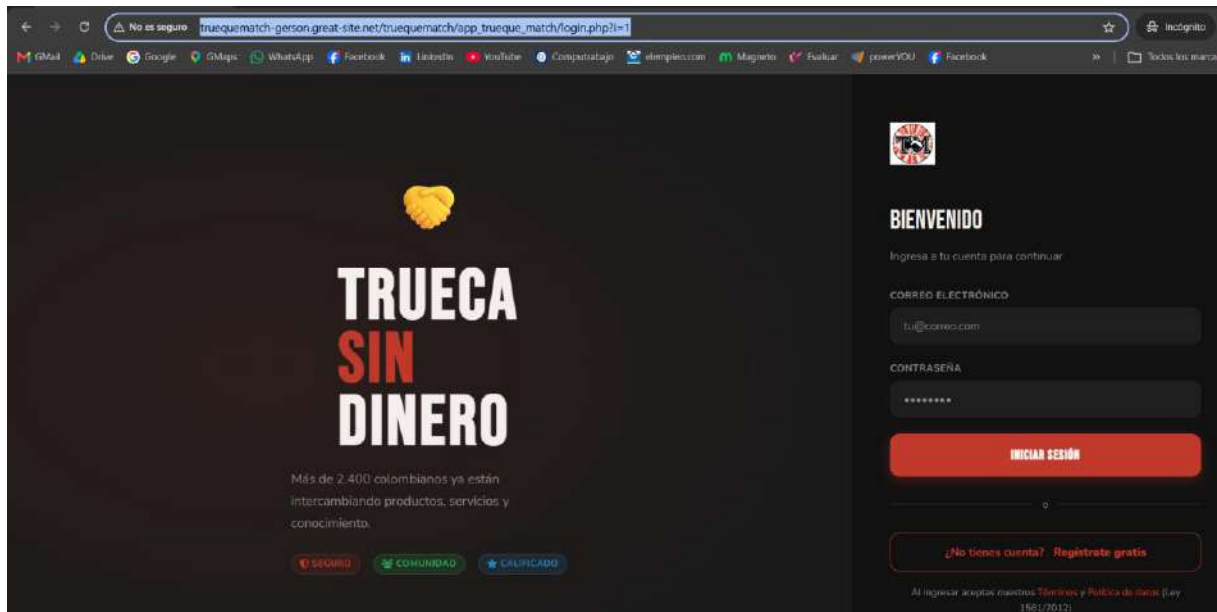


Paso 6: Probar el login en vivo (versión corregida y ya verificada)

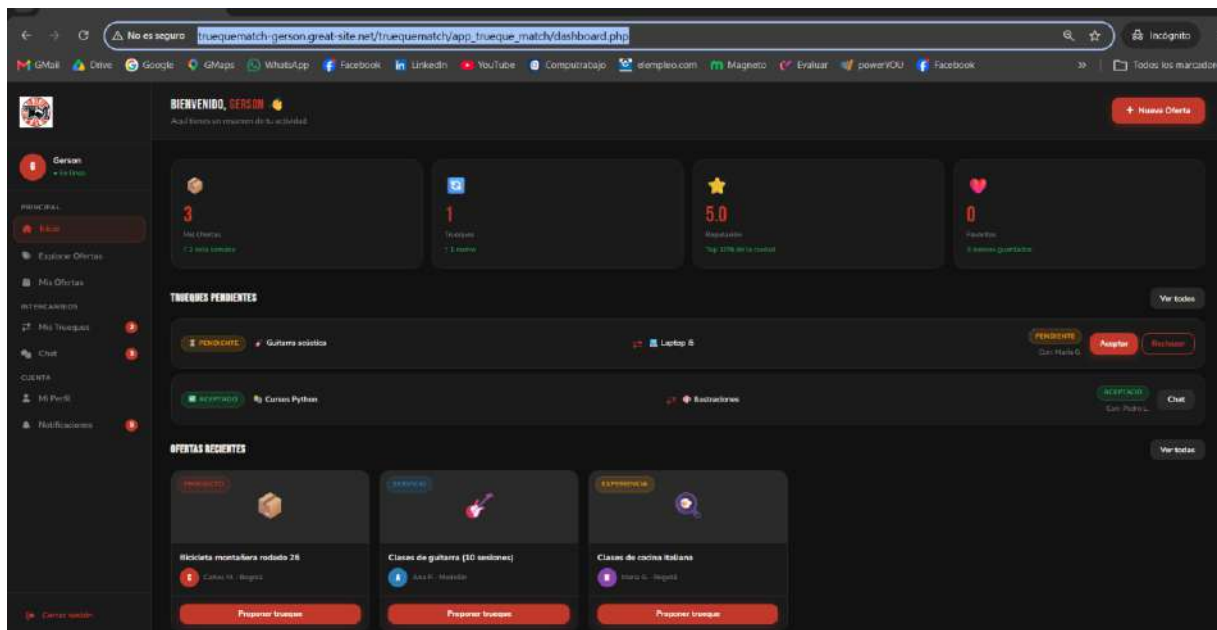
- Abrimos una pestaña nueva (o incógnito).
- No dirigimos a a:

http://truequematch-gerson.great-site.net/truequematch/app_trueque_match/login.php

Es `login.php`, no `login.html`, ese archivo ya no existe, era el mockup del Bug 2.



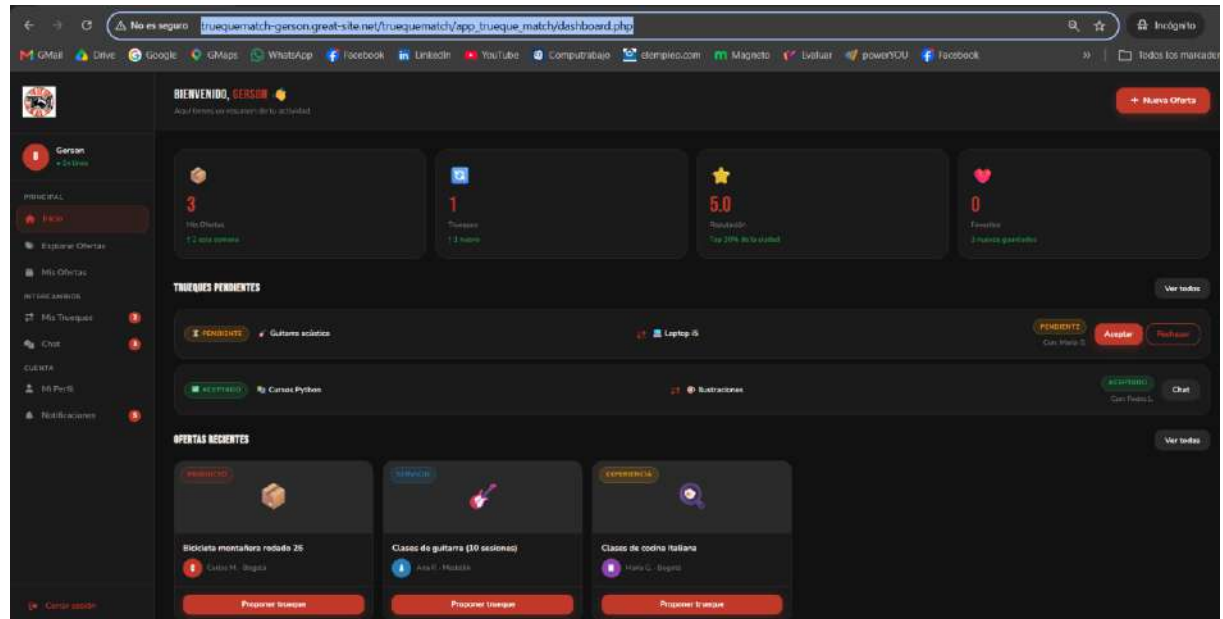
- Iniciamos sesión con tu usuario real: gerson@trueque.com, clave: **Gerson2026**



- **Resultado real, ya confirmado:** entramos directo a **dashboard.php** con "BIENVENIDO GERSON" y datos reales.

Paso 7: Pruebas de funcionalidad:

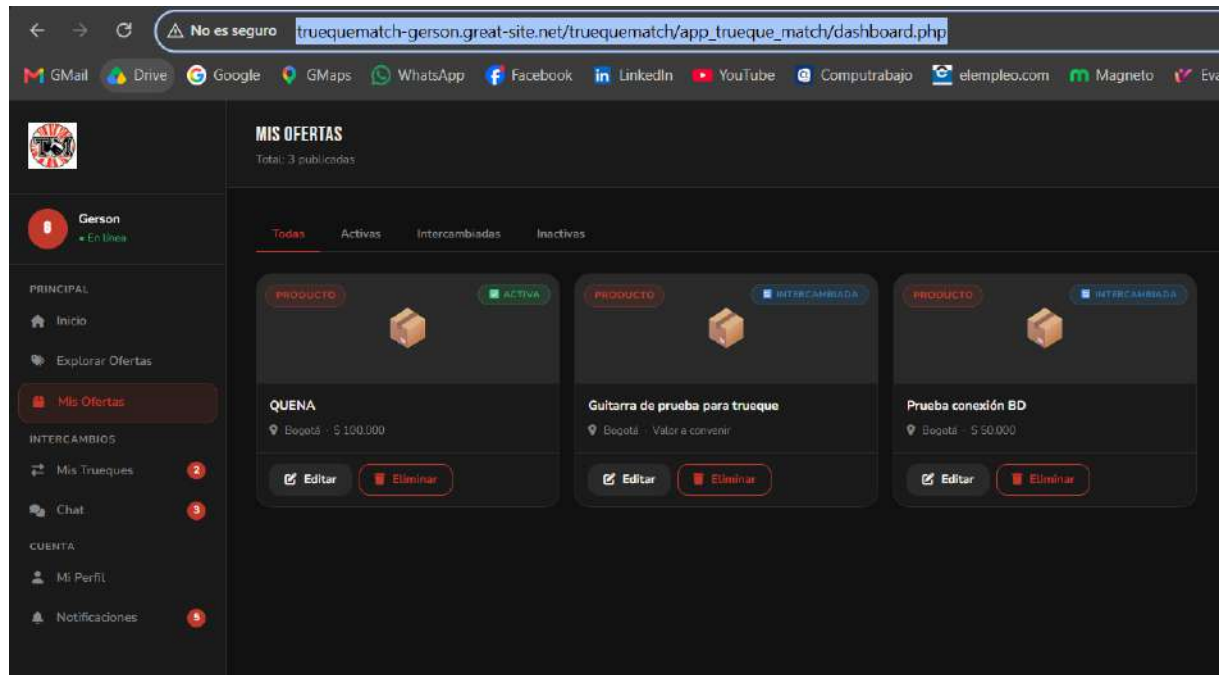
- **Pantalla del dashboard con "BIENVENIDO GERSON" con mis datos reales** como bien decía el plan original, esa es la evidencia más importante de toda la parte de despliegue.



- Probemos también **Mis Ofertas** y **Trueques** desde el **dashboard** en línea, para confirmar que esas dos consultas (las que tocan oferta, trueque, **evaluacion**, **notificacion**) también quedaron bien con el arreglo de minúsculas, total, ya subí todo lo necesario, pero nunca está de más comprobarlo.

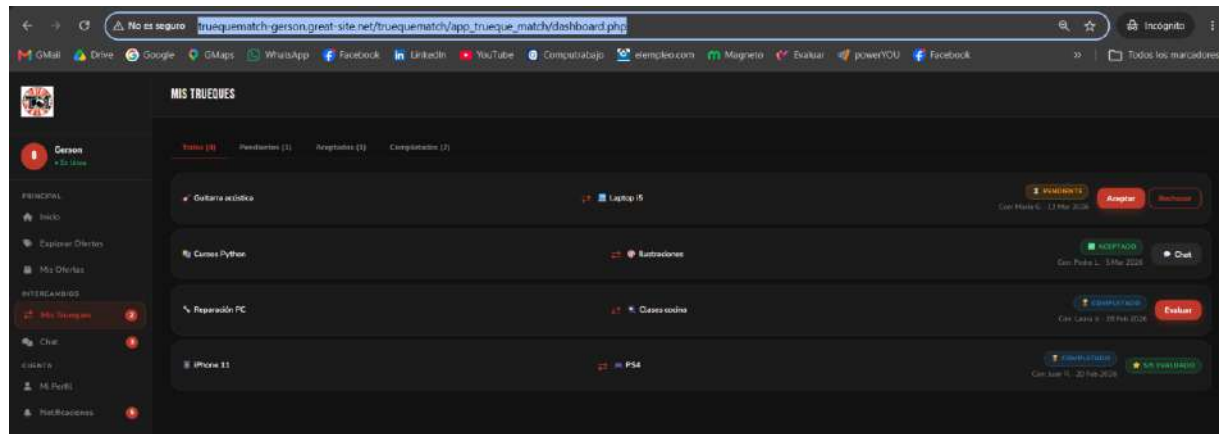
✓ Probar Mis Ofertas:

En el menú lateral, hacemos clic en 'Mis Ofertas'. Esa sección usa [mis_ofertas_ajax.php](#), que consulta la tabla oferta. Si carga la lista de mis ofertas (o un mensaje de 'aún no tienes ofertas'), quedó bien. Si ves un error en rojo o la rueda girando sin parar.



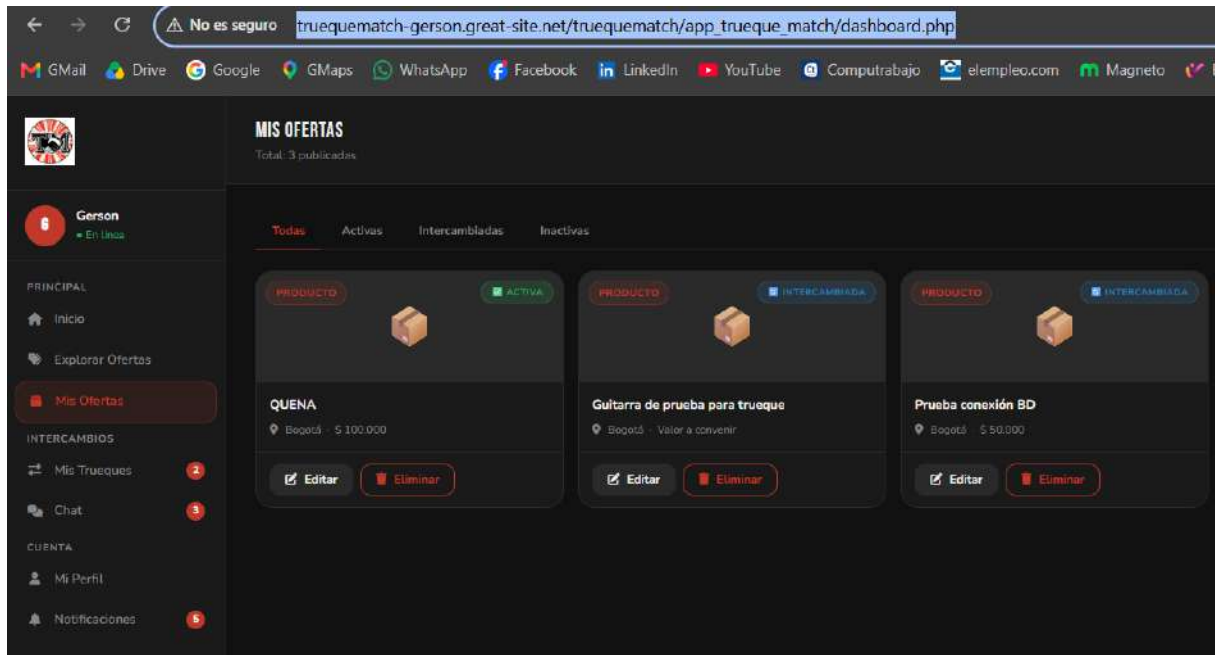
✓ Probar Mis Trueques:

Hacemos clic en 'Mis Trueques'. Esta sección viene ya cargada desde [dashboard.php](#), que consulta trueque, **evaluacion**, **usuario_oferta** y **notificacion**. Revisamos que el contador de arriba ('Trueques', 'Reputación', 'Favoritos') muestre números y no un error.



✓ Probar publicar una oferta:

Como bonus, probamos crear una oferta nueva desde el botón '+ Nueva Oferta' del dashboard. Esa usa [guardar_oferta.php](#). Si te deja publicarla y aparece en la lista, esa API también quedó confirmada.



Se Publico "QUENA" con [guardar_oferta.php](#) (un INSERT real) y apareció al toque en la lista, pasando de 2 a 3 ofertas, eso confirma que no solo lee, también escribe bien en producción.

Paso8: Actualización del README.md con el despliegue a producción

Objetivo: dejar documentado en el repositorio oficial de GitHub que Trueque Match ya está en producción, con el enlace real y el detalle de lo que se verificó.

- **Revisión del README.md actual**

Antes de modificar nada, revisó el contenido completo del archivo directamente desde el repositorio (raw.githubusercontent.com), para partir del contenido real y no de una versión desactualizada en memoria.

- **Identificación de los puntos a actualizar**

Se definieron los apartados que necesitaban reflejar el nuevo estado del proyecto:

- ✓ Un enlace visible a la URL de producción, cerca del encabezado.
- ✓ Una fila nueva en la tabla de Stack Tecnológico para el hosting.
- ✓ Una fila nueva en la tabla de Evidencias SENA para la evidencia de despliegue (GA10).
- ✓ Una sección completa dedicada a explicar el despliegue en producción y lo que se verificó ahí.
- ✓ La actualización del estado general del proyecto al final del documento.

- **Edición directa en GitHub**

Se editó el archivo desde el propio repositorio, usando el botón de edición (ícono de lápiz) sobre README.md, sin necesidad de pasar por Git Bash para este cambio puntual de documentación.

- **Incorporación del contenido nuevo**

Se agregó la línea con la URL de producción, la fila de hosting, la fila de la evidencia GA10, y la sección ". Despliegue en producción" con el detalle de las cuatro validaciones hechas en vivo: **login**, lectura de ofertas, lectura de **trueques/evaluaciones/notificaciones**, y escritura real de una oferta nueva.

- **Actualización de datos adicionales**

Aprovechando la edición, se corrigieron también los nombres de las tablas en la sección de Base de Datos (a minúscula, coherente con la base real), se agregó el enlace del video de Postman que estaba pendiente, y se precisaron los dos dispositivos móviles usados en las pruebas (Redmi 9 y Redmi 10C).

- **Confirmación del cambio**

Se guardó el cambio con un commit descriptivo y se verificó el resultado final contra el archivo real del repositorio, confirmando que todo el contenido nuevo quedó bien reflejado.

- **Resumen de todo lo que quedó bien, confirmado archivo por archivo:**

- ✓ Link de producción arriba del todo
- ✓ Fila de Hosting/Producción en el Stack Tecnológico
- ✓ Evidencia GA10 en la tabla de Evidencias SENA
- ✓ Tablas de la base de datos en minúscula (coherente con el código real)
- ✓ Árbol de estructura del proyecto, dentro de su bloque de código
- ✓ Video de sustentación y video de Postman, ambos con link
- ✓ Sección completa de "Despliegue en producción"
- ✓ Estado del proyecto actualizado
- ✓ Dispositivos de prueba correctos (Redmi 9 de Gerson + Redmi 10C de Sandra)

Paso 9: Trueque Match, en resumen, quedó en producción real:

- ✓ Login, **dashboard**, ofertas y trueques funcionando con datos reales, en Internet
- ✓ Base de datos en **InfinityFree**, sincronizada con el proyecto real

- ✓ 3 hallazgos reales encontrados y corregidos en el camino (mayúsculas, mockups huérfanos, y la lección de que GitHub y el hosting son mundos separados)

Resumen de todo lo que quedó bien, confirmado archivo por archivo:

- Ok. Link de producción arriba del todo
- Ok. Fila de Hosting/Producción en el Stack Tecnológico
- Ok. Evidencia GA10 en la tabla de Evidencias SENA
- Ok. Tablas de la base de datos en minúscula (coherente con el código real)
- Ok. Árbol de estructura del proyecto, dentro de su bloque de código
- Ok. Video de sustentación y video de Postman, ambos con link
- Ok. Sección completa de "Despliegue en producción"
- Ok. Estado del proyecto actualizado
- Ok. Dispositivos de prueba correctos (Redmi 9 de Gerson + Redmi 10C de Sandra)

The screenshot shows a GitHub repository page for 'TRUEQUE MATCH'. The README includes a title, a description of the platform as a Colombian exchange without money, a list of technologies used, and a table of SENA evidence.

TRUEQUE MATCH

Plataforma Colombiana de Intercambio Sin Dinero

Estudiante: Gerson Jonhathan López Oviedo
 Fecha: 31/05/24
 Instructor: Arnaldo Alfonso Montiel Brun
 Programa: Tecnólogo en Análisis y Desarrollo de Software
 Empresa: G.L. Software Solutions
 SENA 2026

En producción: http://truequematch-gerson.github.io:3000/truequematch/app/trueque_match/

Descripción

Trueque Match es una aplicación web y móvil que permite a los colombianos intercambiar productos, servicios, conocimientos y experiencias sin necesidad de dinero. Promueve la economía colaborativa conectando personas que tienen algo que ofrecer con quienes lo necesitan.

Stack Tecnológico

Capa	Tecnología	Estado
Backend Web	PHP 8.2 + MariaDB	Funcional
Frontend Web	HTML5 + CSS3 + JavaScript	Funcional
Base de Datos	MariaDB via XAMPP puerto 3307	Funcional
App Móvil	React Native + Expo (Android)	Probada en Redmi 9 (Gerson) y Redmi 10C (Sandra)
API REST	11 endpoints PHP	Todas probadas en Postman con datos reales
Control de versiones	Git + GitHub	Funcional
IDE	Visual Studio Code	Configurado
Hosting / Producción	InfinityFree	En línea desde septiembre 2026

Evidencias SENA

Evidencia	Descripción	Estado
GA4-220501005-AA1-EV02	Modelo conceptual y lógico del proyecto	✓
GA4-220501005-AA2-EV02	Informe de entregables para el proyecto	✓
GA6-220501006-AA1-EV01	Modelo Relacional + Normalización 1FN/2FN/3FN	✓
GA6-220501006-AA1-EV02	Modelo Entidad Relación (ER/SQL, Workbench)	✓

Enlaces de GitHub del proyecto:

Enlace 1: https://github.com/lopezoviedogersonjonnathan-rgb/proyecto_sena

Enlace 2: https://github.com/lopezoviedogersonjonnathan-rgb/proyecto_sena/tree/main/app_trueque_match

Enlace 3: https://github.com/lopezoviedogersonjonnathan-rgb/proyecto_sena/blob/main/README.md

10. Conclusiones de la evidencia llevar Trueque Match de verdad a Internet:

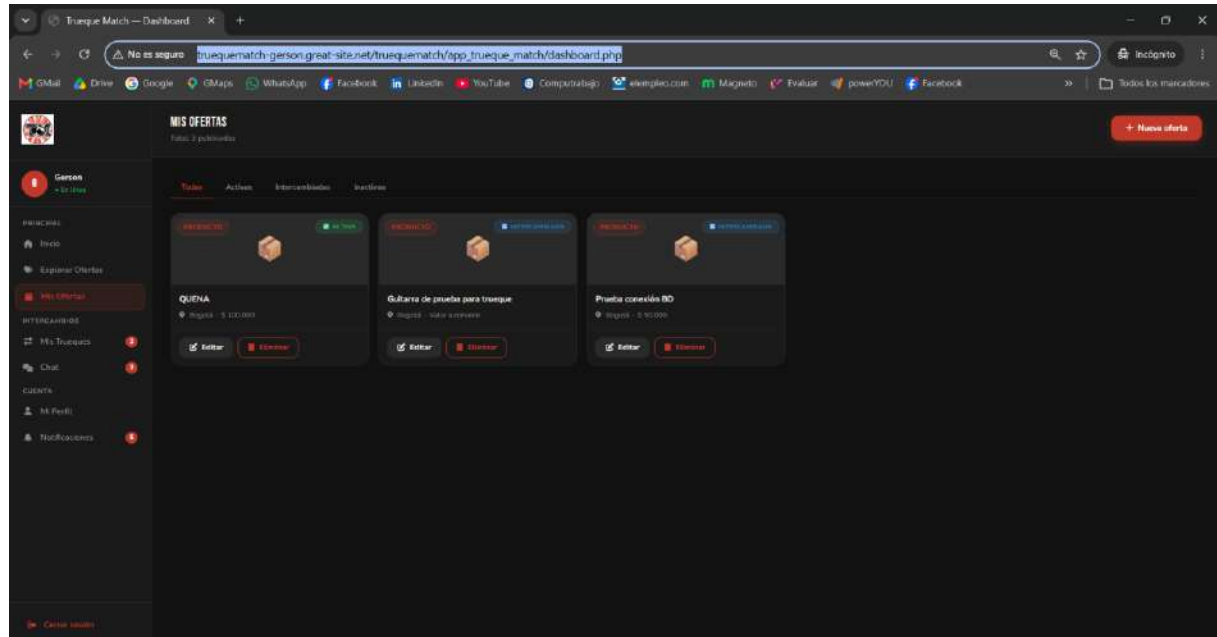
Conclusiones

- **Transición del desarrollo local a producción real.** Se migró la arquitectura de Trueque Match desde un entorno local aislado (XAMPP, puerto 3307) hacia un servidor de producción en la nube (InfinityFree), validando que la aplicación responde correctamente en un entorno público real, no solo en un ambiente controlado.
- **Separación de credenciales entre entornos.** El archivo de conexión de producción se mantiene completamente fuera del repositorio público de GitHub (nunca se llamó ni se subió como `conexion.php` genérico allí), mientras que el `conexion.php` del repo solo contiene las credenciales locales de XAMPP. Esta separación, reforzada con una regla en `.gitignore`, protege las credenciales reales de la base de datos contra cualquier acceso no autorizado.
- **Resolución de una incompatibilidad real entre sistemas operativos.** El despliegue sobre un servidor Linux permitió detectar que el código PHP hacía referencia a las tablas de la base de datos en mayúscula (OFERTA, USUARIO, etc.). Windows nunca marcó error porque no distingue mayúsculas de minúsculas en estos nombres; Linux sí. La corrección de esas referencias en 10 archivos no de las tablas en sí, que ya estaban bien nombradas desde su creación, garantizó la compatibilidad total con el motor de base de datos remoto.
- **Detección de una segunda falla, independiente de la primera: interfaces sin conectar.** Se encontró que `login.html` y `registro.html` eran maquetas que simulaban el inicio de sesión y el registro únicamente con JavaScript, sin comunicarse nunca con la base de datos, y estaban enlazadas desde 17 puntos distintos del sitio. Se corrigieron los enlaces hacia las versiones reales (`login.php`, `registro.php`) y se eliminaron las maquetas del repositorio.

- **La sincronización entre GitHub y el hosting no es automática.** Se comprobó en la práctica que actualizar el repositorio con git push no actualiza el sitio en InfinityFree: cada corrección debe subirse aparte, manualmente, a través del Administrador de Archivos. Este hallazgo llevó a una falla real (el mismo bug de mayúsculas volvió a aparecer en producción después de corregido en el código) y quedó documentado como una lección de proceso, no solo de código.
- **Verificación transaccional completa, en vivo.** Las pruebas confirmaron que la aplicación en producción conserva la integridad de sus funciones: autenticación real (login.php), consulta de dashboard dinámico, lectura AJAX de ofertas y trueques, y creación real de un registro nuevo mediante INSERT (comprobado con la publicación de la oferta "QUENA").
- **Documentación abierta y trazable en GitHub.** La actualización del README.md consolida la trazabilidad del proyecto: stack tecnológico completo, evidencias superadas (incluyendo GA10), enlaces a los videos explicativos, estructura de archivos y la URL oficial de producción, permitiendo que el repositorio funcione como un portafolio de software real y verificable.

Enlaces del proyecto:

- Producción: http://truequematch-gerson.great-site.net/truequematch/app_trueque_match/
- Repositorio: https://github.com/lopezoviedogersonjonnathan-rgb/proyecto_sena
- Video de sustentación: <https://youtu.be/Kis8zE8BeT4?si=l61Aqz02DDFPFNM7>
- Video de pruebas Postman: <https://youtu.be/Klnpu9v9o8Y?si=G9hOcj40Blfm2daB>



2. Evidencias tenidas en cuenta para elaborar el manual técnico.

2.6. Diagrama de componentes

Este tema se desarrolló en la evidencia:

Evidencia GA4-220501095-AA2-EV05. Desarrollar la arquitectura de software, de acuerdo con el patrón de diseño seleccionado.

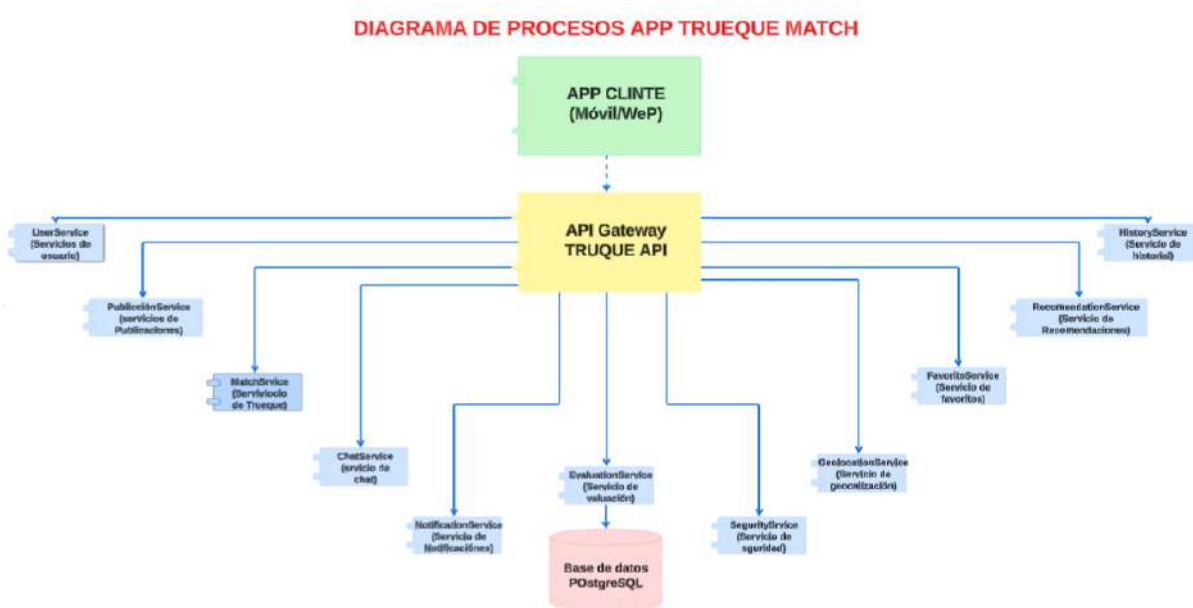


Diagrama de componentes actualizado

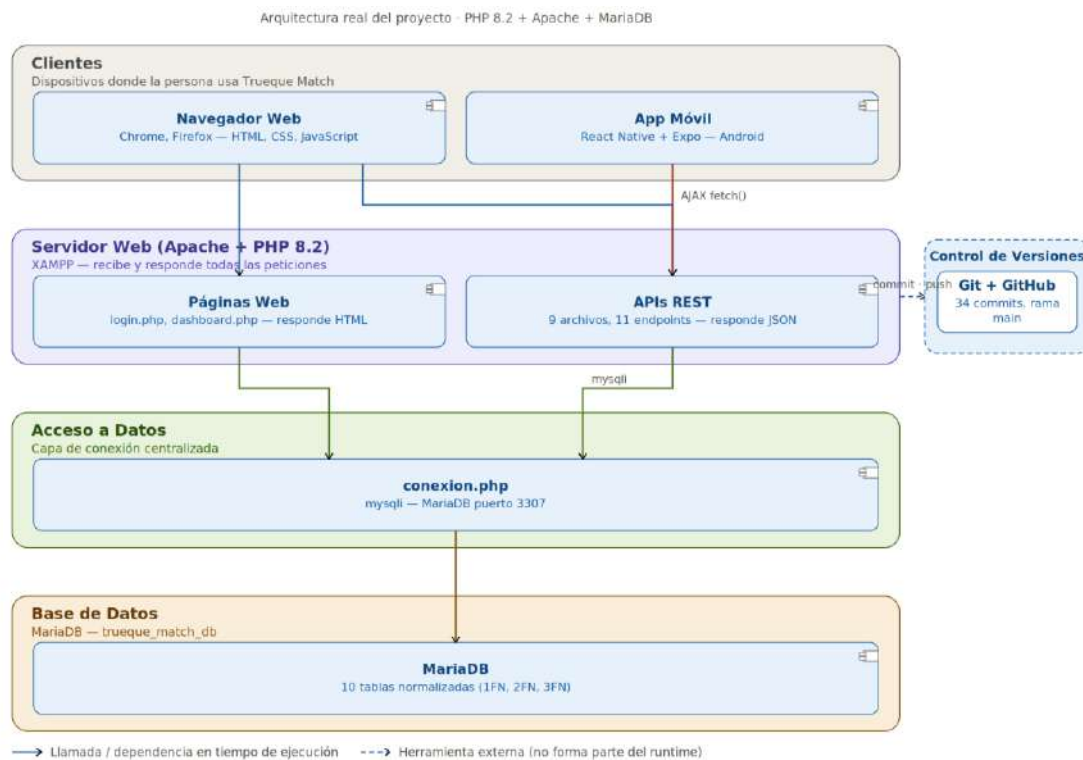


Diagrama de componentes e identificación de capas

El diagrama de componentes muestra, a grandes rasgos, las piezas grandes de todo el sistema Trueque Match y cómo se comunican entre sí. Lo incluyo de nuevo aquí porque es clave para entender en qué punto exacto se conecta el módulo móvil con el resto del proyecto: la app Android no habla directamente con la base de datos, siempre pasa por las APIs REST del servidor.

Figura 1. Diagrama de componentes de Trueque Match, con la app móvil como uno de los dos clientes que consumen las APIs REST.

Un ejemplo para entenderlo: pensemos en un restaurante. El navegador web y la app móvil son como dos ventanillas distintas de pedidos (el mesero de mesa y el domiciliario), pero ambos gritan el pedido a la misma cocina (las APIs REST), y esa cocina es la única que puede sacar los ingredientes de la despensa (la base de datos). Ninguna ventanilla entra directo a la despensa a servirse sola.

Con este diagrama identifico claramente en qué capa vive cada componente de la app móvil:

Capa	Dónde vive en la app móvil
Presentación	Las pantallas de React Native (Login, Home, Trueques, Chat, Perfil) y los componentes visuales (botones, tarjetas).
Lógica de negocio	No vive en el celular: vive en las APIs REST de PHP en el servidor. El celular solo pide y muestra resultados.
Acceso a datos	Tampoco vive en el celular: vive en conexion.php, en el servidor, usando mysqli.
Datos	La base de datos MariaDB, en el mismo servidor, nunca directamente accesible desde el celular.

Esta separación es intencional y es una de las decisiones de arquitectura más importantes del módulo móvil: la app Android es 'tonta' a propósito, en el buen sentido. No guarda reglas de negocio ni toca la base de datos directamente. Si mañana cambio de motor de base de datos o reescribo el backend en otro lenguaje, la app móvil no debería enterarse, porque solo conoce las APIs REST, no lo que hay detrás de ellas.

Conclusiones

Con este manual técnico dejo reunidos, en un solo documento, los 7 elementos que sustentan cómo está construido Trueque Match: desde qué se necesita para instalarlo, hasta cómo están organizados sus componentes y su base de datos. La mayoría de este contenido ya existía repartido en evidencias anteriores del proceso formativo, aquí lo organicé, lo verifiqué contra el código y la base de datos reales, y corregí las partes que ya no coincidían con el estado actual del proyecto (como el diccionario de datos, que en su versión anterior era en realidad un glosario de conceptos y no una descripción real de las tablas).